

IoT-Based Smart Irrigation System for Optimized Water Distribution



Project ID: Fall-2025-03

Session: BSCS Fall 2022 to 2026

Project Advisor: Mr. Hammad Hassan

Submitted By

Shoaib Khan	70135056
Muhammad Ahmad	70135149
Muhammad Tayyab	70139803

Department of Computer Science &
IT The University of Lahore
Lahore, Pakistan

Declaration

We have read the project guidelines and we understand the meaning of academic dishonesty, in particular plagiarism and collusion. We hereby declare that the work we submitted for our final year project, entitled **IoT-Based Smart Irrigation System for Optimized Water Distribution** is original work and has not been printed, published or submitted before as final year project, research work, publication or any other documentation.

Group Member 1 Name: Shoaib Khan
SAP No: 70135056

Signature:

Group Member 2 Name: Muhammad Ahmad
SAP No: 70135149

Signature:

Group Member 3 Name: Muhammad Tayyab
SAP No: 70139803

Signature:

Statement of Submission

This is to certify that **Shoaib Khan** Roll No. **70135056**, **Muhammad Ahmad** Roll No. **70135149** and **Muhammad Tayyab** Roll No. **70139803** have successfully submitted the final project named as: **IoT-Based Smart Irrigation System for Optimized Water Distribution**, at Computer Science & IT Department, The University of Lahore, Lahore Pakistan, to fulfill the partial requirement of the degree of **BS in Computer Science**.

Supervisor Name: Mr. Hammad Hassan

Signature:

Date:

Dedication

This project is dedicated to my respected father and mother, whose guidance, patience, and unwavering support laid the foundation for my academic journey. Their values of continuous learning, discipline, and perseverance have been a constant source of motivation. I also sincerely dedicate this work to my supervisor, Mr. Hammad Hassan, for his professional guidance, technical insights, and consistent encouragement throughout this project.

Acknowledgement

We sincerely acknowledge the cooperation and valuable support provided by Mr. Hammad Hassan, Assistant Professor (UOL) Supervisor, Dr Salman Rashid, Dir IT (LGU) whose technical guidance and continuous mentoring played a crucial role in the successful completion of this project. His expertise, constructive feedback, and encouragement remained a constant source of motivation throughout the project lifecycle.

We are also grateful to our friends and families for their patience, moral support, and understanding, which greatly contributed to the timely completion of this work.

Date:
June 4, 2026

Abstract

Agriculture plays a vital role in Pakistan's economy; however, inefficient irrigation practices and increasing water scarcity have significantly impacted crop productivity and resource sustainability. Traditional irrigation methods rely heavily on manual control and outdated techniques, resulting in excessive water wastage, uneven distribution, and increased labor dependency. Climate change has further intensified these challenges by introducing unpredictable rainfall patterns and prolonged drought conditions, making efficient water management a critical necessity. This project presents an IoT-based Smart Irrigation System designed to address these issues by integrating real-time soil moisture sensing, environmental monitoring, and weather-aware automation. The proposed system continuously collects sensor data and external weather information to intelligently control irrigation processes. Automated decision-making ensures water is supplied only when required, while a web-based dashboard enables remote monitoring and system management for both farmers and administrators. The implementation of this system significantly reduces water consumption, improves crop yield, minimizes human intervention, and supports data-driven agricultural practices. By offering a cost-effective, scalable, and locally adaptable solution, the project contributes to sustainable agriculture, enhanced water conservation, and improved livelihoods for farming communities.

Area of the Project

The project falls within the domain of Internet of Things (IoT), Smart Agriculture, and Embedded Systems, integrating hardware and software technologies for intelligent resource management. It involves real-time sensor data acquisition, automated control systems, and cloud-based monitoring to optimize irrigation processes. The system utilizes through network connectivity, data analytics, and decision-support algorithms to enhance agricultural efficiency. Additionally, it incorporates web technologies and database management systems for remote access and data persistence. The project also addresses sustainable water management and climate-aware automation in precision farming environments.

Technologies used

The project utilizes Internet of Things (IoT) technology for real-time sensing and automated irrigation control. Microcontroller-based embedded systems (Arduino with Ethernet interface) are used for data acquisition and actuator management. Web technologies (HTML, CSS, JavaScript, and RESTful APIs) enable remote monitoring and system interaction. Database management systems (MySQL/SQL-based) are employed for secure data storage and logging. Weather APIs provide real-time and forecasted climate data to support intelligent decision-making. Additionally, networking protocols such as HTTP and MQTT facilitate reliable communication between system components.

Table of Content

Dedication	3
Acknowledgement	4
Abstract	5
Area of the Project.....	6
Technologies used	6
Table of Content.....	7
List of Figures.....	13
List of Tables.....	15
Chapter 1: Introduction to the Problem	1
1.1 Introduction	1
1.2 Purpose	2
1.3 Objectives	2
1.4 Existing Solutions	3
1.5 Proposed Solution.....	4
Chapter 2: Software Requirement Specification	6
2.1 Introduction	6
2.1.1 Purpose	6
2.1.1.1 Intended Audience	6
2.1.2 Scope.....	6
2.1.2.1 The System Will	6
2.1.2.2 The System Will NOT	7
2.1.2.3 Application of the System.....	7
2.1.3 Definitions, Acronyms, and Abbreviations 2.1.3.1 Definitions	7
2.1.3.2 Acronyms.....	7
2.1.3.3 Abbreviations.....	7
2.2 Overall Description.....	8
2.2.1 Product Perspective	8
2.2.1.1 System Interfaces	8
2.2.1.2 User Interfaces	8

2.2.1.3	Hardware Interfaces	8
2.2.1.4	Software Interfaces.....	8
2.2.1.5	Communication Interfaces.....	8
2.2.1.6	Memory Requirements.....	9
2.2.1.7	Operations	9
2.2.2	Product Functions	9
2.2.2.1	Basic Workflow	9
2.2.3	UserCharacteristics 2.2.3.1 Intended Users	9
2.2.3.2	User Skills.....	9
2.2.3.3	Educational Background	9
2.2.4	Constraints	9
2.2.5	Assumptions and Dependencies	10
2.2.6	Apportioning of Requirements.....	10
2.3	Specific Requirements.....	10
2.3.2	Non-functional Requirements.....	15
Chapter 3: Use Case Analysis		17
3.1	User Registration.....	17
3.2	User Login	18
3.3	View Dashboard	18
3.4	Manage Field.....	19
3.5	Manage Zone	20
3.5	Select Crop Type.....	21
3.6	Monitor Sensor Data	21
3.7	Fetch Weather Data	22
3.8	Automatic Irrigation Control	23
3.9	Manual Irrigation Control.....	24
3.10	Zone-Based Irrigation	24
3.11	Irrigation Scheduling	25
3.12	Log Irrigation Activity	26
3.13	Generate Reports & Analytics	26
3.14	Alerts & Notification.....	27
3.15	Device Health Monitoring	28

3.16	System Configuration	28
3.17	IOT Controller Setup	29
3.18	Data Backup & Restore	30
3.19	Logout	30
	Chapter 4: Design.....	32
4.1	Architecture Diagram	32
4.2	ERD with data dictionary	34
4.3	Data Flow diagram	35
4.3.2	The level 1	35
4.4	Class Diagram	38
4.5	Activity Diagram	40
4.5.2	User Registration.....	40
4.5.3	User Login	41
4.5.4	View Dashboard	41
4.5.5	Manage Field.....	42
4.5.6	Manage Zone	42
4.5.7	Select Crop Type.....	43
4.5.8	Monitor Sensor Data	43
4.5.9	Fetch Weather Data	44
4.5.10	Automatic Irrigation Control	44
4.5.11	Manual Irrigation Control	45
4.5.12	Zone-Based Irrigation.....	45
4.5.13	Irrigation Scheduling	46
4.5.14	Log Irrigation Activity.....	46
4.5.15	Generate Reports & Analytics.....	47
4.5.16	Alerts & Notification	47
4.5.17	Device Health Monitoring.....	48
4.5.18	System Configuration.....	48
4.5.19	IOT Controller Setup	49
4.5.20	Data Backup & Restore	49
4.5.21	Logout.....	50
4.6	Sequence Diagram	52

4.7	Collaboration Diagram	53
4.8	State Transition Diagram.....	54
4.9	Component Diagram.....	55
4.10	Deployment Diagram	57
	Chapter 5: Testing	58
5.1	Test Case Descriptions	58
A.	Test Case 1: FR_01 – Create Account	58
B.	Test Case 2: FR_02 – Login	59
C.	Test Case 3: FR_03 – View Dashboard	60
D.	Test Case 4: FR_04 – Manage Field	61
E.	Test Case 5: FR_05 – Select Crop Type.....	62
F.	Test Case 6: FR_06 – Read Sensor Data	63
H.	Test Case 8: FR_08 – Automatic Irrigation	65
I.	Test Case 9: FR_09 – Manual Irrigation Control	66
J.	Test Case 10: FR_10 – Irrigation Scheduling	67
K.	Test Case 11: FR_11 – Activity Logging	68
L.	Test Case 12: FR_12 – Generate Reports.....	69
M.	Test Case 13: FR_13 – Notifications	70
N.	Test Case 14: FR_14 – Device Health Monitoring	71
O.	Test Case 15: FR_15 – System Configuration	71
P.	Test Case 16: FR_16 – IoT Controller Setup	72
Q.	Test Case 17: FR_17 – Backup and Restore	73
R.	Test Case 18: FR_18 – Logout	74
5.2	Black Box Test Cases.....	75
5.2.1	Equivalence Partitions (EP).....	75
5.2.2	Boundary Value Analysis	76
A.	Soil-moisture lower threshold (start trigger)	76
B.	Soil-moisture upper threshold (stop trigger)	77
C.	Rain-skip probability threshold	77
D.	Irrigation duration (manual / schedule)	77
E.	Other bounded inputs (summary).....	77
5.2.3	Decision Table Testing	78

Decision Table 1 – Per-zone irrigation decision (FR_08).....	78
Decision Table 2 – Login authentication (FR_02)	78
5.2.4 State Transition Testing.....	78
A. Irrigation-event lifecycle	78
Invalid-transition tests.....	79
B. Device status lifecycle	79
5.2.5 Use Case Testing	79
5.3 White Box Test Cases	80
5.3.1 Cyclomatic Complexity	80
Complexity of other key modules	81
5.4 Performance Testing	81
5.5 Stress Testing	81
5.6 System Testing	82
5.7 Regression Testing	82
5.7.1 Selecting Regression Tests	82
5.7.2 Regression Testing Steps.....	83
Chapter 6: Tools and Techniques	84
6.1 Languages.....	84
6.2 Frameworks, Libraries and Runtime.....	84
6.3 External Services and Hardware.....	84
6.4 Development Tools	84
6.5 Engineering Techniques	85
Chapter 7: Summary and Conclusion	86
Chapter 8: User Manual	87
8.1 Logging In	87
8.2 The Overview Dashboard	87
8.3 Managing Fields, Zones and Crops	88
8.4 Registering an IoT Device	88
8.5 Manual Irrigation.....	89
8.6 Scheduling Irrigation	89
8.7 Viewing Alerts	89
8.8 Generating Reports	89

8.9 System Settings (Admin)	90
8.10 Backup and Restore (Admin).....	90
8.11 Logging Out	91
Chapter 9: Lessons Learnt and Future Work.....	92
9.1 Lessons Learnt.....	92
9.2 Future Work	92
Appendix	93
A REST API Endpoint Reference	93
B System Configuration Defaults.....	93
C Functional Requirement → Source Code Mapping.....	93
D Database Tables.....	94
References	95

List of Figures

Figure 1 Use case Diagram User Registration.....	17
Figure 2 Usecase Diagram User Login.....	18
Figure 3 Usecase Diagram View Dashboard.....	18
Figure 4 Usecase Diagram Manage Field.....	19
Figure 5 Usecase Diagram Manage Zone.....	20
Figure 6 Usecase Diagram Select Crop Type.....	21
Figure 7 Usecase Diagram Monitor Sensor Data	21
Figure 8 Usecase Diagram Fetch Weather Data.....	22
Figure 9 Usecase Diagram Automatic Irrigation Control.....	23
Figure 10 Usecase Diagram Manual Irrigation Control	24
Figure 11 Usecase Diagram Zone-Based Irrigation	24
Figure 12 Usecase Diagram Irrigation Scheduling.....	25
Figure 13 Usecase Diagram Log Irrigation Activity	26
Figure 14 Usecase Diagram Generate Reports & Analytics.....	26
Figure 15 Usecase Diagram Alerts Notification.....	27
Figure 16 Usecase Diagram Device Health Monitoring.....	28
Figure 17 Usecase Diagram System Configuration.....	28
Figure 18 Usecase Diagram IOT Controller Setup.....	29
Figure 19 Usecase Diagram Data Backup & Restore.....	30
Figure 20 Usecase Diagram Logout	30
Figure 21 Architecture Diagram.....	33
Figure 22 ERD with data dictionary.....	34
Figure 23 Level 0 DFD	35
Figure 24 Level 1 DFD	37
Figure 25 Class Diagram.....	39
Figure 26 Activity Diagram User Registration.....	40
Figure 27 Activity Diagram User Login.....	41
Figure 28 Activity Diagram View Dashboard.....	41
Figure 29 Activity Diagram Manage Field.....	42
Figure 30 Activity Diagram Manage Zone.....	42
Figure 31 Activity Diagram Select Crop.....	43
Figure 32 Activity Diagram Monitor Sensor Data	43
Figure 33 Activity Diagram Fetch Weather Data.....	44
Figure 34 Activity Diagram Automatic Irrigation.....	44
Figure 35 Activity Diagram Manual Irrigation Control	45
Figure 36 Activity Diagram Zone-Based Irrigation.....	45
Figure 37 Activity Diagram Irrigation Scheduling.....	46
Figure 38 Activity Diagram Log Irrigation	46
Figure 39 Activity Diagram Generate Report	47
Figure 40 Activity Diagram Alert Notification	47
Figure 41 Activity Diagram Device Health Monitoring.....	48
Figure 42 Activity Diagram System Configuration.....	48
Figure 43 Activity Diagram IOT Controller Setup.....	49
Figure 44 Activity Diagram Data Backup.....	49
Figure 45 Activity Diagram Logout	50
Figure 46 Activity Diagram Create	51
Figure 47 Sequence Diagram	52
Figure 48 Collaboration Diagram.....	54
Figure 49 State Transition Diagram	55

Figure 50 Component Diagram.....	56
Figure 51 Deployment Diagram.....	57
Figure 52 Logging In.....	87
Figure 53 Overview Dashboard.....	88
Figure 54 Managing Fields, Zones and Crops.....	88
Figure 55 Registering an IoT Device	89
Figure 56 Scheduling Irrigation.....	89
Figure 57 Generating Reports	90
Figure 58 System Settings (Admin)	90
Figure 59 Backup and Restore (Admin).....	91
Figure 60 Logging Out.....	91

List of Tables

Table 1 Create Account.....	10
Table 2 Login	11
Table 3 View Dashboard.....	11
Table 4 Manage Field.....	11
Table 5 Select Crop Type.....	12
Table 6 Read Sensor Data	12
Table 7 Fetch Weather Data.....	12
Table 8 Automatic Irrigation	12
Table 9 Manual Irrigation Control.....	13
Table 10 Irrigation Scheduling.....	13
Table 11 Activity Logging	13
Table 12 Generate Reports.....	14
Table 13 Notifications	14
Table 14 Device Health Monitoring.....	14
Table 15 System Configuration.....	14
Table 16 IoT Controller Setup.....	15
Table 17 Backup and Restore.....	15
Table 18 Logout	15
Table 19 User Registration.....	17
Table 20 Basic Flow.....	17
Table 21 Alternate Flow.....	17
Table 22 User Login.....	18
Table 23 Basic Flow.....	18
Table 24 Alternate Flow.....	18
Table 25 View Dashboard.....	19
Table 26 Basic Flow.....	19
Table 27 Alternate Flow.....	19
Table 28 Manage Field.....	19
Table 29 Basic Flow.....	20
Table 30 Alternate Flow.....	20
Table 31 Manage Zone.....	20
Table 32 Basic Flow.....	20
Table 33 Alternate Flow.....	20
Table 34 Crop Type.....	21
Table 35 Basic Flow.....	21
Table 36 Alternate Flow.....	21
Table 37 Monitor Sensor Data.....	22
Table 38 Basic Flow.....	22
Table 39 Alternate Flow.....	22
Table 40 40 Fetch Weather Data	22
Table 41 Table 41 Basic Flow.....	22
Table 42 Alternate Flow.....	23
Table 43 Automatic Irrigation Control.....	23
Table 44 Basic Flow.....	23
Table 45 Alternate Flow.....	23
Table 46 Manual Irrigation Control.....	24
Table 47 Basic Flow.....	24
Table 48 Alternate Flow.....	24
Table 49 Zone-Based Irrigation.....	25

Table 50 Basic Flow.....	25
Table 51 Alternate Flow.....	25
Table 52 Irrigation Scheduling.....	25
Table 53 Basic Flow.....	25
Table 54 Alternate Flow.....	26
Table 55 Log Irrigation Activity	26
Table 56 Basic Flow.....	26
Table 57 Alternate Flow.....	26
Table 58 Generate Reports & Analytics.....	27
Table 59 Basic Flow.....	27
Table 60 Alternate Flow.....	27
Table 61 Alerts & Notification.....	27
Table 62 Basic Flow.....	27
Table 63 Alternate Flow.....	28
Table 64 Device Health Monitoring.....	28
Table 65 Basic Flow.....	28
Table 66 Alternate Flow.....	28
Table 67 System Configuration.....	29
Table 68 Basic Flow.....	29
Table 69 Alternate Flow.....	29
Table 70 IOT Controller Setup.....	29
Table 71 Basic Flow.....	29
Table 72 Alternate Flow.....	30
Table 73 Data Backup & Restore.....	30
Table 74 Basic Flow.....	30
Table 75 Alternate Flow.....	30
Table 76 Logout.....	31
Table 77 Basic Flow.....	31
Table 78 Alternate Flow.....	31
Table 79 Create Account Positive Test Case.....	58
Table 80 Create Account Negative Test Case	59
Table 81 Login Positive Test Case	59
Table 82 Login Negative Test Case	60
Table 83 View Dashboard Positive Test Case.....	60
Table 84 View Dashboard Negative Test Case	61
Table 85 Manage Field Positive Test Case.....	61
Table 86 Manage Field Negative Test Case.....	62
Table 87 Select Crop Type Positive Test Case.....	62
Table 88 Select Crop Type Negative Test Case.....	63
Table 89 Read Sensor Data Positive Test Case	63
Table 90 Read Sensor Data Negative Test Case.....	64
Table 91 Fetch Weather Data Positive Test Case.....	64
Table 92 Fetch Weather Data Negative Test Case	65
Table 93 Automatic Irrigation Postive Test Case.....	65
Table 94 Automatic Irrigation Negative Test Case	66
Table 95 Manual Irrigation Control Positive Test Case	66
Table 96 Manual Irrigation Control Negative Test Case.....	66
Table 97 Irrigation Scheduling Positive Test Case.....	67
Table 98 Irrigation Scheduling Negative Test Case	67
Table 99 Activity Logging Positive Test Case	68
Table 100 Activity Logging Negative Test Case.....	68
Table 101 Generate Reports Positive Test Case.....	69

Table 102 Generate Reports Negative Test Case	69
Table 103 Notifications Positive Test Case	70
Table 104 Notifications Negative Test Case	70
Table 105 Device Health Monitoring Positive Test Case	71
Table 106 Device Health Monitoring Negative Test Case	71
Table 107 System Configuration Positive Test Case	72
Table 108 System Configuration Negative Test Case	72
Table 109 IoT Controller Setup Positive Test Case	73
Table 110 IoT Controller Setup Negative Test Case	73
Table 111 Backup and Restore Positive Test Case	74
Table 112 Backup and Restore Negative Test Case	74
Table 113 Logout Positive Test Case	74
Table 114 Logout Negative Test Case	75
Table 115 Equivalence Partitions for the Smart Irrigation System	76
Table 116 BVA – Soil-moisture lower threshold (start trigger)	76
Table 117 BVA – Soil-moisture upper threshold (stop trigger)	77
Table 118 BVA – Rain-skip probability threshold	77
Table 119 Irrigation duration	77
Table 120 BVA – Summary of remaining bounded inputs	78
Table 121 Decision Table – Per-zone irrigation decision	78
Table 122 Decision Table – Login authentication	78
Table 123 State-transition table – Irrigation event	79
Table 124 State-transition table – Invalid irrigation transitions	79
Table 125 State-transition table – Device status	79
Table 126 Use-case test scenarios	80
Table 127 Use-case test scenarios	80
Table 128 Cyclomatic complexity of key modules	81
Table 129 Performance test results	81
Table 130 Stress test scenarios and outcomes	82
Table 131 Programming languages	84
Table 132 Frameworks, libraries and runtime	84
Table 133 External services and hardware	84

Chapter 1: Introduction to the Problem

1.1 Introduction

Agriculture is the backbone of Pakistan's economy, contributing nearly 19% to the national GDP and providing employment to over 40% of the labor force [2-3]. Despite its significance, the sector continues to face severe challenges, the most critical being water scarcity and inefficient irrigation practices. Pakistan's per capita water availability has dropped below 1,000 m³ per person, classifying it as a water-scarce nation. More than 90% of the country's freshwater resources are consumed by agriculture, yet traditional flood-irrigation techniques dominate rural farming communities [4]. These outdated irrigation methods lead to massive water wastage, reduced soil productivity, and uneven distribution of water across irrigation channels, intensifying conflicts among farmers. Climate change further worsens these challenges, causing irregular rainfall, prolonged droughts, floods, and rising humidity, all of which threaten national food security and the livelihood of millions of farmers [5].

In recent years, the need for precision agriculture has grown rapidly. Traditional irrigation systems rely heavily on farmer judgment rather than scientific measurement of soil moisture or environmental conditions, resulting in poor crop yield and inefficient water usage. IoT technology offers a transformative solution by providing real-time monitoring and intelligent automation to optimize irrigation processes. An IoT-based Smart Irrigation System uses soil moisture sensors, weather forecasts, automated valves, and data analytics to ensure that crops receive the right amount of water at the right time [1]. This reduces human error, conserves water, increases productivity, and promotes sustainable agricultural practices.

However, existing IoT-based irrigation systems in Pakistan are still limited in scope. Most solutions focus only on soil moisture and lack advanced features such as AI-based decision-making, multi-zone water distribution, predictive models, or integration with real-time weather forecasting. Additionally, many commercial smart irrigation systems are expensive, imported, and difficult for smallholder farmers to adopt. They fail to address Pakistan's complex agricultural landscape, which includes small fragmented landholdings, canal-based irrigation systems, and highly variable climatic conditions. These limitations highlight the pressing need for a locally adaptable, cost-effective, and intelligent irrigation framework.

The proposed IoT-based Smart Irrigation System aims to bridge this gap by introducing a comprehensive, automated, and climate-aware solution designed specifically for Pakistan's agricultural environment. The system utilizes moisture sensors, IoT controllers, wireless communication, and AI-driven decision algorithms to automate irrigation based on soil conditions and weather predictions. By integrating real-time weather data, zone-based water distribution, and smart analytics, the system enhances water efficiency, reduces manual labor, and improves crop yield [7]. This solution not only addresses water scarcity but also strengthens climate resilience and ensures equitable water distribution for farmers, making it a valuable contribution to modern agriculture in Pakistan.

1.2 Purpose

The purpose of this project is to design and develop an IoT-based Smart Irrigation System that automates irrigation using real-time environmental data, saving water and ensuring optimal crop growth. The main aim is to empower farmers with a low-cost, easy-to-use, and highly efficient solution that addresses the limitations of traditional irrigation methods.

This system is developed to:

- Reduce water wastage in agriculture through sensor-driven decision-making
- Provide accurate moisture-based irrigation instead of relying on guesswork
- Help farmers remotely monitor their fields through digital dashboards
- Minimize manual labor and operational costs
- Increase crop productivity by delivering water at the right time
- Support sustainable agricultural practices using smart technology

In modern agriculture, where time, resources, and water efficiency are crucial, this solution directly addresses the need for automation, precision, and reliability. The system contributes to improving farmers' livelihoods while promoting environmentally responsible farming.

1.3 Objectives

Effective water management in agriculture requires accurate monitoring, timely decision-making, and efficient delivery of water to crops. Traditional irrigation practices in Pakistan often lack precision and rely on manual judgment, which leads to water wastage, inconsistent crop growth, and reduced productivity. With increasing water scarcity and unpredictable climatic patterns, there is a critical need for a system that can intelligently adjust irrigation based on real-time environmental conditions. The objective of this project is to develop an IoT-based Smart Irrigation System that automates and optimizes the irrigation process by integrating soil moisture data, weather information, and intelligent decision-making algorithms. The system aims to assist farmers in improving water efficiency, increasing crop yield, reducing labor, and enhancing overall sustainability in agricultural operations.

To achieve this vision, the project defines the following key objectives:

1. To design and develop an automated irrigation system that uses IoT sensors to monitor soil moisture, humidity, and environmental conditions in real time.
2. To optimize water usage by activating irrigation only when moisture levels fall below a predefined threshold, thus preventing water wastage.
3. To integrate real-time weather forecasting to avoid unnecessary irrigation during expected rainfall or unfavorable climatic conditions.
4. To enable remote monitoring and control through a user-friendly dashboard accessible via web or mobile application.
5. To incorporate decision-making for more intelligent irrigation scheduling.

6. To provide logging and reporting of irrigation activities, sensor readings, and water consumption trends.
7. To design a cost-effective and scalable system that can be deployed across small, medium, and large agricultural farms in Pakistan.
8. To reduce manual labor and operational burden by automating repetitive irrigation tasks and minimizing direct farmer intervention.
9. To improve crop yield and soil health by ensuring crops receive consistent and accurate amounts of water at the correct times.
10. To enhance climate resilience and resource sustainability by encouraging data-driven agricultural decision-making and water conservation practices.

1.4 Existing Solutions

Agricultural irrigation systems have evolved over time, yet most existing solutions still fail to provide the level of intelligence, adaptability, and efficiency required in modern farming. The most commonly used method in many regions is manual irrigation, where farmers rely on traditional practices and their personal judgment to decide when and how much water should be applied to crops. This approach is labor-intensive and time-consuming and often leads to inaccurate decisions because it does not consider precise soil moisture levels or real-time environmental conditions. As a result, manual irrigation frequently causes over-irrigation, which wastes water and affects soil health, or under-irrigation, which leads to poor crop growth and reduced yield.

Another widely used practice is the timer-based irrigation system. These systems operate according to a fixed schedule, turning the water pump on every morning for a set number of minutes. Although this method reduces manual effort, it lacks intelligence and responsiveness. Timer-based systems do not account for actual soil moisture, weather forecasts, or rainfall conditions [8]. Consequently, irrigation may occur even when the soil already has sufficient moisture or when rainfall is imminent, resulting in significant water wastage and energy consumption. These limitations make timer-based systems inefficient and unsuitable for areas facing water scarcity or unpredictable climate patterns.

Some modern farms have adopted conventional sensor-based irrigation systems that use soil moisture sensors to determine when irrigation should be activated [9]. While this approach is an improvement over manual and timer-based systems, it still has several shortcomings. Many such systems are limited in functionality, lacking integration with weather data or predictive analytics. They often do not support remote monitoring, decision automation, or data logging, making them inadequate for large-scale or multi-zone agricultural fields. In addition, many available sensor-based solutions are expensive, difficult to maintain, or designed for industrial use, making them inaccessible to small or medium-scale farmers.

Commercial smart farming systems, although technologically advanced, are often proprietary and costly. They typically require specialized hardware, subscription-based cloud services, or professional technical support, creating deployment barriers for local farmers [10-12]. These

systems may also lack customization, making it difficult for users to tailor irrigation behavior to specific crop types, soil structures, or environmental conditions.

Overall, the existing solutions in the market suffer from several common drawbacks, including a lack of real-time automation, limited integration of environmental data, high operational costs, and minimal scalability. Many systems do not adequately support data-driven agriculture, leaving farmers to make decisions based on assumptions rather than accurate information. The absence of intelligent, affordable, and easily deployable smart irrigation technologies highlights a significant gap in the agricultural sector. This gap forms the foundation for the proposed IoT-based Smart Irrigation System, which aims to overcome these limitations through automation, efficiency, and intelligent decision-making.

1.5 Proposed Solution

The proposed IoT-based Smart Irrigation System is designed to address the limitations and inefficiencies of existing irrigation methods by introducing automation, intelligent decision-making, and real-time monitoring. This solution integrates soil moisture sensors, humidity and Level sensors, IoT controllers, wireless communication modules, and a centralized management platform to create a complete, intelligent irrigation framework. By continuously monitoring environmental parameters such as soil moisture, weather conditions, and atmospheric variables, the system determines the precise water requirements of crops at any given time. Unlike traditional or timer-based irrigation systems that operate on fixed schedules or manual judgment, the proposed system activates irrigation only when it is genuinely needed, thereby conserving water, optimizing crop growth, and minimizing operational costs.

A key component of the proposed solution is the integration of real-time weather forecasting through weather APIs, allowing the system to anticipate rainfall or unfavorable climatic conditions and adjust irrigation schedules accordingly. This prevents unnecessary watering and maximizes water efficiency. The system also offers remote monitoring and control through a web or mobile dashboard, enabling farmers to access live sensor readings, irrigation status, and system alerts from any location. Additionally, the system supports zone-based irrigation, allowing farmers to divide their fields into multiple sections and irrigate each zone according to its individual moisture level or crop requirement. This multi-zone capability enhances water distribution efficiency, especially in fields with varying soil types or crop patterns.

To improve operational reliability, the system automatically logs irrigation activities, sensor readings, decision outcomes, and hardware status, enabling farmers to track performance trends and make informed decisions based on historical data. Alerts and notifications are generated whenever moisture thresholds are reached, hardware faults occur, or unusual environmental conditions are detected, ensuring timely action and continuous system reliability. The proposed solution is designed to be low-cost, energy-efficient, and easily deployable, making it suitable for smallholder farmers as well as larger agricultural enterprises. Its modular structure allows seamless integration with existing setups, while its scalable architecture ensures that additional zones, sensors, or devices can be incorporated without major modifications.

Compared to commercial smart farming solutions, this system offers a more affordable and customizable alternative that supports different crop types, field sizes, and farming practices. By leveraging IoT and intelligent automation, the proposed system significantly reduces manual labor, minimizes human error, improves crop yield, and contributes to sustainable use of water resources. Overall, the solution provides a reliable, user-friendly, and technologically advanced approach to irrigation management, effectively addressing the challenges of water scarcity and ensuring long-term agricultural sustainability.

Chapter 2: Software Requirement Specification

2.1 Introduction

The Software Requirements Specification (SRS) provides a complete description of the functionalities, constraints, interfaces, and performance requirements of the IoT-Based Smart Irrigation System. It serves as a formal agreement between stakeholders, developers, and end-users to ensure that all required system features and behaviors are clearly communicated and understood. This SRS outlines how the system will monitor soil moisture, integrate weather data, automate irrigation, handle user interactions, log activities, and ensure optimal water distribution for agricultural fields. The document also defines non-functional requirements, including performance, reliability, usability, maintainability, and security parameters.

2.1.1 Purpose

The primary purpose of this Software Requirements Specification (SRS) is to describe the operational and functional behavior of the IoT-based Smart Irrigation System. The SRS ensures that developers, designers, testers, and stakeholders understand the system requirements and expectations before implementation begins. It defines what the software will accomplish and the features it will support, ensuring accuracy and preventing misunderstandings during development.

2.1.1.1 Intended Audience

This document is intended for:

- Developers: to design architecture, modules, and backend logic.
- Testers: to verify system functionality through test cases.
- Project Supervisors: to evaluate system completeness.
- Academic Reviewers: to assess adherence to BSCS thesis standards.
- End-users (Farmers / System Operators): to understand system capabilities.

2.1.2 Scope

The IoT-based Smart Irrigation System is a complete automation solution designed to assist farmers in optimizing irrigation using real-time soil and environmental data. The software interacts with IoT sensors, controllers, valves, pumps, and external weather APIs to automate water distribution across multiple zones.

2.1.2.1 The System Will:

- Continuously monitor soil moisture, humidity, and Level.
- Connect with IoT sensors and relay modules.
- Fetch weather forecast using external APIs.
- Automate irrigation based on decision rules and thresholds.
- Provide manual irrigation override functions.
- Send alerts and notifications.

- Provide dashboards for monitoring and reporting.
- Maintain logs of irrigation, sensor data, and health devices.

2.1.2.2 The System Will NOT:

- Control pesticide spraying.
- Handle crop disease detection.
- Replace complex industrial-level irrigation management systems.
- Provide large-scale satellite-based monitoring.

2.1.2.3 Application of the System

The application is intended for:

- Small and large farms.
- Greenhouses.
- Research farms.
- Remote agricultural lands.
- Areas suffering water scarcity.

The system aims to increase water efficiency, improve crop yield, reduce manual labor, and promote climate-resilient agriculture.

2.1.3 Definitions, Acronyms, and Abbreviations

2.1.3.1 Definitions

- IoT (Internet of Things): Network of interconnected devices that collect and exchange data.
- Smart Irrigation: Automated irrigation system driven by environmental and sensor data.
- Threshold: A predefined moisture level used to trigger irrigation.
- Decision Engine: Component responsible for intelligent irrigation decisions.

2.1.3.2 Acronyms

- IoT – Internet of Things
- API – Application Programming Interface
- GUI – Graphical User Interface
- HTTP – Hyper Text Transfer Protocol
- LED – Light Emitting Diode
- DBMS – Database Management System

2.1.3.3 Abbreviations

- DB – Database
- AP – Access Point
- FT – Fault Tolerance
- UI – User Interface

- μ C – Microcontroller

2.2 Overall Description

This section describes how the product fits into the system environment and how it interacts with users, hardware, and external interfaces.

2.2.1 Product Perspective

The Smart Irrigation System is an independent and stand-alone software product closely integrated with IoT hardware components such as sensors, microcontrollers, relay modules, valves, and pumps. It communicates with external weather APIs to fetch predictive climate information and uses a centralized backend architecture to store and process sensor data.

2.2.1.1 System Interfaces

- Sensor interfaces for soil moisture, humidity, and Level.
- API integration with weather forecasting services.
- Communication interface with IoT Controller through LAN.

2.2.1.2 User Interfaces

The system provides:

- Web dashboard.
- Visual charts, zone summaries, irrigation logs, and notifications.

2.2.1.3 Hardware Interfaces

- Humidity, Level, and Soil Moisture Sensors
- IoT Controller
- Relay Modules
- Solenoid Valves
- Pump Motor

2.2.1.4 Software Interfaces

- Communication protocols: HTTP, MQTT
- Weather API: OpenWeatherMap API
- Database: MySQL / Firebase / MongoDB
- Backend Framework: Node.js / Python / Django

2.2.1.5 Communication Interfaces

- LAN
- HTTP/HTTPS
- MQTT messaging
- Local Network Protocols

2.2.1.6 Memory Requirements

- Minimum 2 GB RAM for the server
- 5 GB minimum storage for logs and databases
- 256 KB flash memory for microcontroller firmware

2.2.1.7 Operations

- Daily backup and restore operations
- Fault monitoring
- Logging of irrigation and sensor data
- Recovery from disconnects
- Site Adaptation Requirements
- Local network configuration
- Soil calibration for correct moisture readings

2.2.2 Product Functions

The product functions are aligned with the system's use cases and include all modules of the application.

2.2.2.1 Basic Workflow:

- A. Enter user details
- B. Submit form
- C. System validates and stores the data

2.2.3 User Characteristics

2.2.3.1 Intended Users

- Farmers (Basic IT knowledge)
- Agriculture field workers (low technical skills)
- System Admin (IT-skilled user)

2.2.3.2 User Skills

- Ability to use a smartphone/System with a web browser.
- No programming knowledge required.
- Admin requires basic database handling knowledge.

2.2.3.3 Educational Background

- General farmers with minimal academic background.
- Admin: BSCS/IT-related education.

2.2.4 Constraints

The Smart Irrigation System is subject to constraints such as:

- Limited rural network connectivity.
- Hardware limitations of microcontrollers and sensors.
- Compliance with electrical safety standards.
- Time synchronization for weather integration.
- Security constraints for remote access.
- Power fluctuations in farming areas.
- API rate limiting from weather forecasting services.

2.2.5 Assumptions and Dependencies

- It is assumed that farmers will have access to internet connectivity.
- Sensors must be properly calibrated before deployment.
- Power supply must be stable for IoT controller operation.
- Weather API services remain available.
- Field conditions follow expected agricultural norms.

2.2.6 Apportioning of Requirements

Some features should be postponed for future versions:

- AI-driven irrigation prediction.
- Mobile app (native).
- Voice-controlled irrigation.
- Integration with drone sensors.
- Advanced machine learning crop forecasting.

2.3 Specific Requirements

2.3.1 Functional Requirements

FR_01 – Create Account

Field	Description
ID	FR_01
Name	Create Account
Description	Allows the admin or new user to create a system account by entering required information such as name, email, password, and role.
Input	Name, Email, Password, Role
Output	“Account Successfully Created”
Requirements	Internet connection, valid email format
Basic Workflow	1. User enters account details. 2. Clicks submit. 3. System validates data. 4. System saves the account in database.

Table 1 Create Account

FR_02 – Login

Field	Description
ID	FR_02
Name	Login
Description	Allows a registered user to log into the system using email and password.
Input	Email, Password
Output	Successful login → Dashboard Failed login → Error alert
Requirements	Valid credentials, Internet connection
Basic Workflow	1. Enter email and password. 2. System validates credentials. 3. Login granted or rejected.

*Table 2 Login***FR_03 – View Dashboard**

Field	Description
ID	FR_03
Name	View Dashboard
Description	Displays summarized information such as soil moisture, humidity, Level , irrigation status, alerts, and recent activity.
Input	None
Output	Dashboard graphical data & statistics
Requirements	Logged-in user
Basic Workflow	1. User selects dashboard. 2. System fetches sensor values. 3. Visual display shown.

*Table 3 View Dashboard***FR_04 – Manage Field**

Field	Description
ID	FR_04
Name	Manage Field
Description	Allows users to create, update, or delete field information such as field name, size, and soil type.
Input	Field details
Output	Field added/updated/deleted
Requirements	Authenticated user
Basic Workflow	1. User enters field details. 2. System validates and stores information.

*Table 4 Manage Field***FR_05 – Select Crop Type**

Field	Description
ID	FR_05
Name	Select Crop Type

Description	Allows users to choose the crop type which defines moisture thresholds.
Input	Crop name
Output	Crop thresholds applied
Requirements	Internet connection (if crop data fetched online)
Basic Workflow	1. User selects crop. 2. System loads preset rules.

Table 5 Select Crop Type

FR_06 – Read Sensor Data

Field	Description
ID	FR_06
Name	Read Sensor Data
Description	System retrieves real-time sensor values (moisture, humidity, Level).
Input	Request from device
Output	Real-time sensor readings
Requirements	IoT controller must be online
Basic Workflow	1. Controller sends data. 2. System records and displays readings.

Table 6 Read Sensor Data

FR_07 – Fetch Weather Data

Field	Description
ID	FR_07
Name	Fetch Weather Data
Description	System communicates with weather API to retrieve forecast data.
Input	API request
Output	Weather forecast (rain, Level , humidity)
Requirements	Weather API key, Internet
Basic Workflow	1. System calls API. 2. Receives forecast. 3. Updates decision engine.

Table 7 Fetch Weather Data

FR_08 – Automatic Irrigation

Field	Description
ID	FR_08
Name	Automatic Irrigation
Description	System decides when to irrigate based on soil moisture, crop type, and weather data.
Input	Sensor values, forecast
Output	Pump/valve activation
Requirements	Working sensors, weather data
Basic Workflow	1. Compare moisture with threshold. 2. Check weather. 3. Irrigate if needed.

Table 8 Automatic Irrigation

FR_09 – Manual Irrigation Control

Field	Description
ID	FR_09
Name	Manual Irrigation Control
Description	Allows users to manually start or stop irrigation for selected zones.
Input	Zone selection, ON/OFF command
Output	Manual irrigation execution
Requirements	User authorization
Basic Workflow	1. User sends manual command. 2. System activates/deactivates valves.

*Table 9 Manual Irrigation Control***FR_10 – Irrigation Scheduling**

Field	Description
ID	FR_10
Name	Irrigation Scheduling
Description	Allows users to set irrigation schedules for specific times.
Input	Date, time, duration
Output	Scheduled irrigation
Requirements	Valid time settings
Basic Workflow	1. User sets schedule. 2. System activates irrigation automatically.

*Table 10 Irrigation Scheduling***FR_11 – Activity Logging**

Field	Description
ID	FR_11
Name	Activity Logging
Description	System logs sensor data, irrigation events, weather data, and status changes.
Input	System events
Output	Logs stored
Requirements	Database storage
Basic Workflow	1. Event occurs. 2. Log created.

*Table 11 Activity Logging***FR_12 – Generate Reports**

Field	Description
ID	FR_12
Name	Generate Reports
Description	Creates daily/weekly/monthly irrigation and sensor performance reports.
Input	Date filter
Output	PDF/graphical report
Requirements	Log history
Basic Workflow	1. User selects report type. 2. System generates output.

Table 12 Generate Reports

FR_13 – Notifications

Field	Description
ID	FR_13
Name	Notifications
Description	System sends alerts for low moisture, faults, and warnings.
Input	Trigger event
Output	SMS/email/app notification
Requirements	Notification service
Basic Workflow	1. Trigger detected. 2. Alert sent.

Table 13 Notifications

FR_14 – Device Health Monitoring

Field	Description
ID	FR_14
Name	Device Health Monitoring
Description	Monitors sensors, valves, and controllers for faults or connectivity loss.
Input	Device status
Output	Warning or “Device Healthy”
Requirements	Periodic heartbeat signals
Basic Workflow	1. System checks device health. 2. Sends alerts if device fails.

Table 14 Device Health Monitoring

FR_15 – System Configuration

Field	Description
ID	FR_15
Name	System Configuration
Description	Allows admin to configure system settings such as thresholds, units, and notification modes.
Input	Configuration values
Output	Updated settings
Requirements	Admin access
Basic Workflow	1. Admin edits settings. 2. System updates configuration.

Table 15 System Configuration

FR_16 – IoT Controller Setup

Field	Description
ID	FR_16
Name	IoT Controller Setup
Description	Allows pairing and configuration of IoT devices with the system.

Input	Device ID, Wi-Fi credentials
Output	Device connected
Requirements	Controller must be powered
Basic Workflow	1. User enters device details. 2. System initializes controller.

*Table 16 IoT Controller Setup***FR_17 – Backup and Restore**

Field	Description
ID	FR_17
Name	Backup and Restore
Description	System backs up all logs, settings, and accounts; can restore from older backups.
Input	Backup/restore command
Output	Backup file / restored system
Requirements	Storage space
Basic Workflow	1. User selects backup or restore. 2. System executes process.

*Table 17 Backup and Restore***FR_18 – Logout**

Field	Description
ID	FR_18
Name	Logout
Description	Ends user session and redirects to login screen.
Input	Logout command
Output	User logged out
Requirements	User must be logged in
Basic Workflow	1. User clicks logout. 2. Session ends.

*Table 18 Logout***2.3.2 Non-functional Requirements****Usability**

The system shall have a simple and intuitive UI accessible to non-technical farmers.

Reliability

System uptime of at least 95% is required, with auto-restart recovery for IoT nodes.

Performance

Sensor readings must update every 10–30 seconds.

API responses must be processed within 3 seconds.

Design Constraints

Operates on low-power microcontrollers. The Weather API is limited to available endpoints.

Portability

System runs on desktops, tablets, and mobile devices using browsers.

Maintainability

Modular code architecture for easy updates.

Security

Authentication required for dashboard access.
Data encryption for API communication.

License Agreement

Open-source or custom academic license as per institutional requirements.

Chapter 3: Use Case Analysis

3.1 User Registration

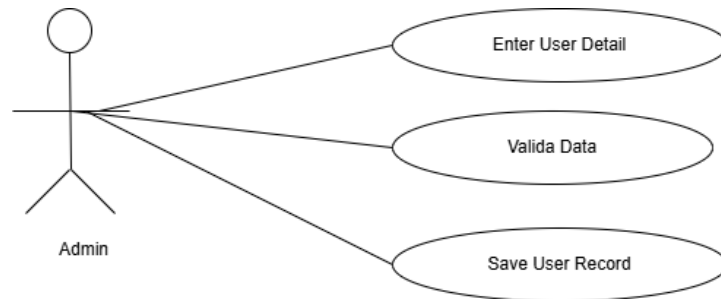


Figure 1 Use case Diagram User Registration

Use Case ID	UC_01
Use Case Name	Create Account
Description	Allows the Admin to create a new user account by entering required information. The system validates and stores the details.
Primary Actor	Admin
Secondary Actor	None
Pre-Condition	Admin must be logged in.
Post-Condition	New user account is created and stored in the database.

Table 19 User Registration

Basic Flow.

Actor Action	System Action
Admin opens Create Account page.	System displays registration form.
Admin enters details.	System validates and checks for duplicates.
Admin submits.	System stores account and shows success message.

Table 20 Basic Flow

Alternate Flow.

Condition	System Response
Missing fields	System shows error messages.
Duplicate username	Alerts admin to choose a new username.

Table 21 Alternate Flow

3.2 User Login

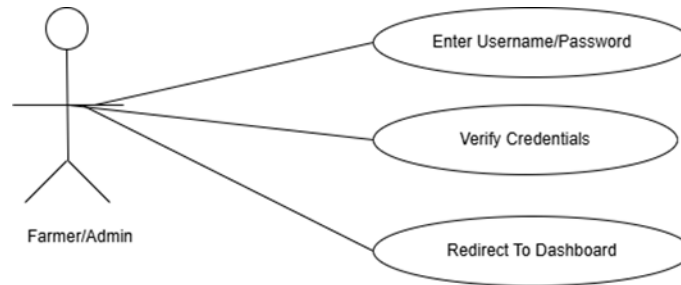


Figure 2 Usecase Diagram User Login

Use Case ID	UC_02
Use Case Name	User Login
Description	Validates user credentials to access dashboard.
Primary Actor	Farmer / Admin
Secondary Actor	None
Pre-Condition	User must have an existing account.
Post-Condition	User is logged in and redirected to dashboard.

Table 22 User Login

Basic Flow.

Actor Action	System Action
User enters username/password.	System validates credentials.
User submits login.	System creates session and shows dashboard.

Table 23 Basic Flow

Alternate Flow.

Condition	System Response
Incorrect credentials	Shows error message.
Account inactive	Shows account status notification.

Table 24 Alternate Flow

3.3 View Dashboard

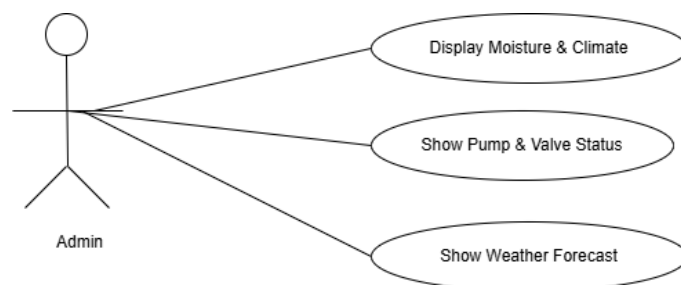


Figure 3 Usecase Diagram View Dashboard

Use Case ID	UC_03
Use Case Name	View Dashboard
Description	Displays real-time sensor data, irrigation status, alerts, and weather updates.
Primary Actor	Farmer
Secondary Actor	System
Pre-Condition	User must be logged in.
Post-Condition	Dashboard shows updated field information.

Table 25 View Dashboard

Basic Flow.

Actor Action	System Action
User navigates to Dashboard.	System loads real-time data and displays it.

Table 26 Basic Flow

Alternate Flow.

Condition	System Response
No sensor data available	Shows “No Data Found”.

Table 27 Alternate Flow

3.4 Manage Field

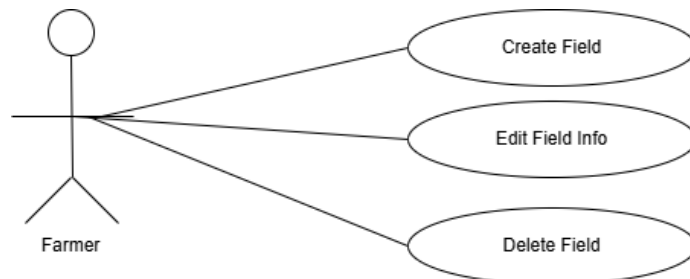


Figure 4 Usecase Diagram Manage Field

Use Case ID	UC_04
Use Case Name	Manage Field
Description	Allows farmer to add/edit field details such as field size, soil type, and area name.
Primary Actor	Farmer
Secondary Actor	None
Pre-Condition	Farmer must be logged in.
Post-Condition	Field data updated in the system.

Table 28 Manage Field

Basic Flow.

Actor Action	System Action
Farmer opens Manage Field.	System displays existing field details.
Farmer edits or adds field info.	System validates and updates the field.

Table 29 Basic Flow.

Alternate Flow.

Condition	System Response
Invalid field details	Displays validation errors.

Table 30 Alternate Flow

3.5 Manage Zone

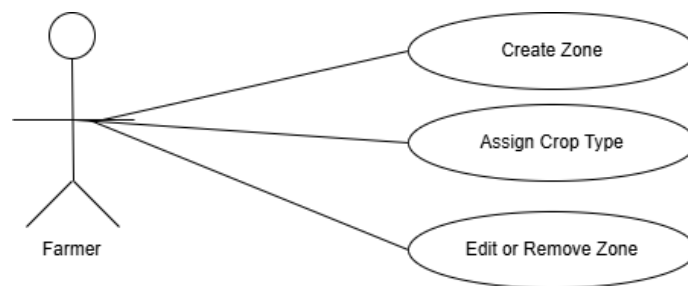


Figure 5 Usecase Diagram Manage Zone

Use Case ID	UC_05
Use Case Name	Manage Zones
Description	Allows farmer to divide the field into irrigation zones.
Primary Actor	Farmer
Secondary Actor	None
Pre-Condition	Field must be created.
Post-Condition	Zones stored in system.

Table 31 Manage Zone

Basic Flow.

Actor Action	System Action
Farmer opens Zone Management.	System displays available field.
Farmer creates/edits zones.	System saves zone data.

Table 32 Basic Flow

Alternate Flow.

Condition	System Response
Overlapping zones	System shows conflict warning.

Table 33 Alternate Flow

3.5 Select Crop Type

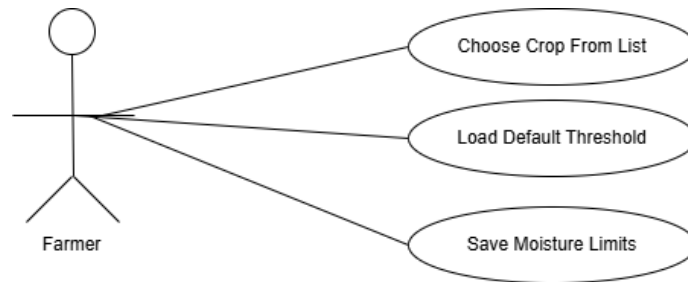


Figure 6 Usecase Diagram Select Crop Type

Use Case ID	UC_06
Use Case Name	Select Crop Type
Description	Farmer selects crop type to adjust irrigation thresholds.
Primary Actor	Farmer
Secondary Actor	Decision Engine
Pre-Condition	Zones or fields must exist.
Post-Condition	Crop type saved and applied to zones.

Table 34 Crop Type

Basic Flow.

Actor Action	System Action
Farmer selects crop.	System loads available crop profiles.
Farmer confirms selection.	System assigns crop to zone/field.

Table 35 Basic Flow

Alternate Flow.

Condition	System Response
Invalid crop selection	Shows error message.

Table 36 Alternate Flow

3.6 Monitor Sensor Data

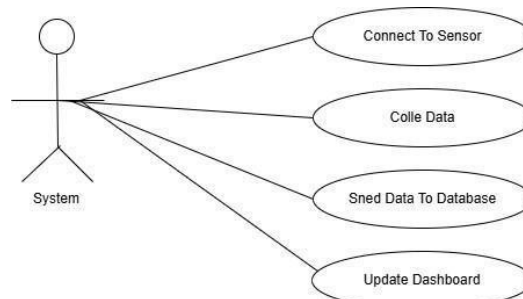


Figure 7 Usecase Diagram Monitor Sensor Data

Use Case ID	UC_07
Use Case Name	Monitor Sensor Data
Description	System retrieves sensor readings (moisture, humidity, Level).
Primary Actor	System
Secondary Actor	Farmer
Pre-Condition	Sensors must be connected.
Post-Condition	Data displayed on dashboard.

Table 37 Monitor Sensor Data

Basic Flow.

Actor Action	System Action
System requests sensor data.	IoT controller sends latest readings.

Table 38 Basic Flow

Alternate Flow.

Condition	System Response
Sensor disconnected	System alerts user.

Table 39 Alternate Flow

3.7 Fetch Weather Data

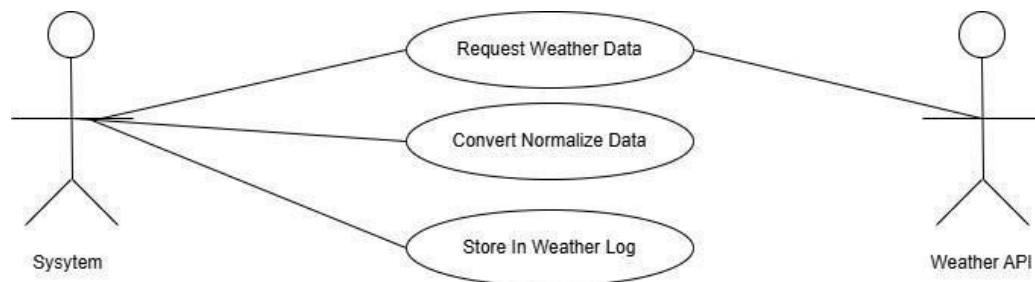


Figure 8 Usecase Diagram Fetch Weather Data

Use Case ID	UC_08
Use Case Name	Fetch Weather Data
Description	System retrieves real-time weather forecast from external API.
Primary Actor	System
Secondary Actor	Weather API
Pre-Condition	Internet must be connected.
Post-Condition	Weather stored and displayed.

Table 40 40 Fetch Weather Data

Basic Flow.

Actor Action	System Action
System sends request to weather API.	API returns weather data.

Table 41 Table 41 Basic Flow

Alternate Flow.

Condition	System Response
API failure	System uses last saved forecast.

Table 42 Alternate Flow

3.8 Automatic Irrigation Control

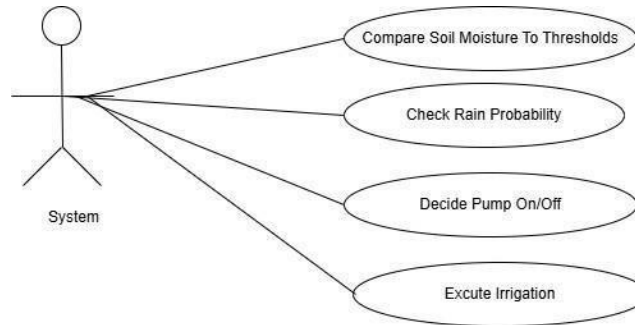


Figure 9 Usecase Diagram Automatic Irrigation Control

Use Case ID	UC_09
Use Case Name	Automatic Irrigation Control
Description	System determines irrigation need and activates pump automatically.
Primary Actor	System
Secondary Actor	Farmer
Pre-Condition	Sensors must be active; thresholds must be configured.
Post-Condition	Irrigation performed and logged.

Table 43 Automatic Irrigation Control

Basic Flow.

Use Case ID	UC_09
Use Case Name	Automatic Irrigation Control
Description	System determines irrigation need and activates pump automatically.
Primary Actor	System
Secondary Actor	Farmer
Pre-Condition	Sensors must be active; thresholds must be configured.
Post-Condition	Irrigation performed and logged.

Table 44 Basic Flow

Alternate Flow.

Condition	System Response
Rain expected	System cancels irrigation.

Table 45 Alternate Flow

3.9 Manual Irrigation Control

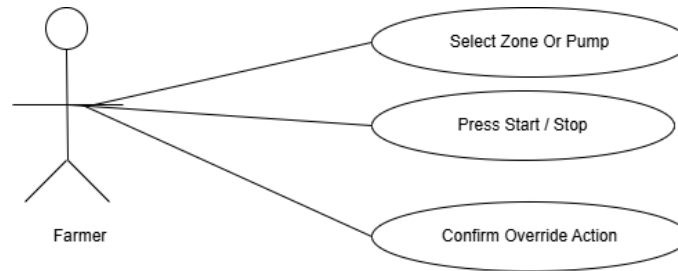


Figure 10 Usecase Diagram Manual Irrigation Control

Use Case ID	UC_10
Use Case Name	Manual Irrigation Control
Description	Farmer manually controls irrigation pump or valves.
Primary Actor	Farmer
Secondary Actor	System
Pre-Condition	User must be logged in.
Post-Condition	Pump/valves state updated.

Table 46 Manual Irrigation Control

Basic Flow.

Actor Action	System Action
Farmer selects manual irrigation ON/OFF.	System sends command to IoT controller.
Controller	Pump/valve changes state.

Table 47 Basic Flow

Alternate Flow.

Condition	System Response
Device not responding	Error shown on dashboard.

Table 48 Alternate Flow

3.10 Zone-Based Irrigation

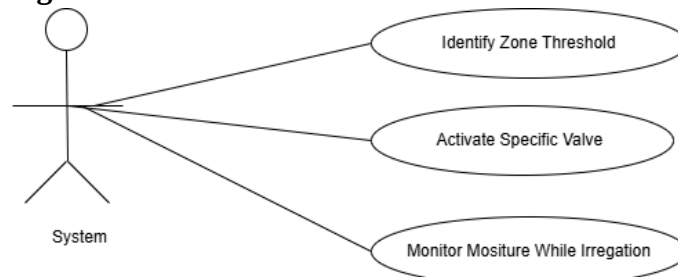


Figure 11 Usecase Diagram Zone-Based Irrigation

Use Case ID	UC_11
Use Case Name	Zone-Based Irrigation
Description	Allows irrigation to be applied to selected zones.
Primary Actor	Farmer / System

Secondary Actor	IoT Controller
Pre-Condition	Zones must exist.
Post-Condition	Selected zones irrigated.

Table 49 Zone-Based Irrigation

Basic Flow.

Actor Action	System Action
Farmer selects zone(s).	System activates valves of selected zones.

Table 50 Basic Flow

Alternate Flow.

Condition	System Response
Faulty valve	System reports valve failure.

Table 51 Alternate Flow

3.11 Irrigation Scheduling

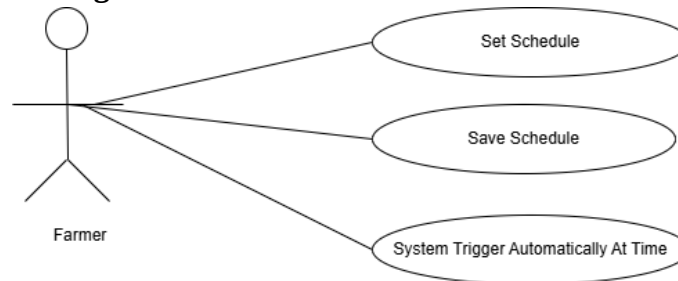


Figure 12 Usecase Diagram Irrigation Scheduling

Use Case ID	UC_12
Use Case Name	Irrigation Scheduling
Description	Farmer sets irrigation schedules for automatic execution.
Primary Actor	Farmer
Secondary Actor	System Scheduler
Pre-Condition	User must be logged in.
Post-Condition	Schedule saved and executed automatically.

Table 52 Irrigation Scheduling

Basic Flow.

Actor Action	System Action
Farmer sets schedule.	System stores schedule.
—	Scheduler triggers irrigation at set time.

Table 53 Basic Flow

Alternate Flow.

Condition	System Response
Schedule conflict	System alerts user to adjust.

Table 54 Alternate Flow

3.12 Log Irrigation Activity

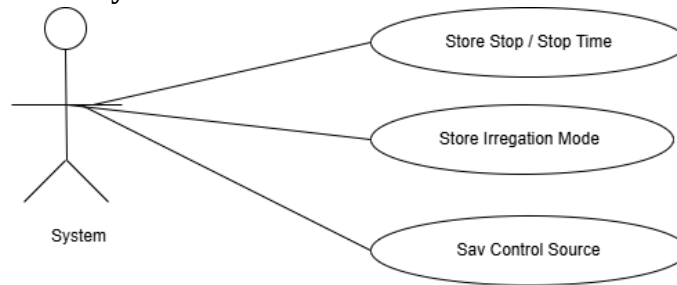


Figure 13 Usecase Diagram Log Irrigation Activity

Use Case ID	UC_13
Use Case Name	Log Irrigation Activity
Description	Records irrigation events including date, time, duration, and type.
Primary Actor	System
Secondary Actor	Farmer
Pre-Condition	Irrigation must occur.
Post-Condition	Logs stored in the database.

Table 55 Log Irrigation Activity

Basic Flow.

Actor Action	System Action
Irrigation activity occurs (start/end)	System creates log entry when irrigation starts/ends.

Table 56 Basic Flow

Alternate Flow.

Condition	System Response
Database unavailable	System temporarily stores logs locally.

Table 57 Alternate Flow

3.13 Generate Reports & Analytics

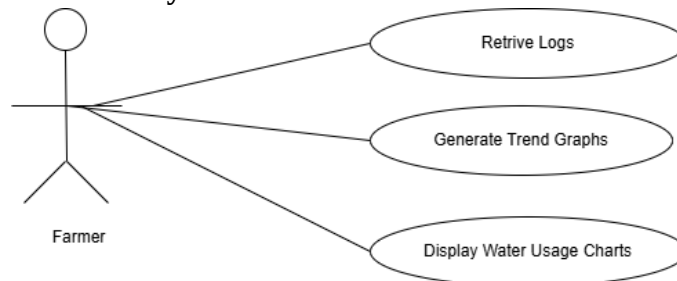


Figure 14 Usecase Diagram Generate Reports & Analytics

Use Case ID	UC_14
Use Case Name	Generate Reports
Description	Generates irrigation analytics and moisture charts.

Primary Actor	Farmer
Secondary Actor	System
Pre-Condition	Logs must exist.
Post-Condition	Report displayed or downloaded.

Table 58 Generate Reports & Analytics

Basic Flow.

Actor Action	System Action
Farmer selects date range.	System retrieves and displays report.

Table 59 Basic Flow.

Alternate Flow.

Condition	System Response
No data available	Shows empty report message.

Table 60 Alternate Flow

3.14 Alerts & Notification

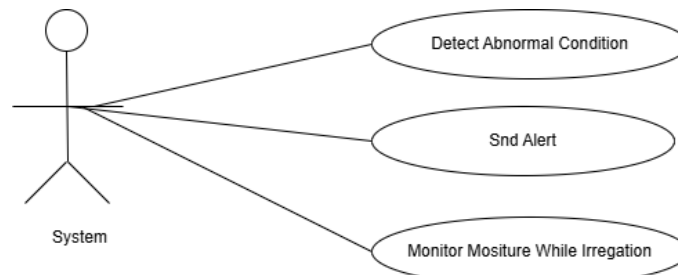


Figure 15 Usecase Diagram Alerts Notification

Use Case ID	UC_15
Use Case Name	Alerts & Notifications
Description	System sends alerts for low moisture, rain prediction, or device faults.
Primary Actor	System
Secondary Actor	Farmer
Pre-Condition	Sensors and controller must be active.
Post-Condition	Notification sent.

Table 61 Alerts & Notification

Basic Flow.

Actor Action	System Action
Event occurs (low moisture, rain alert, or device fault).	System detects event and sends alert.

Table 62 Basic Flow

Alternate Flow.

Condition	System Response
Notification server offline	System retries later.

Table 63 Alternate Flow

3.15 Device Health Monitoring

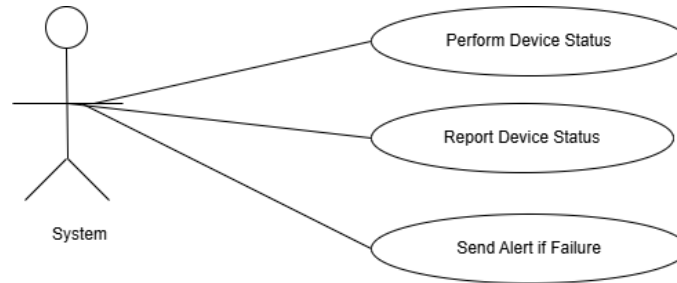


Figure 16 Usecase Diagram Device Health Monitoring

Use Case ID	UC_16
Use Case Name	Device Health Monitoring
Description	Monitors status of sensors, IoT controllers, relays, and valves.
Primary Actor	System
Secondary Actor	Farmer
Pre-Condition	Devices must be registered.
Post-Condition	Health status updated.

Table 64 Device Health Monitoring

Basic Flow.

Actor Action	System Action
System	System checks device status.

Table 65 Basic Flow

Alternate Flow.

Condition	System Response
Device failure detected	System alerts user.

Table 66 Alternate Flow

3.16 System Configuration

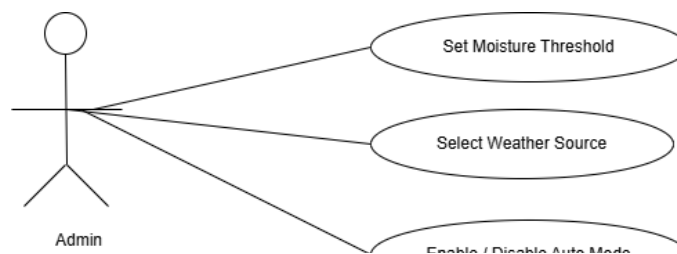


Figure 17 Usecase Diagram System Configuration

Use Case ID	UC_17
Use Case Name	System Configuration
Description	Admin configures thresholds, device mappings, and rules.
Primary Actor	Admin
Secondary Actor	System
Pre-Condition	Admin must be logged in.
Post-Condition	Updated configuration applied.

Table 67 System Configuration

Basic Flow.

Actor Action	System Action
Admin updates settings.	System saves and applies configuration.

Table 68 Basic Flow

Alternate Flow.

Condition	System Response
Invalid configuration	System rejects and notifies admin.

Table 69 Alternate Flow

3.17 IOT Controller Setup

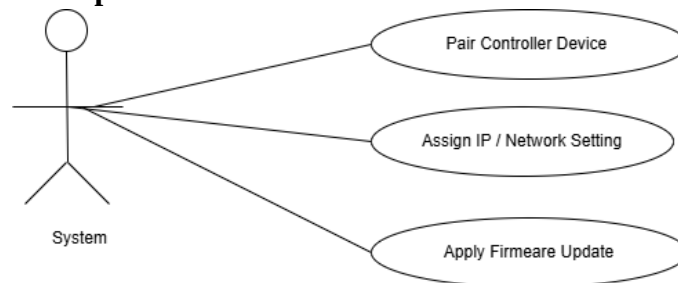


Figure 18 Usecase Diagram IOT Controller Setup

Use Case ID	UC_18
Use Case Name	IoT Controller Setup
Description	Admin registers sensors and actuators with the controller.
Primary Actor	Admin
Secondary Actor	IoT Controller
Pre-Condition	Controller must be connected.
Post-Condition	Sensors and actuators assigned and active.

Table 70 IOT Controller Setup

Basic Flow.

Actor Action	System Action
Admin selects controller.	System scans connected devices.
Admin confirms setup.	System registers all devices.

Table 71 Basic Flow

Alternate Flow.

Condition	System Response
Device not detected	Shows setup error.

Table 72 Alternate Flow

3.18 Data Backup & Restore

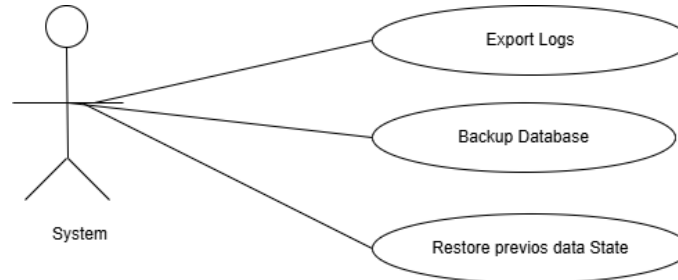


Figure 19 Usecase Diagram Data Backup & Restore

Use Case ID	UC_19
Use Case Name	Data Backup & Restore
Description	System backs up database and allows Admin to restore previous data.
Primary Actor	Admin
Secondary Actor	System
Pre-Condition	Database connection must exist.
Post-Condition	Backup or restore completed.

Table 73 Data Backup & Restore

Basic Flow.

Actor Action	System Action
Admin selects backup/restore.	System executes the operation.

Table 74 Basic Flow

Alternate Flow.

Condition	System Response
Disk full	System cancels operation.

Table 75 Alternate Flow

3.19 Logout

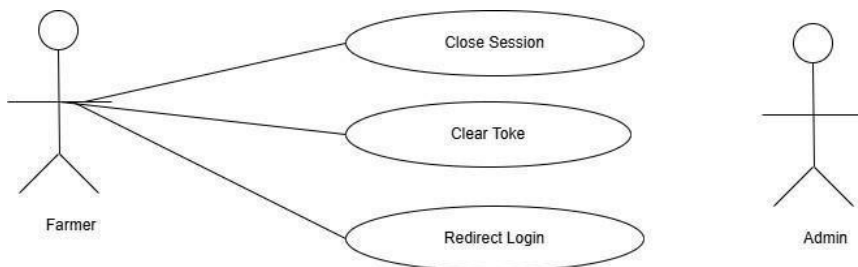


Figure 20 Usecase Diagram Logout

Use Case ID	UC_20
Use Case Name	Logout
Description	Logs user out and ends session securely.
Primary Actor	Farmer / Admin
Secondary Actor	System
Pre-Condition	User must be logged in.
Post-Condition	Session terminated. User redirected to login page.

Table 76 Logout

Basic Flow.

Actor Action	System Action
User clicks logout.	System clears session and redirects to login.

Table 77 Basic Flow

Alternate Flow.

Condition	System Response
Session timeout	System logs out user automatically.

Table 78 Alternate Flow

Chapter 4: Design

In this section, we provide the design analysis of our modules including the following designs

1. Architecture Diagram
2. ERD with data dictionary
3. Data Flow diagram
4. Class Diagram
5. Activity Diagram
6. Sequence Diagram
7. Collaboration Diagram
8. State Transition Diagram
9. Component Diagram
10. Deployment Diagram

4.1 Architecture Diagram

Figure 1 represents the complete workflow of the IoT-Based Smart Irrigation System by dividing it into three layers: Presentation, Logic, and Data. The Presentation Layer contains the Farmer/Admin UI, which allows users to monitor sensor values, view irrigation status, and control watering operations. The Logic Layer includes the Decision Engine, which evaluates soil moisture, weather data, and user inputs to decide when irrigation should start or stop. It serves as the main processing unit connecting the user interface with backend components.

In the Data Layer, three components operate together: the Weather API, the Database, and the IoT Controller. The Weather API provides real-time climate information, while the Database stores user profiles, field settings, logs, and sensor data. The IoT Controller communicates with sensors to collect moisture and environmental readings and activates actuators such as pumps and valves based on instructions from the Decision Engine. Together, these layers create a modular, automated, and intelligent irrigation system suitable for modern agricultural needs.

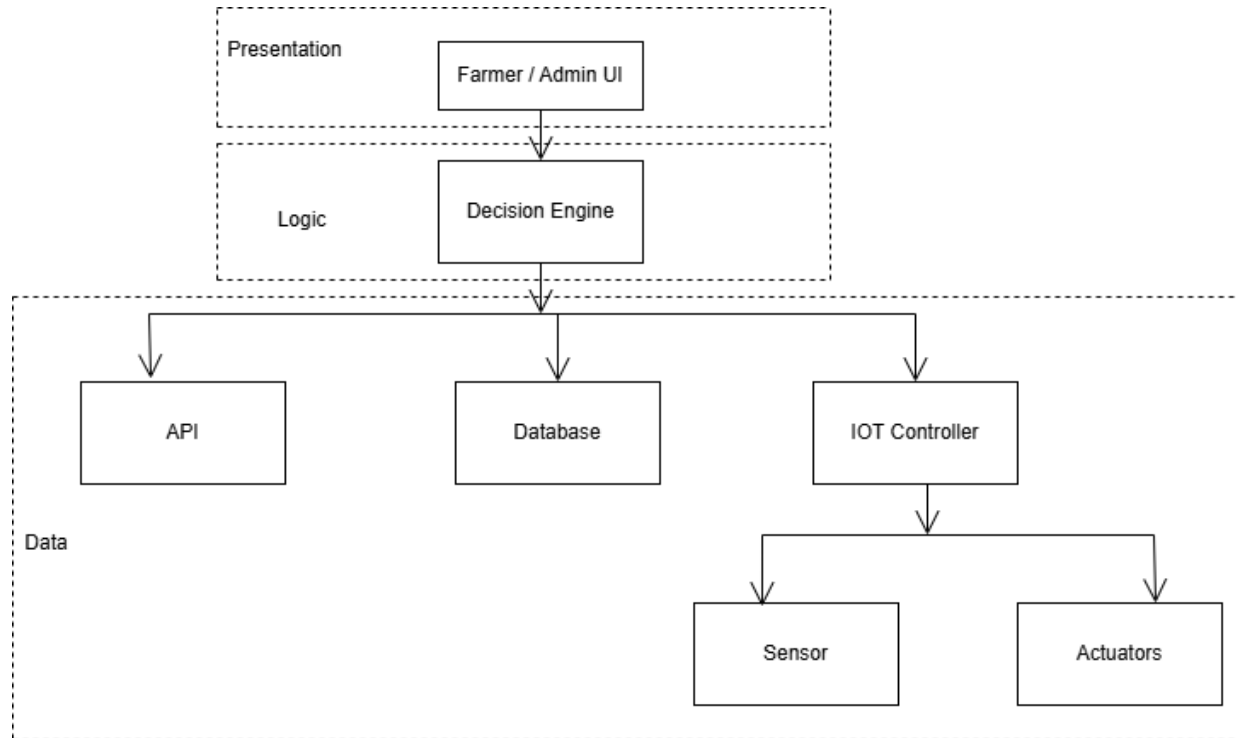


Figure 21 Architecture Diagram

4.2 ERD with data dictionary

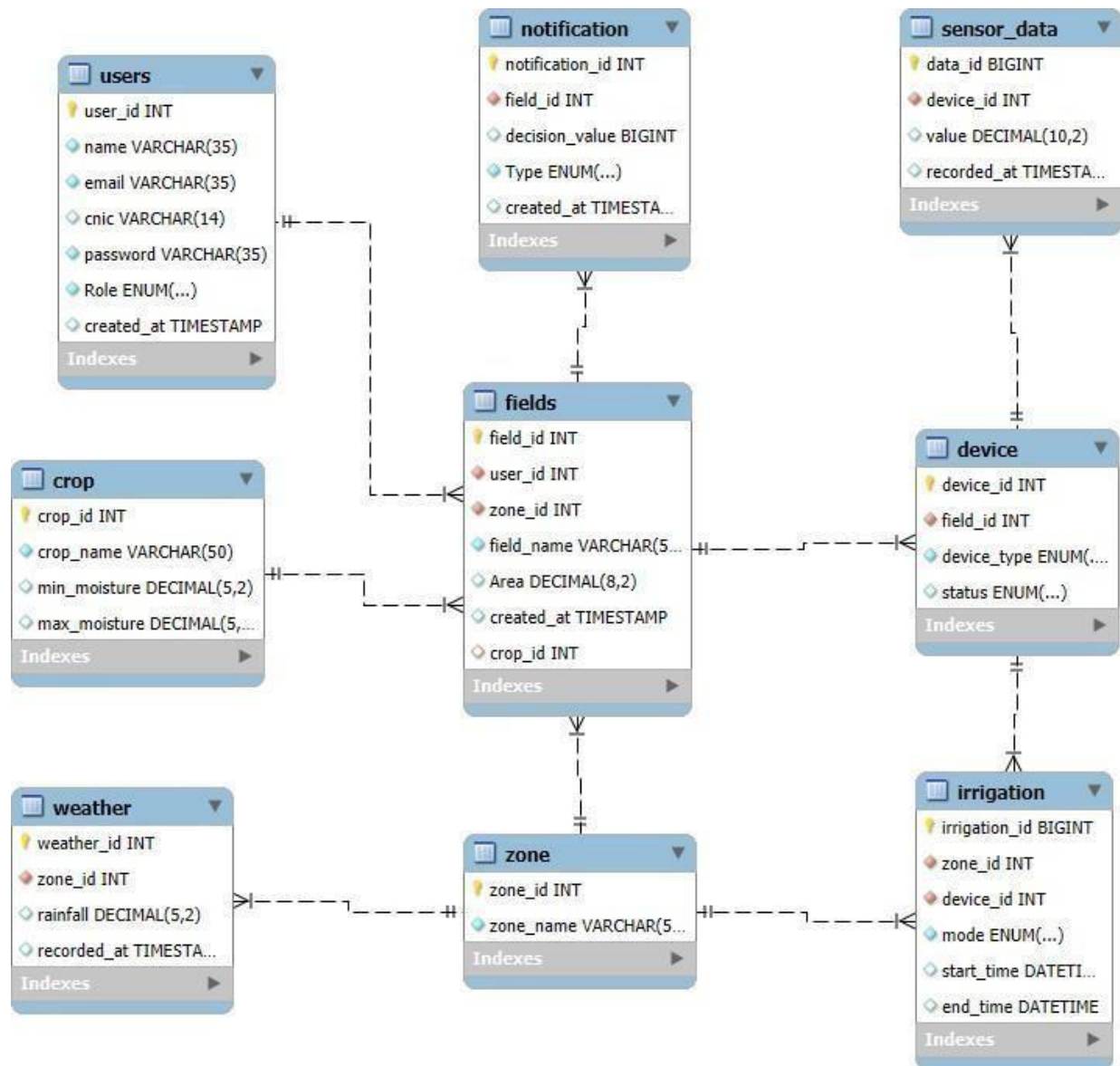


Figure 22 ERD with data dictionary

4.3 Data Flow diagram

4.3.1 The level 0

A high-level overview of how core entities interact with the central Smart Irrigation System is illustrated in Figure 2. Both the User and Admin generate various requests such as authentication, dashboard viewing, irrigation control, and system configuration, which are processed by the System and responded to accordingly. The Database supports these operations by supplying stored information and recording updates, ensuring reliable data management. The System also communicates with the external Weather API by sending requests for real-time and forecasted climate data and receiving responses that help optimize irrigation decisions.

Additionally, the IoT Controller continuously interacts with the System by sending real-time sensor readings from moisture, Level, and humidity sensors. Based on decision logic, the System issues execution commands back to the Controller to activate valves or pumps. This diagram effectively summarizes the complete information flow between all actors, showing how every component contributes to the automated and intelligent irrigation process.

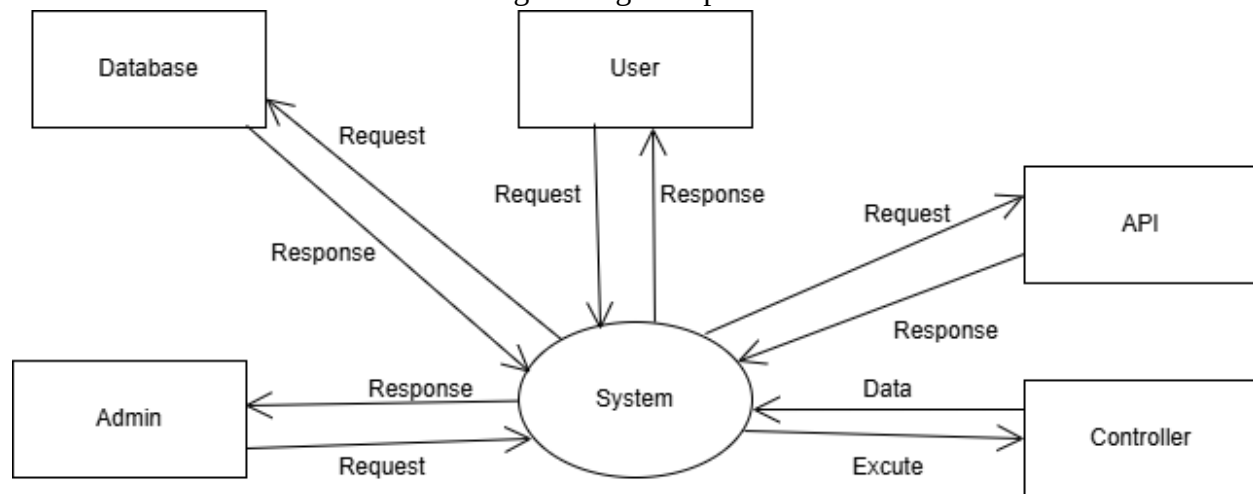


Figure 23 Level 0 DFD

4.3.2 The level 1

The Level-1 Data Flow Diagram, illustrated in Figure 3, shows the internal working of the IoT-Based Smart Irrigation System by decomposing the overall system into multiple interconnected processes. The interaction begins when the user (farmer or Admin) accesses the system through the web or mobile interface. User credentials and commands are first handled by the User Authentication process, which validates access rights before allowing further interaction. Once authenticated, the User Management process manages user profiles, permissions, and system configurations stored in the central database.

The Field Management process enables users to define irrigation zones, crop types, and threshold values for soil moisture, Level and humidity. These configurations are essential for decision-making and are stored in the database for continuous reference. Simultaneously, real-time

environmental data is collected from soil moisture, level, and humidity sensors, while weather forecast data is fetched from the Weather API. Both data sources are forwarded to the Decision Engine, which evaluates current field conditions against predefined thresholds and weather predictions.

Based on this analysis, the Decision Engine and Logic process determines whether irrigation is required. If conditions are met, control commands are generated and sent to the Irrigation Control process, which communicates directly with the IoT Controller (Arduino with Ethernet Shield). The controller then activates or deactivates pumps and solenoid valves accordingly. All irrigation activities, sensor readings, system decisions, and fault notifications are recorded by the Logging and Reporting process, ensuring traceability, performance analysis, and historical data availability for future optimization.

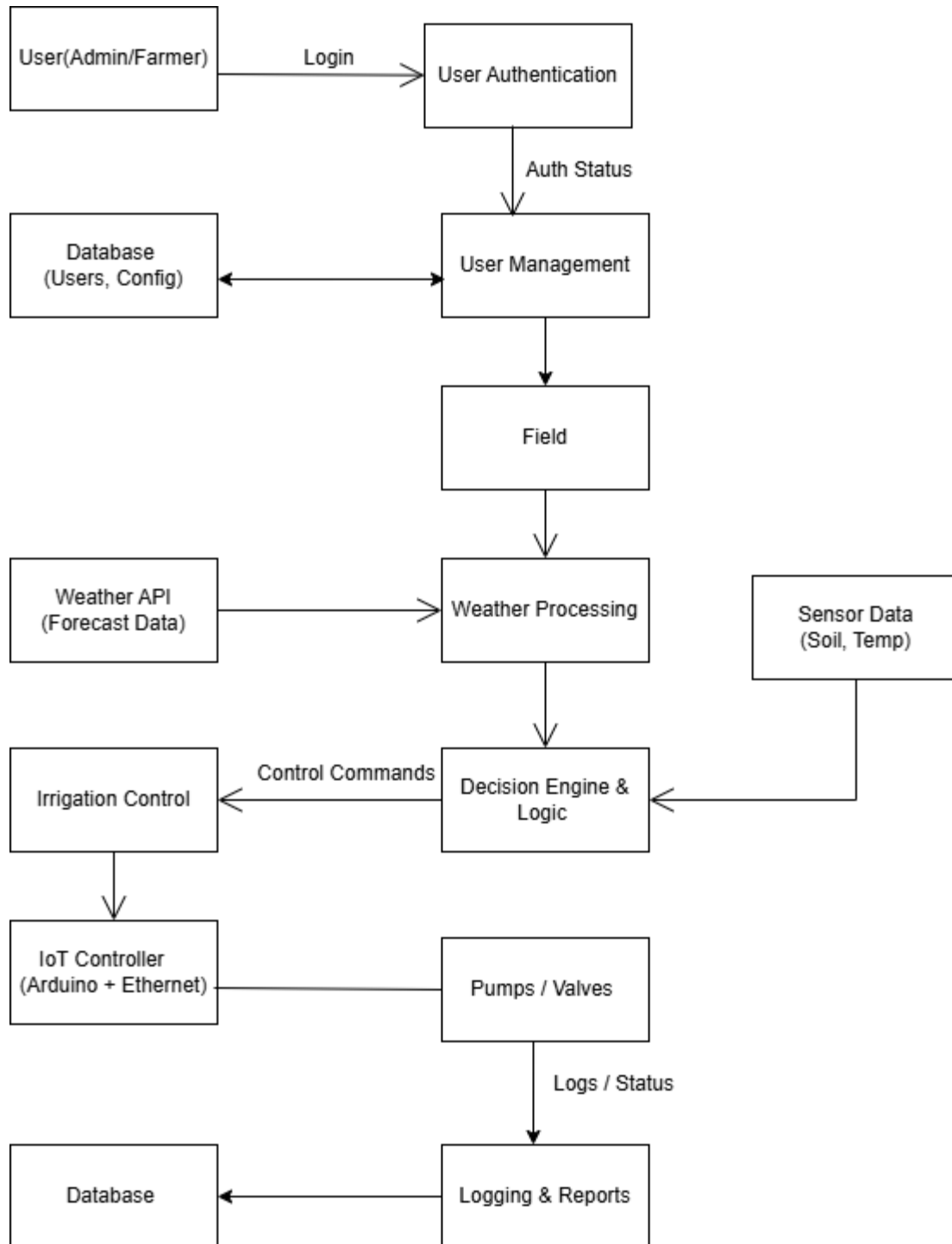


Figure 24 Level 1 DFD

4.4 Class Diagram

The Class Diagram defines the static structure of the Smart Irrigation System by detailing the classes, attributes, methods, and relationships. The central class, System Controller, interacts with most other classes such as Farmer, Dashboard, Sensor Module, Weather Service, Decision Engine, Irrigation System, and Database. Each class is designed with specific responsibilities: the Farmer class manages user profile and commands, the Dashboard displays real-time system data, the Sensor Module captures environmental readings, and the Decision Engine evaluates whether irrigation is required. The Irrigation System class controls pumps and valves, while the Database class handles storage for logs, thresholds, and configurations. Associations among these classes ensure proper communication, and relationships like composition, aggregation, and dependency are clearly represented. The Class Diagram forms the foundation for coding by defining how data and behavior are organized in the system.

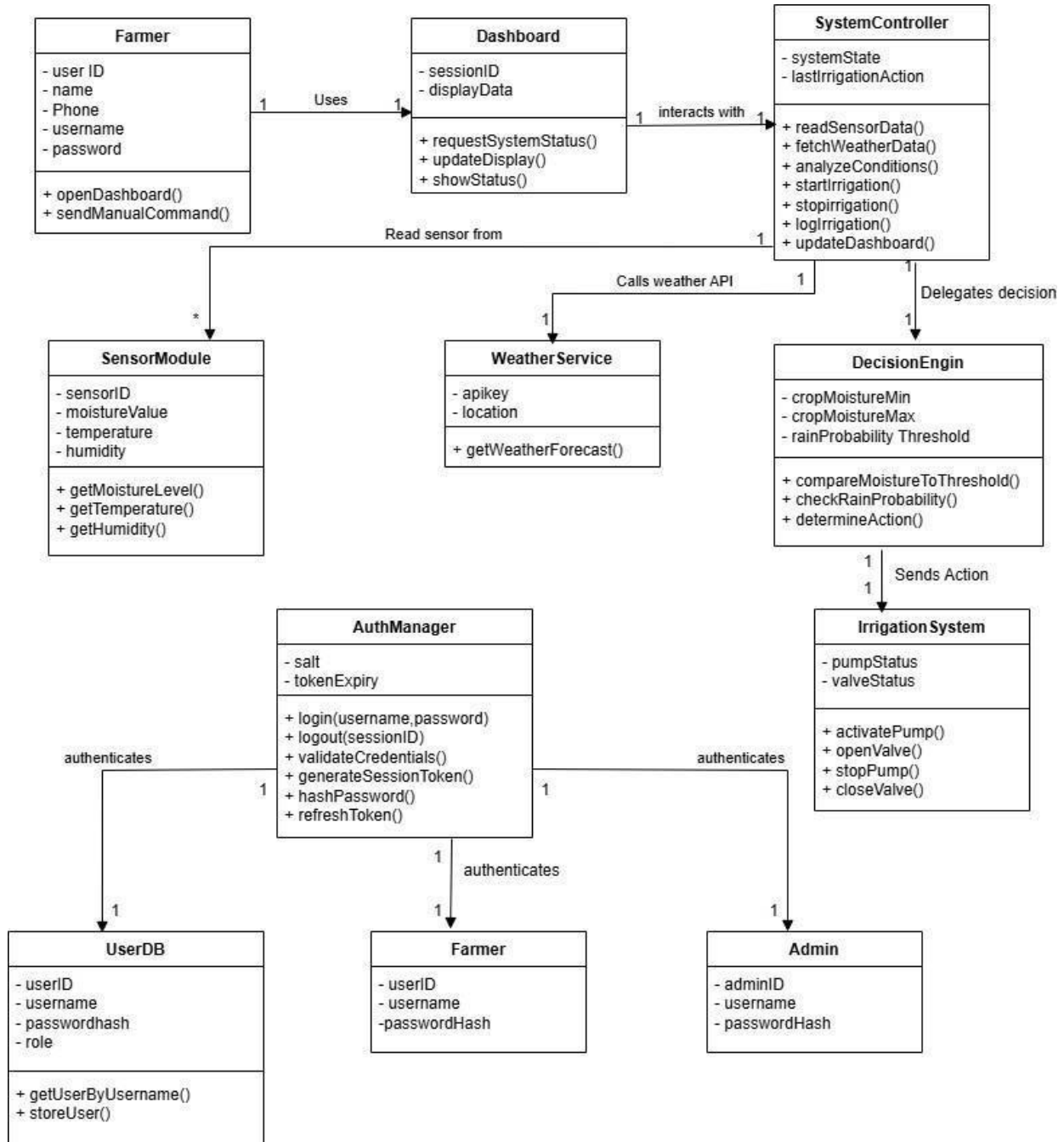


Figure 25 Class Diagram

4.5 Activity Diagram

The Activity Diagram illustrates the complete workflow of the Smart Irrigation System, capturing all major activities from login to irrigation execution. The process begins when the user opens the application and performs authentication. Once authenticated, the dashboard retrieves real-time sensor data and fetches weather information from an external API. These inputs are forwarded to the Decision Engine, which analyzes moisture levels, crop requirements, weather predictions, and configured thresholds. Based on the decision, the system either starts automatic irrigation or waits for manual commands from the user. If irrigation is activated, the system triggers the IoT controller to operate pumps and valves, logs every irrigation event, and sends notifications to the user. Finally, the dashboard is updated to reflect the new irrigation status. This Activity Diagram demonstrates how control flows between automated and manual operations and ensures that the entire irrigation process is logically aligned with the system requirements.

4.5.1 User Registration

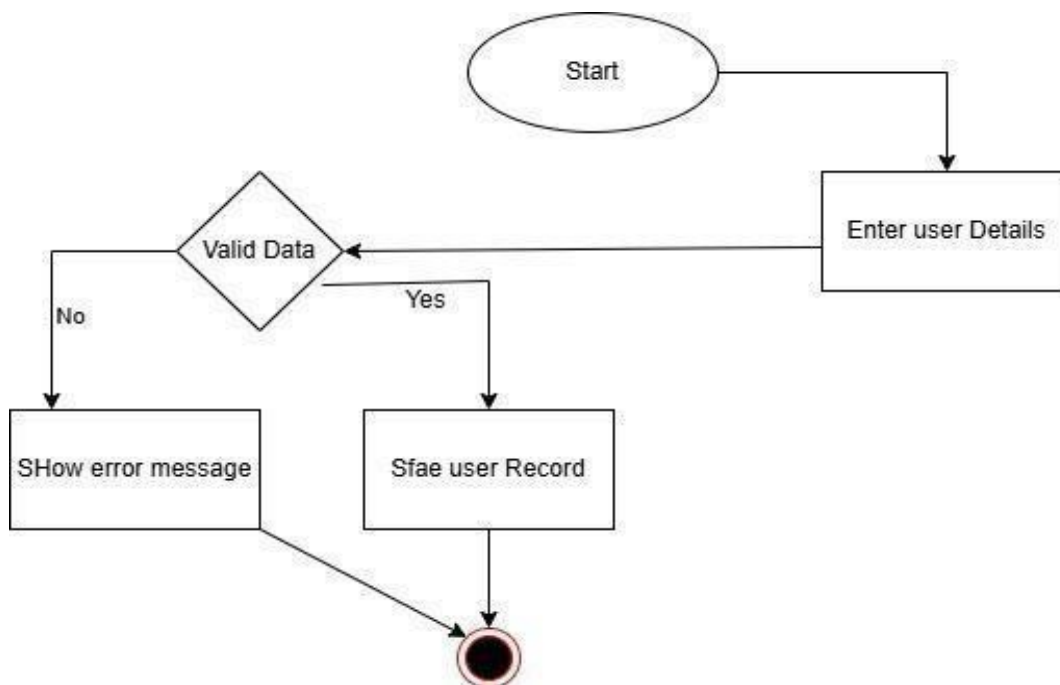


Figure 26 Activity Diagram User Registration

4.5.2 User Login

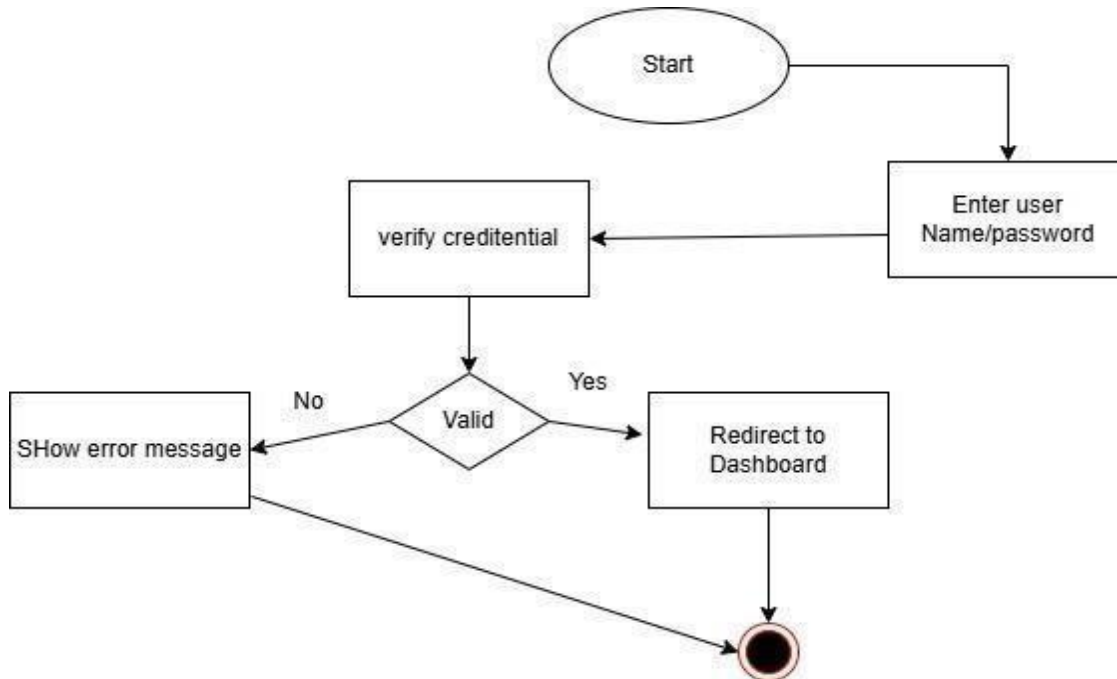


Figure 27 Activity Diagram User Login

4.5.3 View Dashboard

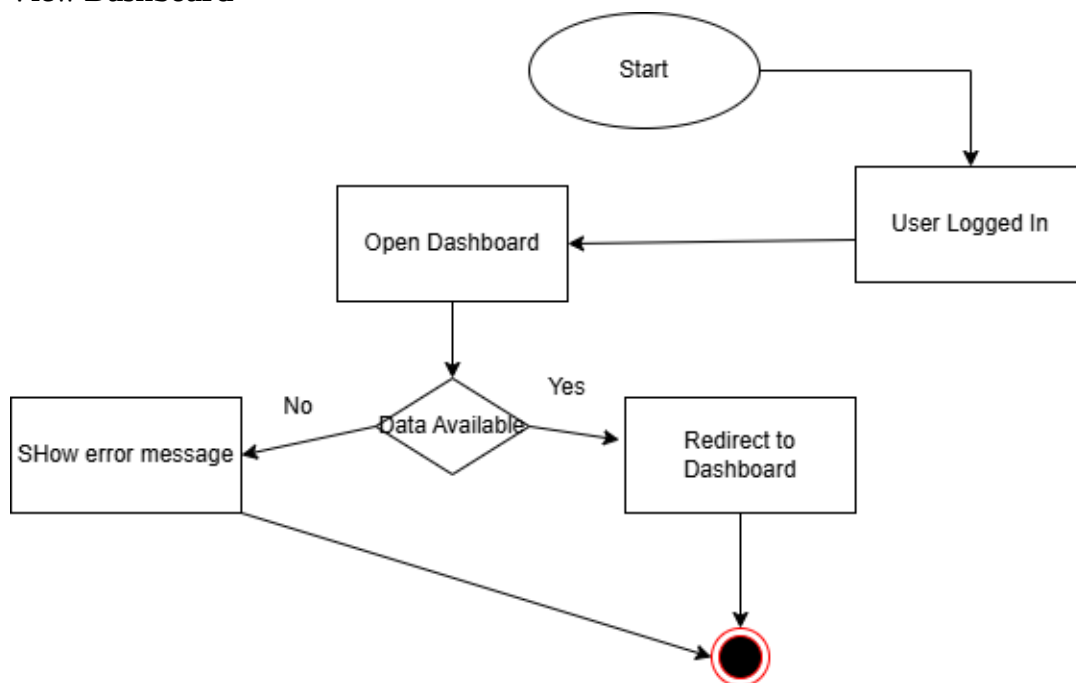


Figure 28 Activity Diagram View Dashboard

4.5.4 Manage Field

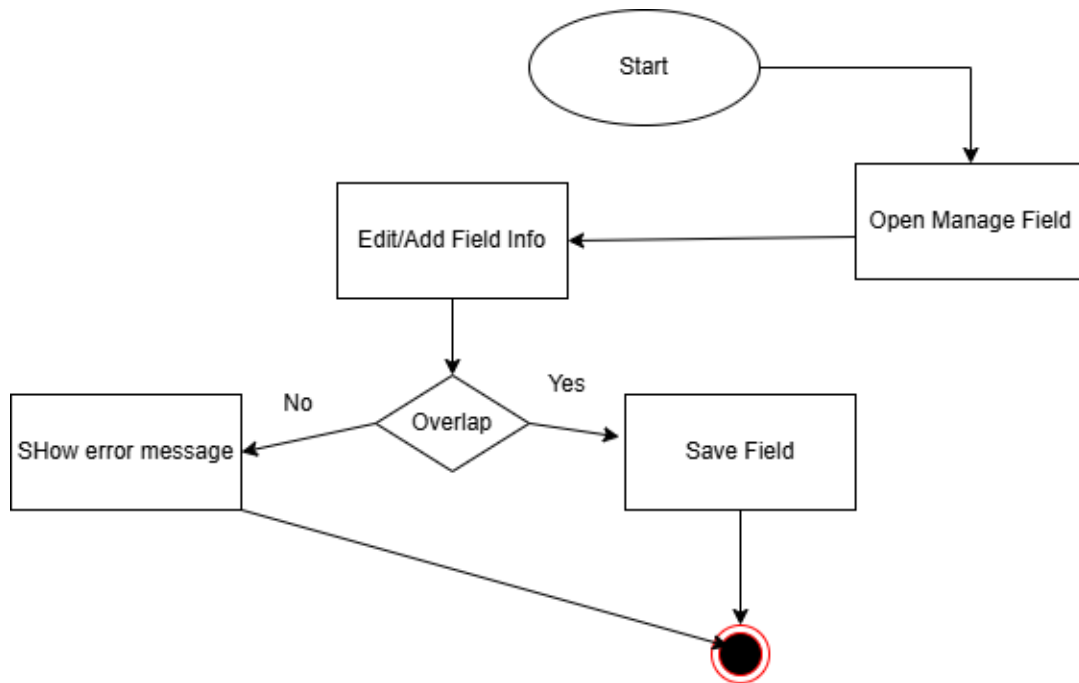


Figure 29 Activity Diagram Manage Field

4.5.5 Manage Zone

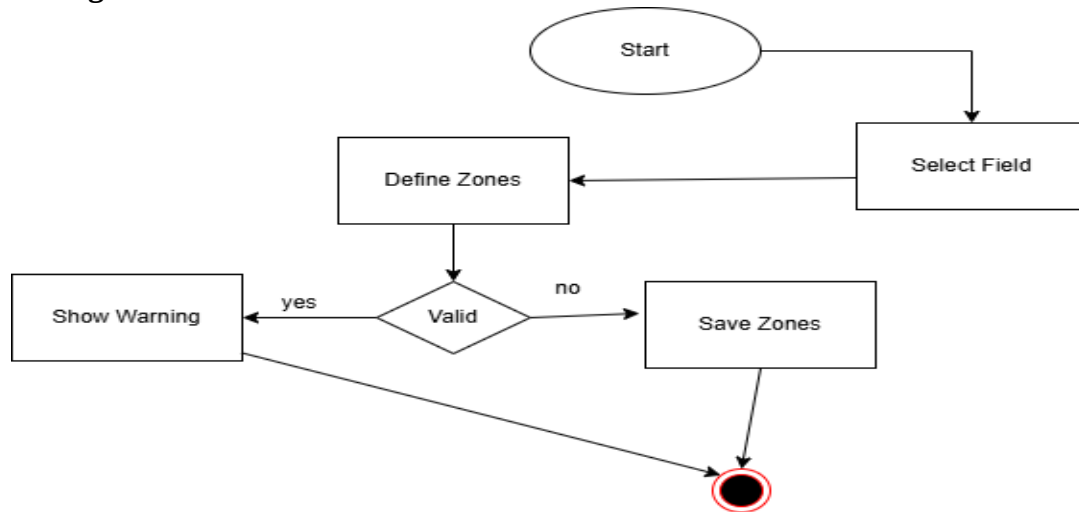


Figure 30 Activity Diagram Manage Zone

4.5.6 Select Crop Type

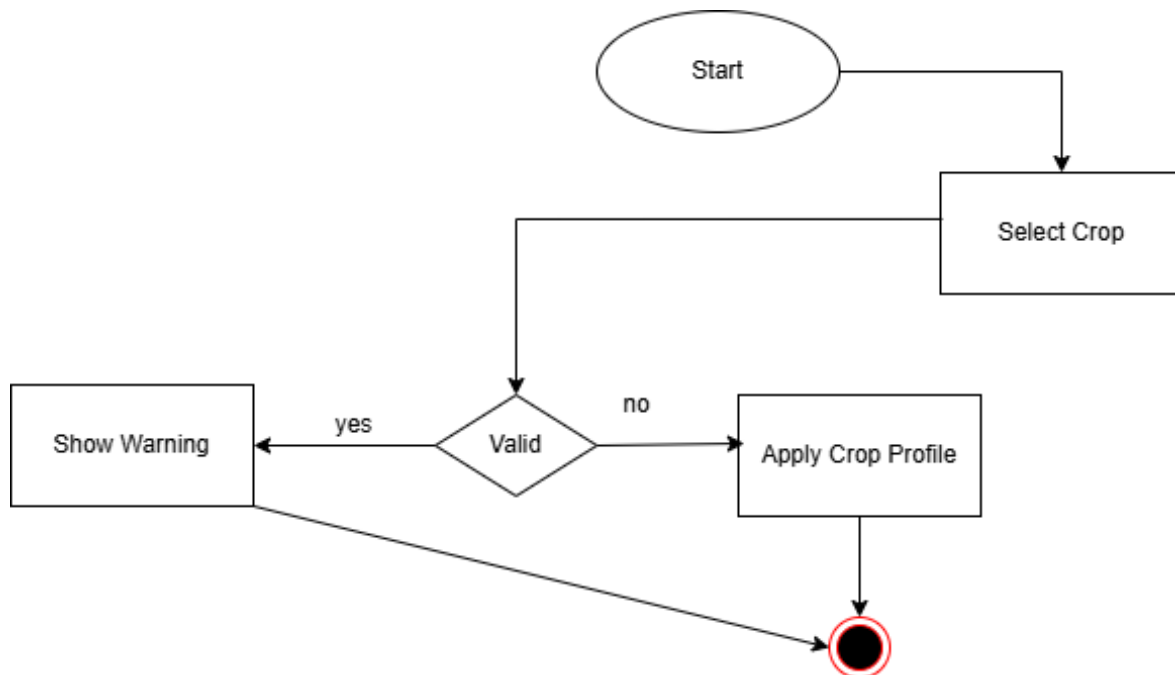


Figure 31 Activity Diagram Select Crop

4.5.7 Monitor Sensor Data

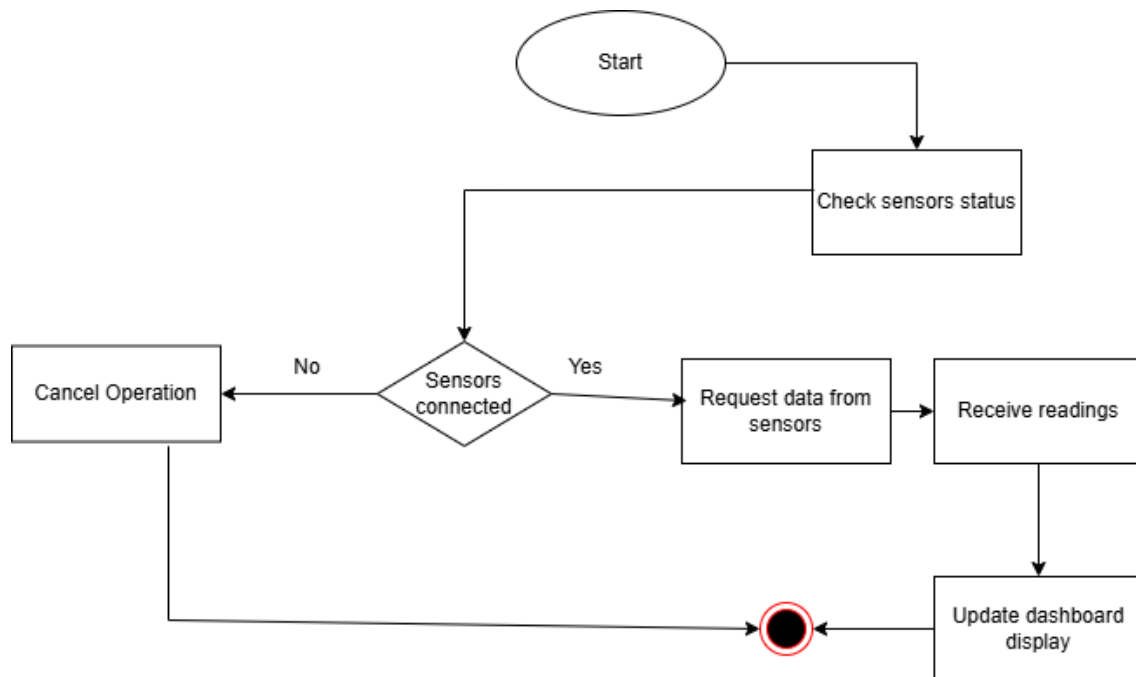


Figure 32 Activity Diagram Monitor Sensor Data

4.5.8 Fetch Weather Data

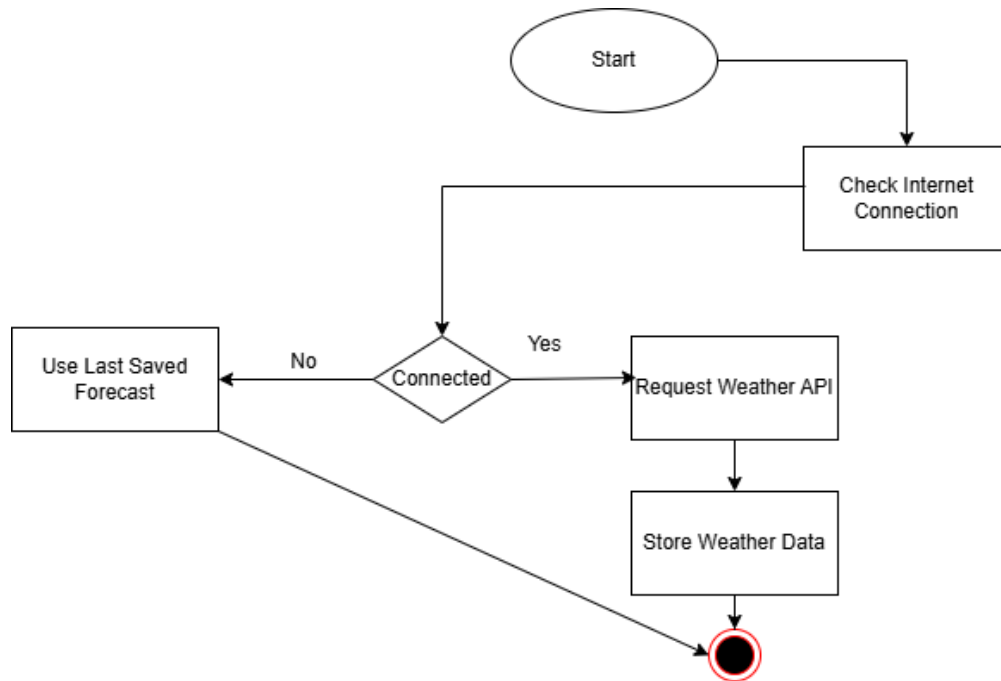


Figure 33 Activity Diagram Fetch Weather Data

4.5.9 Automatic Irrigation Control

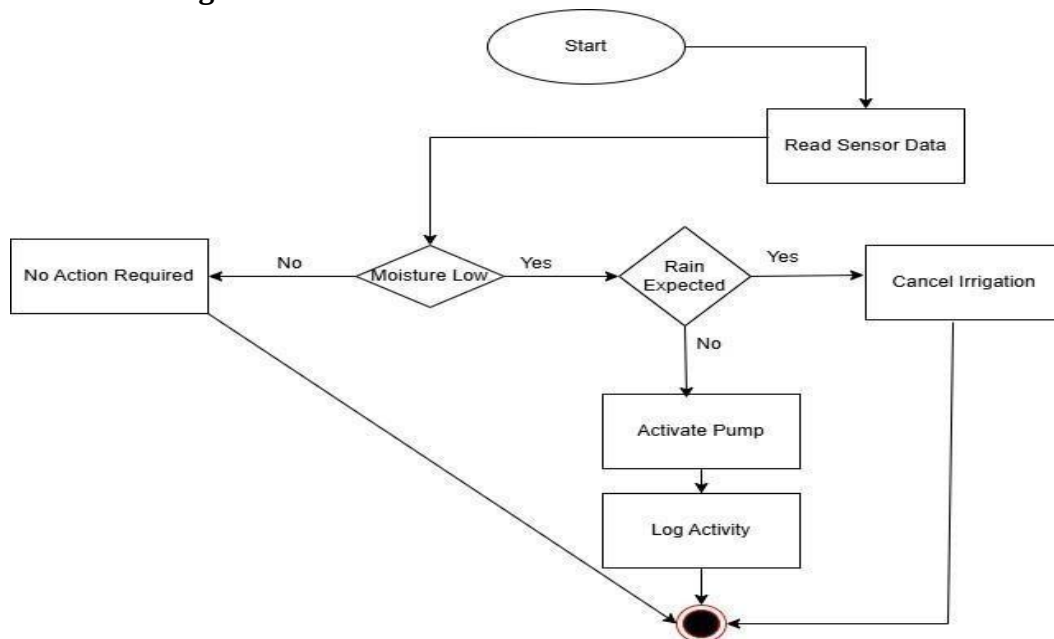


Figure 34 Activity Diagram Automatic Irrigation

4.5.10 Manual Irrigation Control

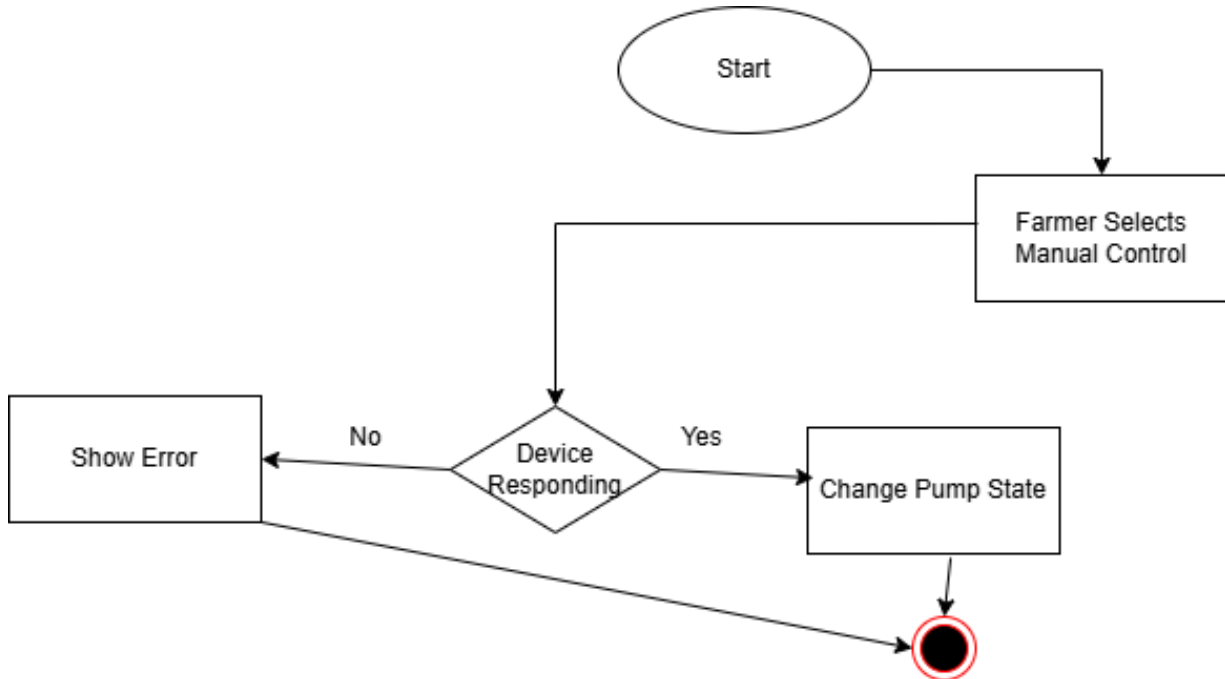


Figure 35 Activity Diagram Manual Irrigation Control

4.5.11 Zone-Based Irrigation

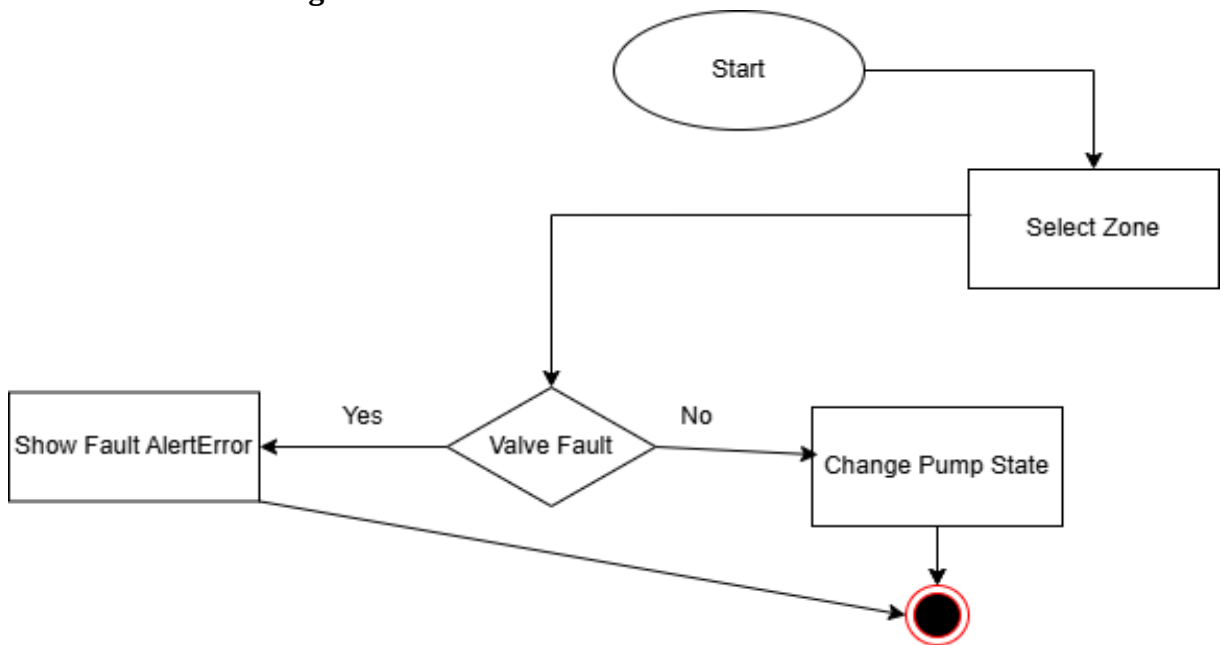


Figure 36 Activity Diagram Zone-Based Irrigation

4.5.12 Irrigation Scheduling

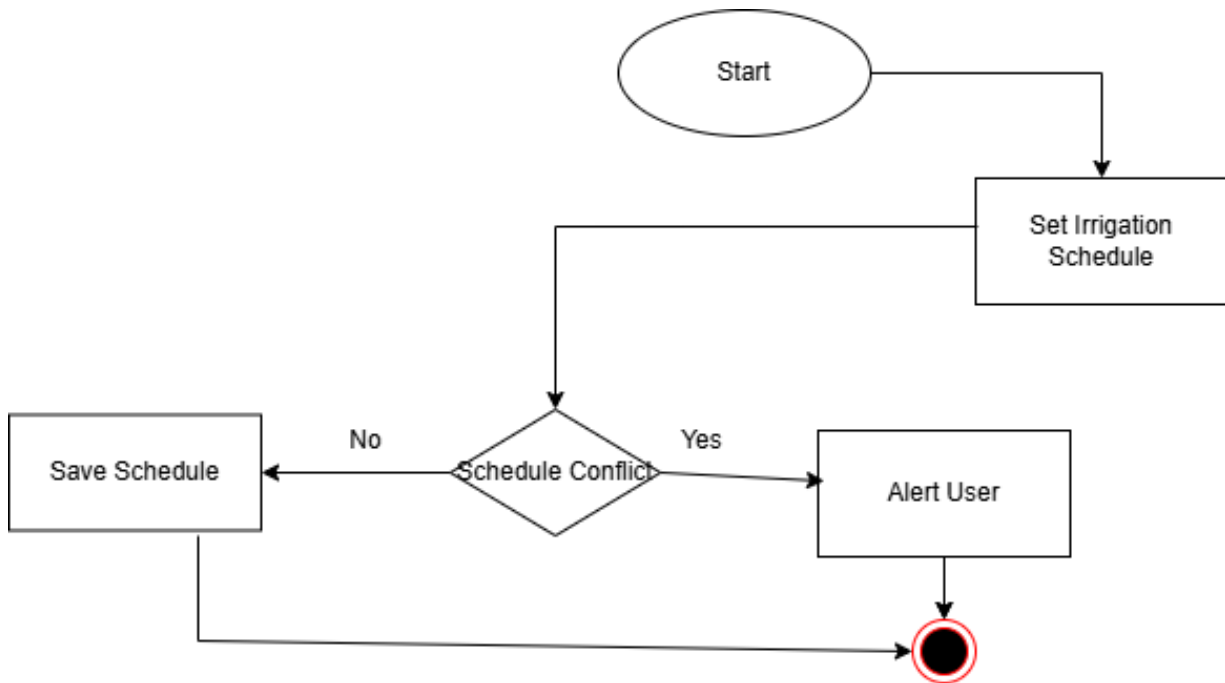


Figure 37 Activity Diagram Irrigation Scheduling

4.5.13 Log Irrigation Activity

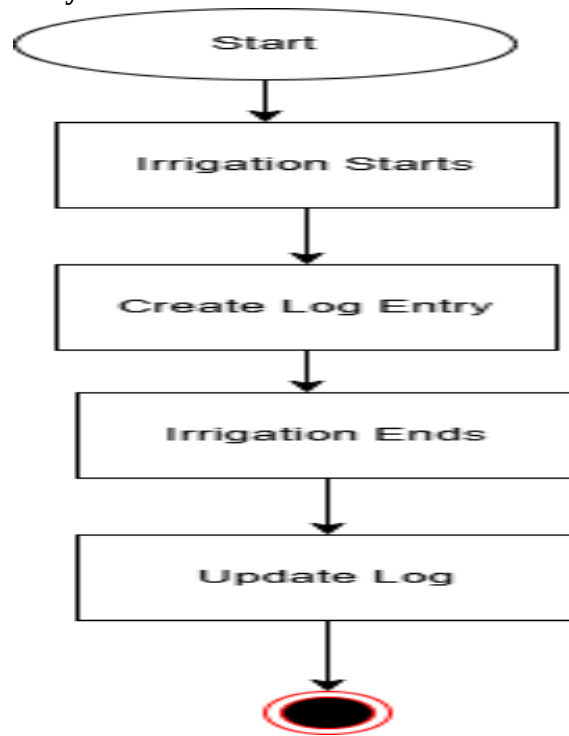


Figure 38 Activity Diagram Log Irrigation

4.5.14 Generate Reports & Analytics

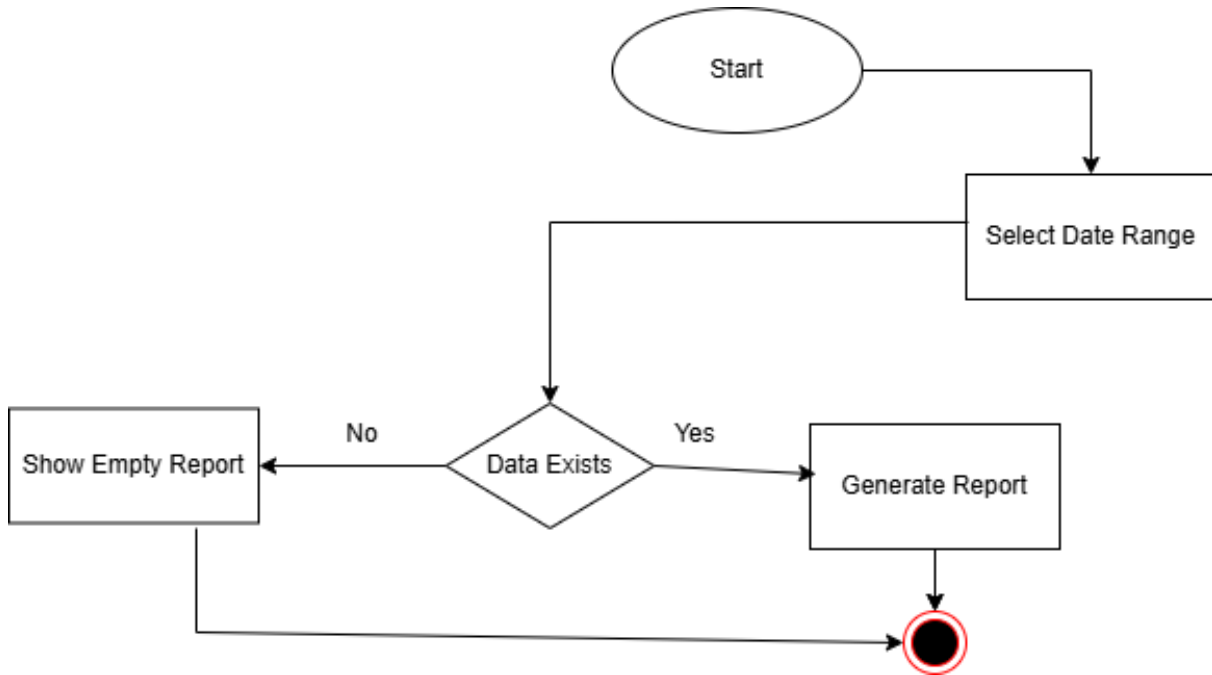


Figure 39 Activity Diagram Generate Report

4.5.15 Alerts & Notification

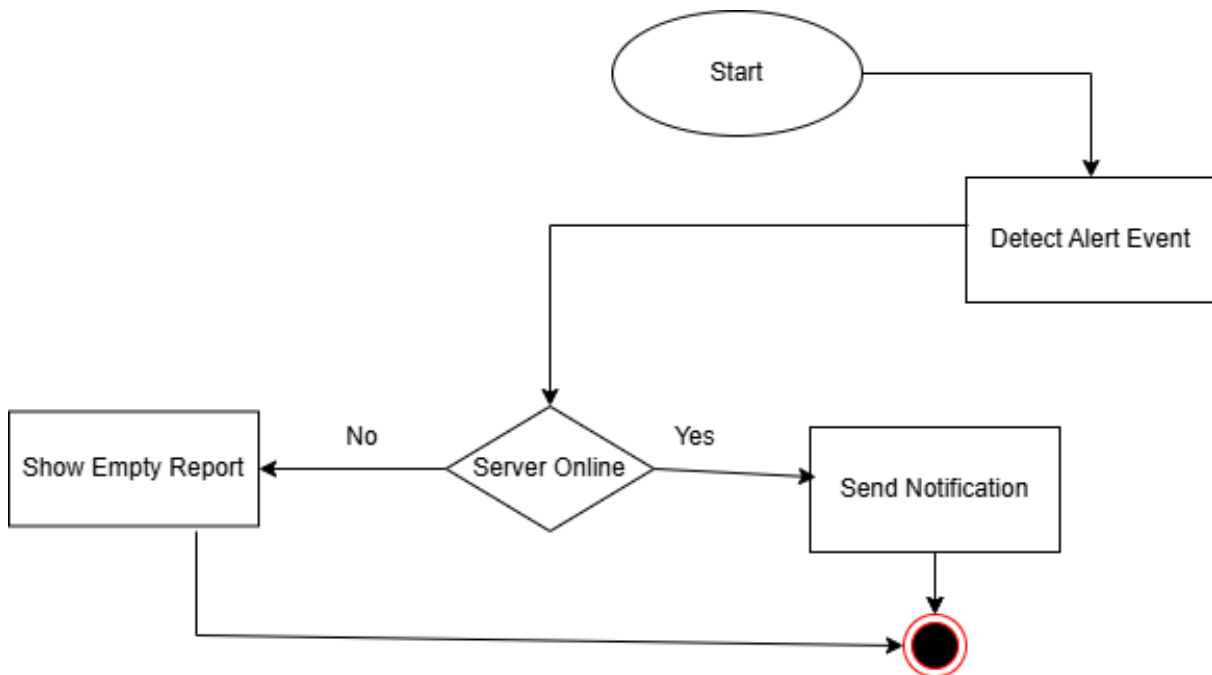


Figure 40 Activity Diagram Alert Notification

4.5.16 Device Health Monitoring

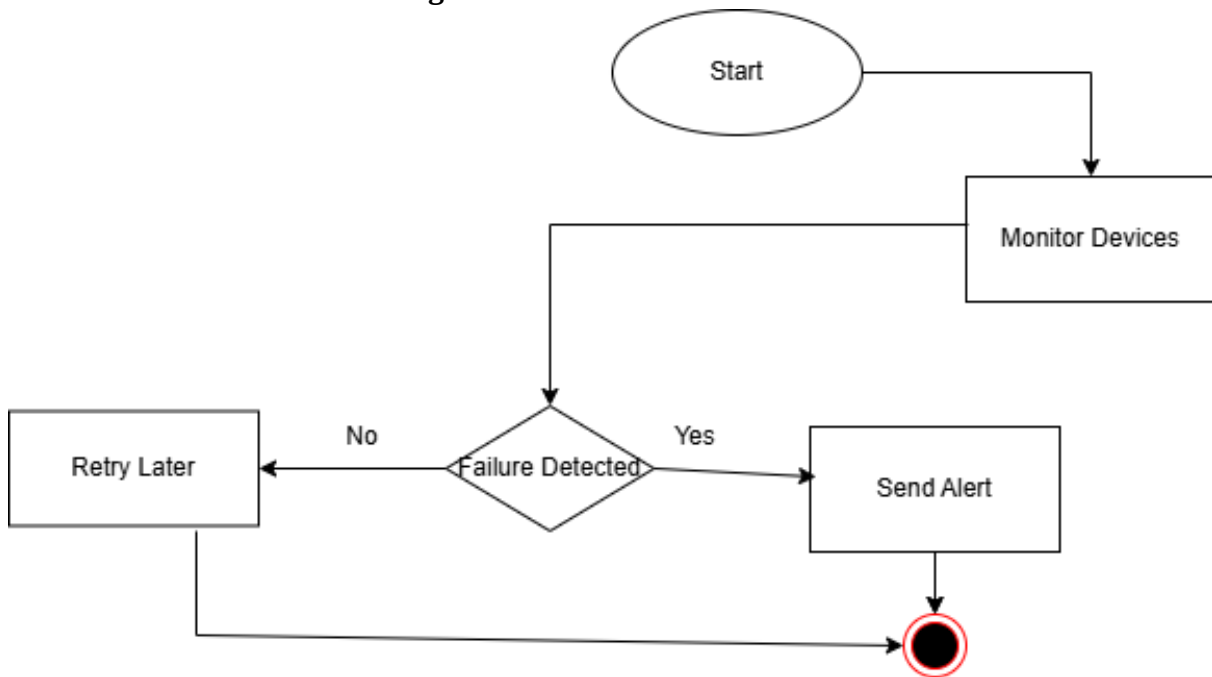


Figure 41 Activity Diagram Device Health Monitoring

4.5.17 System Configuration

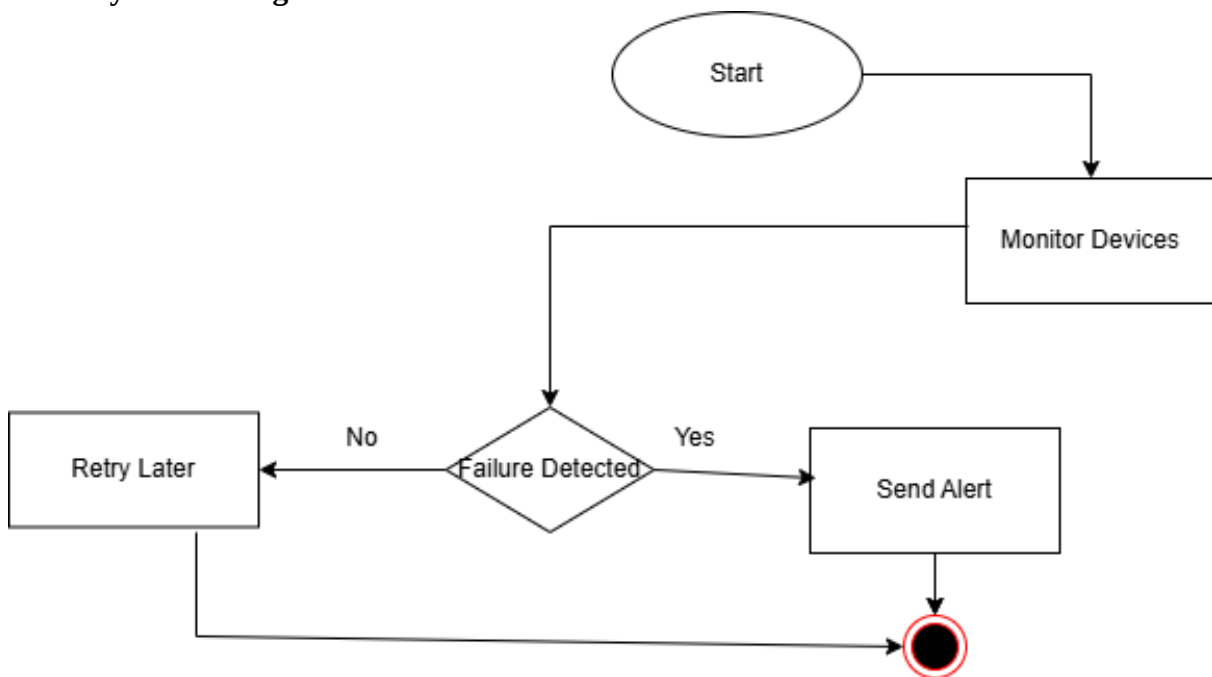


Figure 42 Activity Diagram System Configuration

4.5.18 IOT Controller Setup

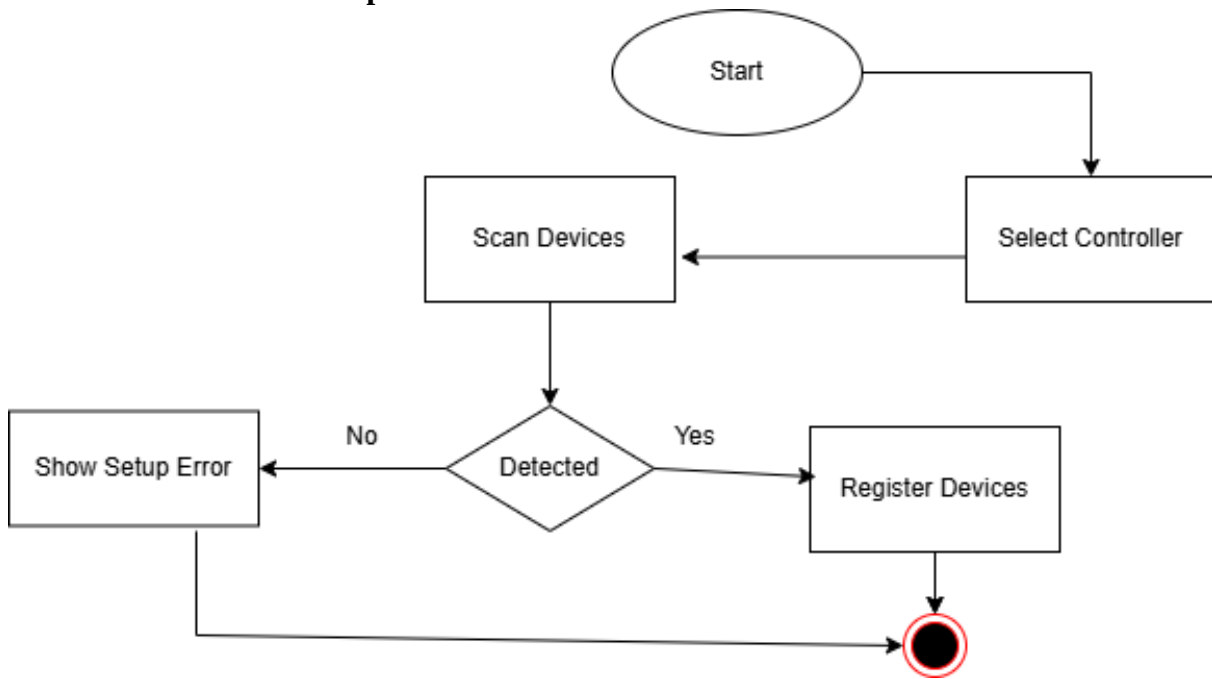


Figure 43 Activity Diagram IOT Controller Setup

4.5.19 Data Backup & Restore

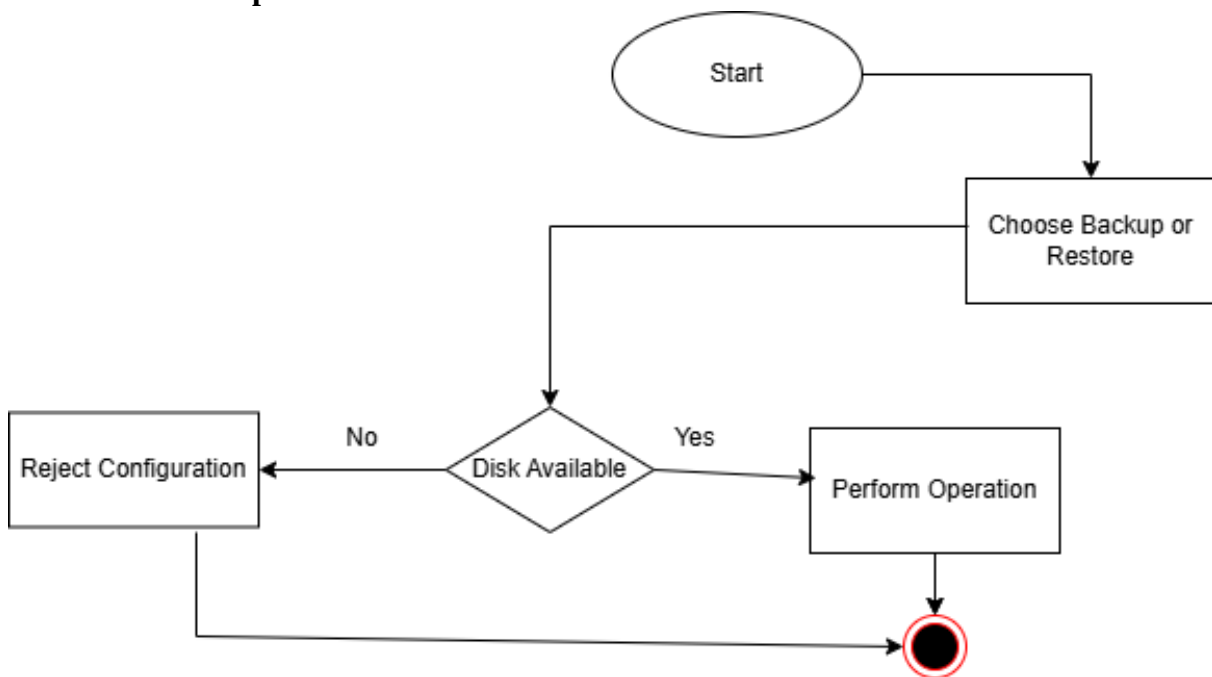


Figure 44 Activity Diagram Data Backup

4.5.20 Logout

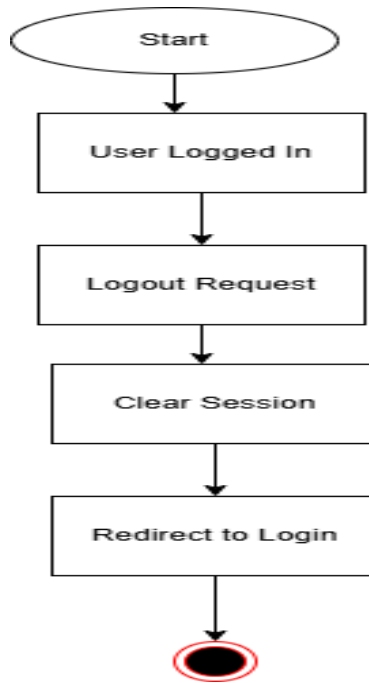


Figure 45 Activity Diagram Logout

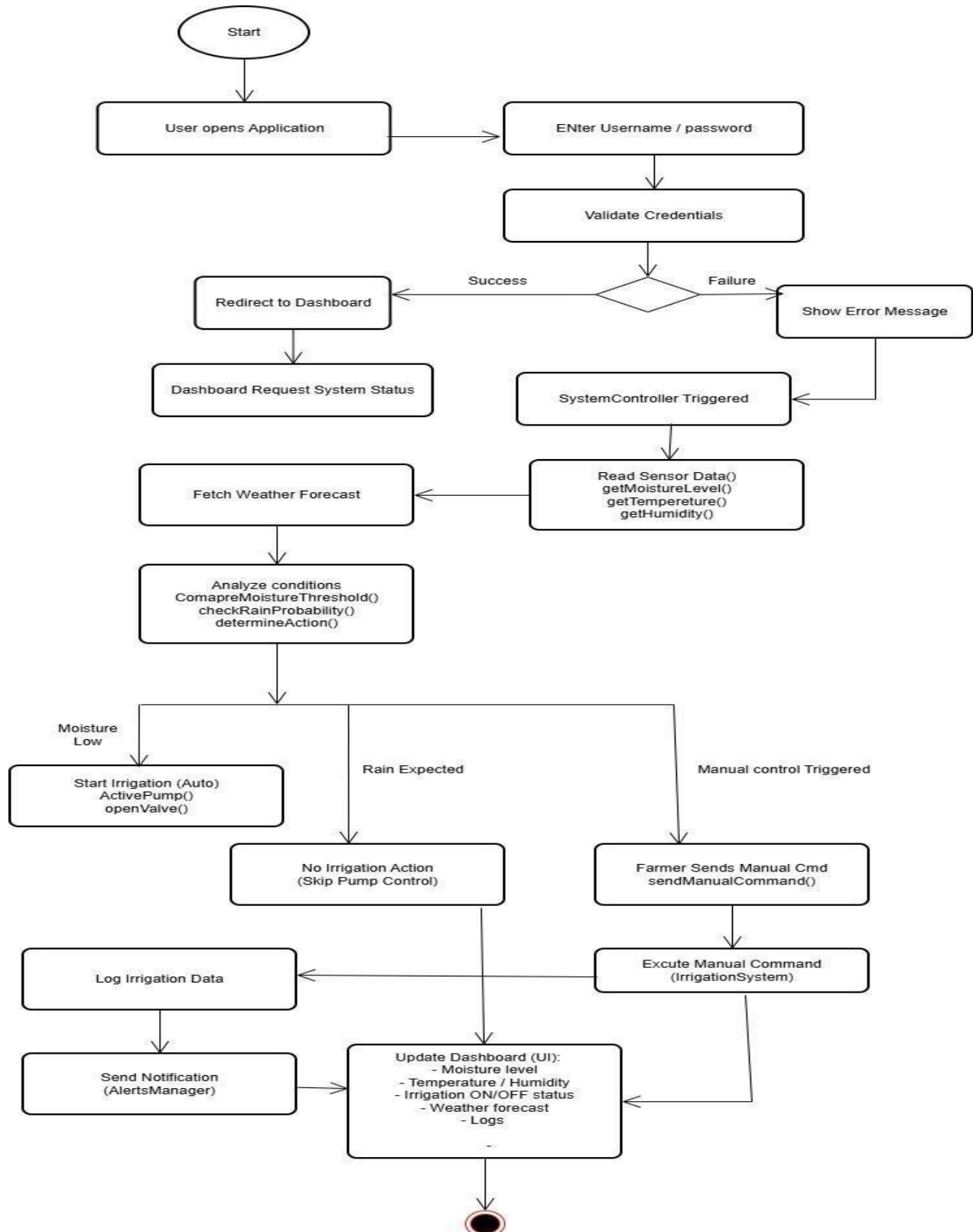


Figure 46 Activity Diagram Create

4.6 Sequence Diagram

The Sequence Diagram presents the time-ordered interaction between system components during an irrigation cycle. The process begins when a Farmer opens the dashboard, which sends a request to the System Controller to fetch system status. The System Controller interacts with the Sensor Module to retrieve moisture, humidity, and Level readings and then contacts the Weather Service to obtain forecast data. The collected data is passed to the Decision Engine, which analyzes environmental conditions and returns a decision to irrigate or not. If irrigation is required, the System Controller sends a command to the Irrigation System to activate pumps and valves. The system logs the irrigation details in the database and returns updated data to the Dashboard for the Farmer to view. This diagram visually represents the dynamic behavior of the system and ensures that each module communicates in the correct sequence, supporting accuracy and reliability during implementation.

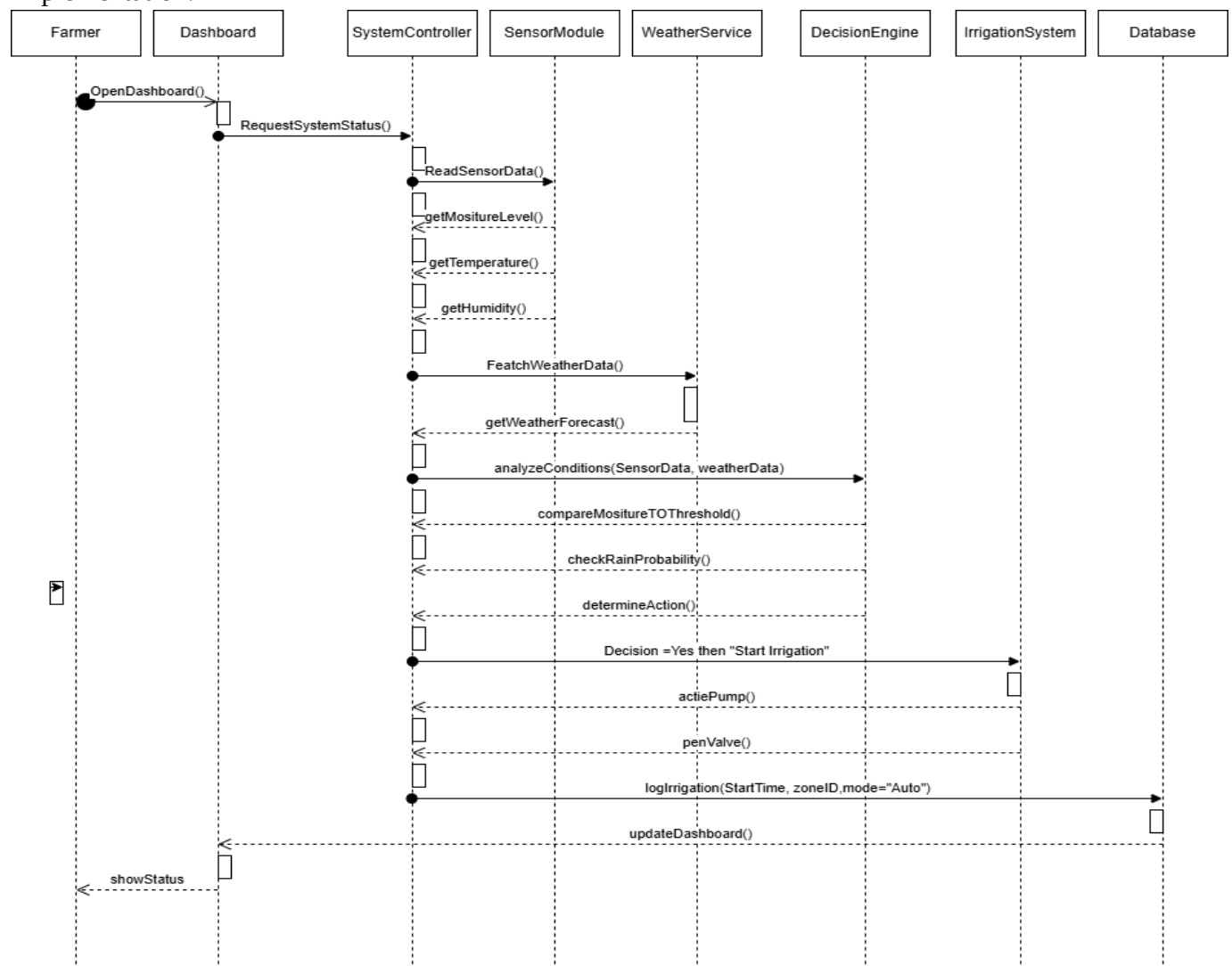


Figure 47 Sequence Diagram

4.7 Collaboration Diagram

The Collaboration Diagram (also known as the Communication Diagram) illustrates how the system objects interact and collaborate to accomplish irrigation tasks. Unlike the Sequence Diagram, which focuses on timing, the Collaboration Diagram highlights structural organization and message exchanges among system components. It shows the Farmer object interacting with the Dashboard, which communicates with the System Controller. The System Controller collaborates with the Sensor Module to obtain environmental readings and uses the Decision Engine to analyze conditions. Based on the decision, it interacts with the Irrigation System to control pumps and valves and with the Database to log irrigation activities. This diagram emphasizes the network of relationships and message flow between objects, providing a clear picture of how responsibilities are distributed among system components and ensuring that each object's role is well-defined and consistent with the system architecture.

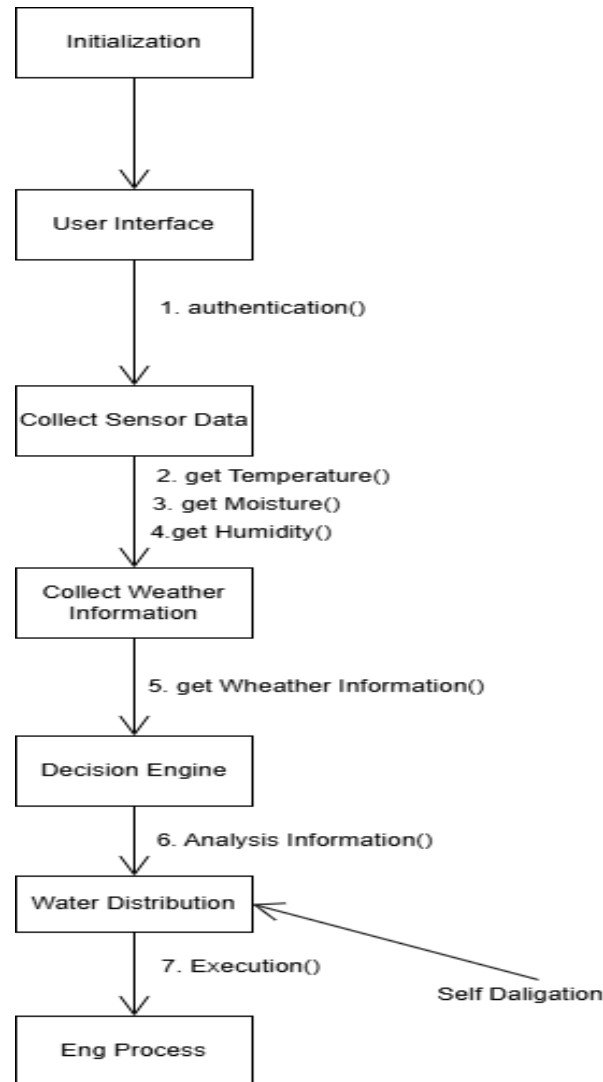


Figure 48 Collaboration Diagram

4.8 State Transition Diagram

The State Transition Diagram illustrates how the smart irrigation process moves between different operational states based on environmental conditions and system events. The system begins in the Idle State, where it continuously monitors soil moisture, Level, and humidity. When the moisture level falls below the defined threshold, the system transitions to the Irrigation Decision State, where it checks crop requirements and weather forecasts. If irrigation is needed, the system enters the Irrigation Active State, activating pumps or valves through the IoT controller. During irrigation, sensor readings are updated, and once optimal moisture is reached, the system moves to the Irrigation Complete State. After completing irrigation, the system logs activity and returns to the Idle State. If any device fault or sensor failure occurs, the system transitions to the Error State, where alerts are generated for users or Admins. Once errors are resolved, the system re-enters

normal operation. This diagram demonstrates how the system responds to environmental inputs and internal events, ensuring automated and efficient irrigation control.

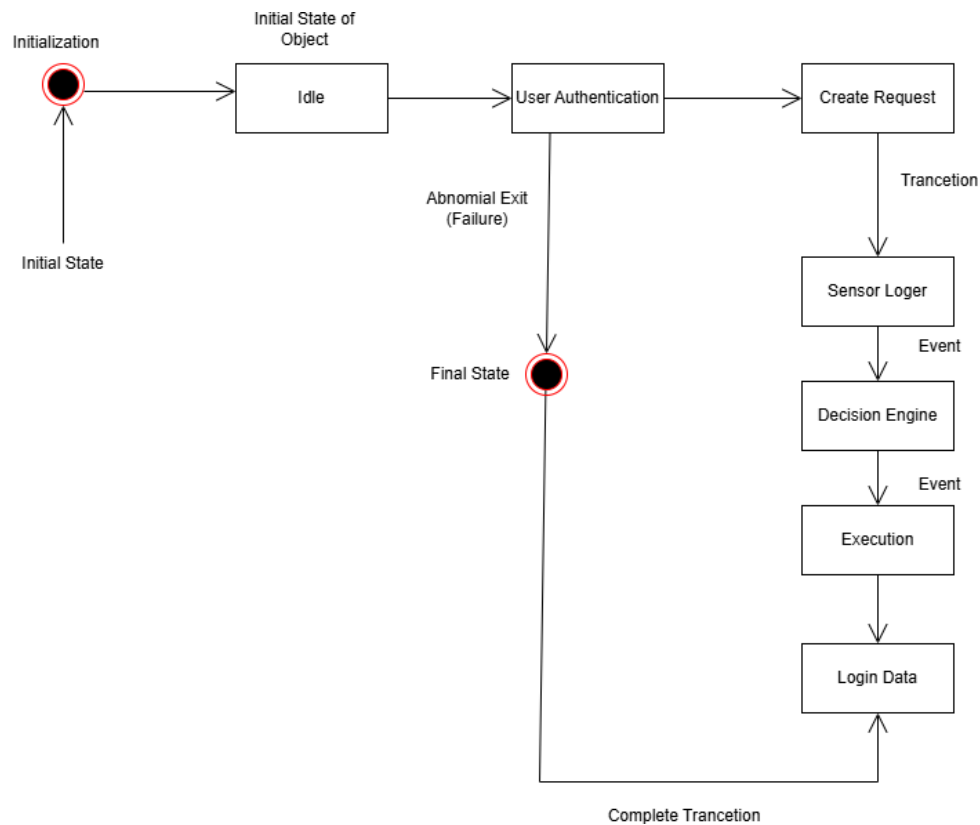


Figure 49 State Transition Diagram

4.9 Component Diagram

The Component Diagram represents the physical and logical software components of the Smart Irrigation System and illustrates how these components interact through well-defined interfaces. The system is divided into modular components to ensure scalability, maintainability, and ease of deployment. At the user end, the Weber Interface Component allows farmers and Admins to monitor field conditions, configure irrigation zones, and view reports. This interface communicates with the Application Server Component, which acts as the core processing unit of the system.

The Application Server Component includes sub-components such as User Management, Field & Zone Management, Decision Engine, and Notification Service. It processes real-time sensor data received from the IoT Communication Component, which handles communication between the Arduino-based IoT controller and the server using Ethernet protocols. The Weather API Integration Component fetches real-time and forecasted weather data and provides it to the decision engine for intelligent irrigation decisions. All system data, including sensor readings, user configurations, and

irrigation logs, are stored and managed by the Database Component. Each component interacts through defined interfaces, ensuring loose coupling and reliable system integration.

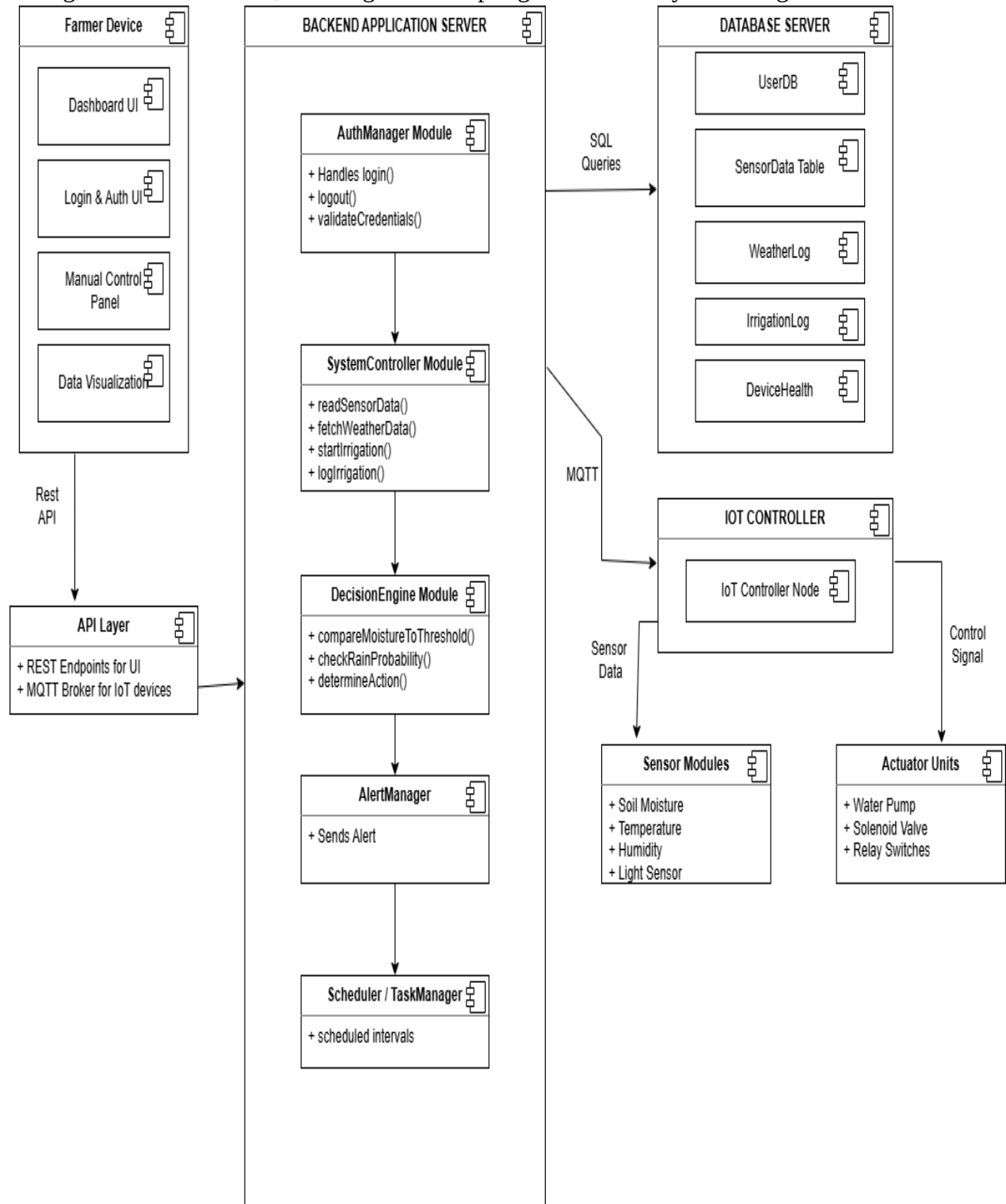


Figure 50 Component Diagram

4.10 Deployment Diagram

The Deployment Diagram illustrates the physical deployment of system components across hardware nodes and network environments. It shows how the Smart Irrigation System is deployed in a real-world agricultural setup. The User Device Node (smartphone, tablet, or PC) hosts the web application, allowing users to interact with the system through the internet. This device communicates with the Web Server Node, which hosts the backend services, decision logic, and APIs.

The Application Server Node is connected to the Database Server Node, which stores all persistent system data such as sensor readings, irrigation schedules, user profiles, and historical logs. In the field environment, the IoT Controller Node (Arduino with Ethernet Shield) is deployed near the irrigation infrastructure. This node connects to multiple Sensor Nodes (soil moisture, Level, humidity) and Actuator Nodes (pumps and solenoid valves). The IoT controller communicates with the application server over a wired Ethernet or local network. External services, such as the Weather API Server, are accessed via the internet to enhance decision-making. This deployment ensures reliability, scalability, and efficient real-time operation.

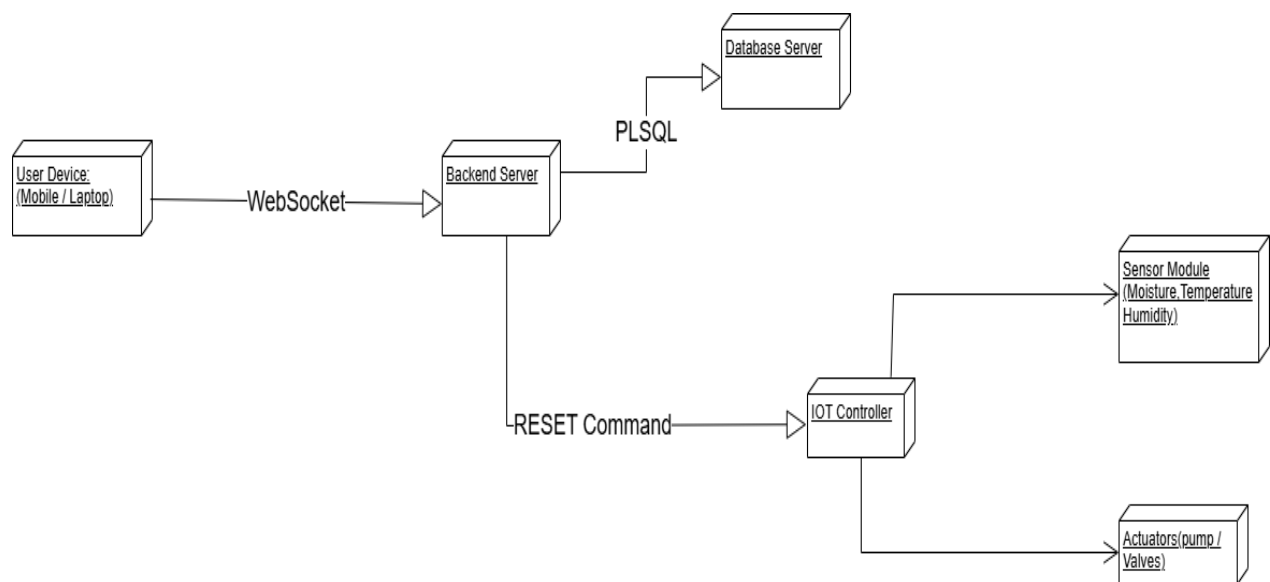


Figure 51 Deployment Diagram

Chapter 5: Testing

In this chapter, the full testing performed for the IoT-Based Smart Irrigation System is discussed. The aim was to ensure that all functional requirements are working as intended, that edge cases and failure scenarios are handled, and that the system behaves under realistic load. Testing was performed in layers: first functional test cases, then black-box and white-box techniques, and finally performance and regression checks.

5.1 Test Case Descriptions

The following test cases cover all 18 functional requirements of the IoT-based Smart Irrigation System. Each requirement has a positive test case to verify correct operation, and a negative test case to verify the system handles bad input or failure conditions without breaking down.

A. Test Case 1: FR_01 – Create Account

Positive Test Case	
Test Case ID	TC_FR01_POS
Priority	High
Description	Check that a new user account gets created without any issues when the admin fills in all required details correctly.
Reference	FR_01 – Create Account
Users	Administrator
Pre-Requisites	A. System is running and reachable at http://localhost:4000 . B. Admin is already logged in. C. The email being registered does not exist in the system yet.
Steps	A. Go to the User Management section from the admin panel. B. Click on Create New Account. C. Enter a valid name, email address, strong password, and select a role (Farmer). D. Hit the Submit button.
Input	Name: Muhammad Ahmad, Email: 70135149@student.uol.edu.pk , Password: ahmad123, Role: Farmer
Expected Result	The account is created without errors. A success notification shows up. The new user appears in the users list and can log in.
Status	Tested, Passed.

Table 79 Create Account Positive Test Case

Negative Test Case	
Test Case ID	TC_FR01_NEG
Priority	High
Description	Verify that the system does not allow account creation if required fields are missing or if the email is already taken.
Reference	FR_01 – Create Account

Users	Administrator
Pre-Requisites	A. Admin is logged in. B. System is online. C. The email admin@smartirrigation.local already exists in the database.
Steps	A. Open the Create Account form. B. Try submitting with the email field empty. C. Then try using an email that is already registered. D. Observe the system response in both cases.
Input	Case 1: Empty email field. Case 2: Email: admin@smartirrigation.local (already exists).
Expected Result	Case 1 — System shows a validation error asking to fill in the email. Case 2 — System displays a message that this email is already in use. No duplicate account is created.
Status	Tested, Passed.

Table 80 Create Account Negative Test Case

B. Test Case 2: FR_02 – Login

Positive Test Case	
Test Case ID	TC_FR02_POS
Priority	High
Description	Confirm that a registered user can log into the system when they enter the correct email and password.
Reference	FR_02 – Login
Users	Farmer / Administrator
Pre-Requisites	A. The server is running. B. The user account is active in the database. C. Internet or local network is connected.
Steps	A. Open the browser and go to http://localhost:4000. B. Type in a valid email address. C. Enter the correct password. D. Press the Login button.
Input	Email: admin@smartirrigation.local, Password: Admin@123
Expected Result	Login is successful. The system redirects the user to the main dashboard. A session token is created and stored securely.
Status	Tested, Passed.

Table 81 Login Positive Test Case

Negative Test Case	
Test Case ID	TC_FR02_NEG
Priority	High
Description	Make sure the system blocks login attempts when wrong credentials are entered.
Reference	FR_02 – Login

Users	Farmer / Administrator
Pre-Requisites	A. System is online. B. A user tries to log in with a wrong password or unregistered email.
Steps	A. Go to the login page. B. Enter an email that is not registered. C. Also test by entering a correct email but wrong password. D. Click Login in both scenarios.
Input	Case 1: Email: ghost@nowhere.pk, Password: random123. Case 2: Email: admin@smartirrigation.local, Password: WrongPass
Expected Result	Login is denied in both cases. An error message saying Invalid credentials appears. The user stays on the login page and no session is created.
Status	Tested, Passed.

Table 82 Login Negative Test Case

C. Test Case 3: FR_03 – View Dashboard

Positive Test Case	
Test Case ID	TC_FR03_POS
Priority	High
Description	Confirm that the dashboard loads correctly and displays up-to-date sensor readings, irrigation status, and weather information after login.
Reference	FR_03 – View Dashboard
Users	Farmer
Pre-Requisites	A. User is logged in. B. At least one field and zone are registered. C. IoT simulator is running and sending data.
Steps	A. Log in to the system. B. Click on the Dashboard option from the navigation menu. C. Wait for the page to fully load. D. Check that sensor charts, zone status, and weather data are showing.
Input	No direct user input — the dashboard pulls data automatically.
Expected Result	Dashboard opens without errors. Soil moisture readings, temperature, irrigation status per zone, and current weather conditions are all displayed with live data.
Status	Tested, Passed.

Table 83 View Dashboard Positive Test Case

Negative Test Case	
Test Case ID	TC_FR03_NEG
Priority	High
Description	Check what the dashboard shows when no sensor data is available or the device is offline.
Reference	FR_03 – View Dashboard
Users	Farmer
Pre-Requisites	A. User is logged in.

	B. No IoT device is transmitting data. C. Simulator is stopped.
Steps	A. Stop the IoT simulator completely. B. Log in and open the dashboard. C. Observe what is shown in the sensor data area.
Input	No sensor data coming in.
Expected Result	Dashboard still opens without crashing. Sensor panels show a No Data Available message. An offline or disconnected alert is visible. No errors are thrown to the user.
Status	Tested, Passed.

Table 84 View Dashboard Negative Test Case

D. Test Case 4: FR_04 – Manage Field

Positive Test Case	
Test Case ID	TC_FR04_POS
Priority	High
Description	Test that a farmer can add a new field with all required details and that the field gets saved correctly.
Reference	FR_04 – Manage Field
Users	Farmer
Pre-Requisites	A. Farmer is logged into the system. B. The field name being used does not already exist.
Steps	A. Go to the Fields section. B. Click on Add New Field. C. Fill in the field name, location, and total area. D. Save the field.
Input	Field Name: North Wheat Block, Location: Kasur, Area: 5 Acres
Expected Result	Field is added and immediately appears in the fields list. The system confirms the save with a success message. The field is available for zone creation.
Status	Tested, Passed.

Table 85 Manage Field Positive Test Case

Negative Test Case	
Test Case ID	TC_FR04_NEG
Priority	High
Description	Verify that the system refuses to save a field when the name or required details are missing.
Reference	FR_04 – Manage Field
Users	Farmer
Pre-Requisites	A. Farmer is logged in.
Steps	A. Open the Add New Field form. B. Leave the field name empty. C. Try to save.

	D. Also test with just spaces in the name field.
Input	Field Name: (empty or spaces only), Location: blank
Expected Result	System shows a validation error message pointing out the missing field name. The field is not saved. The farmer is prompted to fill in the required details.
Status	Tested, Passed.

Table 86 Manage Field Negative Test Case

E. Test Case 5: FR_05 – Select Crop Type

Positive Test Case	
Test Case ID	TC_FR05_POS
Priority	High
Description	Confirm that when a farmer picks a crop type for a zone, the corresponding moisture thresholds are correctly applied to that zone.
Reference	FR_05 – Select Crop Type
Users	Farmer
Pre-Requisites	A. At least one zone exists under a field. B. Farmer is logged in. C. Crop types are loaded in the system.
Steps	A. Open the zone settings. B. Find the Crop Type selection option. C. Choose Wheat from the dropdown. D. Save the selection.
Input	Zone: Zone A, Crop Selected: Wheat
Expected Result	The crop type is saved to the zone. The decision engine picks up the moisture threshold values associated with wheat and applies them going forward. A confirmation message is shown.
Status	Tested, Passed.

Table 87 Select Crop Type Positive Test Case

Negative Test Case	
Test Case ID	TC_FR05_NEG
Priority	High
Description	Check that saving a zone without selecting any crop type is handled properly by the system.
Reference	FR_05 – Select Crop Type
Users	Farmer
Pre-Requisites	A. Farmer is logged in. B. A zone exists but no crop has been assigned.
Steps	A. Open zone settings. B. Leave the crop type field unselected. C. Try to save.
Input	Crop Type: None selected

Expected Result	System either prompts the farmer to pick a crop type or saves the zone with default thresholds and shows a warning. No zone should be left in an inconsistent state.
Status	Tested, Passed.

Table 88 Select Crop Type Negative Test Case.

F. Test Case 6: FR_06 – Read Sensor Data

Positive Test Case	
Test Case ID	TC_FR06_POS
Priority	High
Description	Check that the system receives and stores sensor readings sent from the IoT simulator and displays them on the dashboard.
Reference	FR_06 – Read Sensor Data
Users	System / Farmer
Pre-Requisites	A. Server is running. B. A device key has been registered via the dashboard. C. IoT simulator is configured with the correct device key and zone ID.
Steps	A. Start the simulator using: DEVICE_KEY=dk_device key ZONE_ID=2 INTERVAL_SEC=5 node simulator.js B. Open the dashboard in a browser. C. Navigate to the zone sensor view. D. Wait 10–15 seconds and observe incoming data.
Input	Soil Moisture: 48%, Temperature: 31°C, Humidity: 60%
Expected Result	Readings appear on the dashboard and update every few seconds. The data is stored in the database. Charts reflect the incoming values in real time.
Status	Tested, Passed.

Table 89 Read Sensor Data Positive Test Case

Negative Test Case	
Test Case ID	TC_FR06_NEG
Priority	High
Description	See how the system handles a situation where the IoT device stops sending data or sends data with an invalid device key.
Reference	FR_06 – Read Sensor Data
Users	System
Pre-Requisites	A. Server is running. B. Simulator is intentionally stopped or configured with a wrong device key.
Steps	A. Stop the simulator abruptly. B. Wait for the heartbeat timeout period. C. Also test by running the simulator with a device key that is not registered. D. Check dashboard behavior.
Input	Case 1: No data sent for over 2 minutes. Case 2: DEVICE_KEY=dk_invalid123

Expected Result	Case 1 — Dashboard shows the device as Offline and triggers a device health alert. Case 2 — Server returns 401 Unauthorized and rejects the reading. No data is stored.
Status	Tested, Passed.

Table 90 Read Sensor Data Negative Test Case

G. Test Case 7: FR_07 – Fetch Weather Data

Positive Test Case	
Test Case ID	TC_FR07_POS
Priority	High
Description	Verify that the system successfully contacts the OpenWeatherMap API and retrieves current weather and forecast data for the configured city.
Reference	FR_07 – Fetch Weather Data
Users	System
Pre-Requisites	A. A valid OpenWeatherMap API key is set in the .env file. B. Internet connection is active. C. WEATHER_DEFAULT_CITY is set to Lahore,PK in .env.
Steps	A. Start the backend server. B. Allow the system to make its scheduled weather fetch. C. Open the dashboard weather section. D. Check the weather panel for current data.
Input	API Request to OpenWeatherMap for Lahore, PK
Expected Result	Weather data fetched without errors. Dashboard shows current temperature, humidity, and a rain forecast. The decision engine receives the updated weather context.
Status	Tested, Passed.

Table 91 Fetch Weather Data Positive Test Case

Negative Test Case	
Test Case ID	TC_FR07_NEG
Priority	High
Description	Test what happens when the weather API is unreachable or the API key is invalid.
Reference	FR_07 – Fetch Weather Data
Users	System
Pre-Requisites	A. Either API key is intentionally set to a wrong value or internet is disconnected.
Steps	A. Set WEATHER_API_KEY to an invalid string in .env. B. Restart the server. C. Wait for the weather fetch cycle. D. Check the logs and dashboard.
Input	Invalid API Key: WEATHER_API_KEY=invalidkeytest123
Expected Result	System logs the API failure. Dashboard shows last known weather data or a warning that weather data is unavailable. The decision engine continues working using cached forecast data. No server crash occurs.

Status	Tested, Passed.
--------	-----------------

Table 92 Fetch Weather Data Negative Test Case

H. Test Case 8: FR_08 – Automatic Irrigation

Positive Test Case	
Test Case ID	TC_FR08_POS
Priority	High
Description	Confirm that the decision engine automatically starts irrigation for a zone when soil moisture drops below the configured threshold and no rain is expected.
Reference	FR_08 – Automatic Irrigation
Users	System / Decision Engine
Pre-Requisites	A. Sensors are transmitting data. B. Moisture threshold is set (e.g., 35%) for the zone. C. Weather forecast shows no rain. D. Automatic mode is active.
Steps	A. Set the zone moisture threshold to 35%. B. Let the simulator gradually reduce moisture to 28%. C. Wait for the next decision engine cycle (every 15 seconds). D. Observe the irrigation status on the dashboard.
Input	Current Moisture: 28%, Threshold: 35%, Rain Forecast: 5% probability
Expected Result	System triggers irrigation automatically. Zone status changes to Irrigating on the dashboard. An irrigation event is logged in the database. Moisture starts rising in the simulator.
Status	Tested, Passed.

Table 93 Automatic Irrigation Postive Test Case

Negative Test Case	
Test Case ID	TC_FR08_NEG
Priority	High
Description	Make sure the system does not start irrigation when rainfall is already forecasted, even if moisture is below the threshold.
Reference	FR_08 – Automatic Irrigation
Users	System / Decision Engine
Pre-Requisites	A. Moisture is below threshold. B. Weather API returns a high rain probability (above 60%).
Steps	A. Configure weather service to return rain probability of 75%. B. Let moisture drop below threshold. C. Wait for the decision engine cycle. D. Check if irrigation activates.
Input	Moisture: 28%, Threshold: 35%, Rain Forecast: 75% probability
Expected Result	System skips irrigation. A log entry is created showing Irrigation Skipped — Rain Forecasted. Zone stays in idle state. No pump or valve activation happens.

Status	Tested, Passed.
--------	-----------------

*Table 94 Automatic Irrigation Negative Test Case***I. Test Case 9: FR_09 – Manual Irrigation Control**

Positive Test Case	
Test Case ID	TC_FR09_POS
Priority	High
Description	Test that a farmer can manually turn irrigation on for a selected zone directly from the dashboard.
Reference	FR_09 – Manual Irrigation Control
Users	Farmer
Pre-Requisites	A. Farmer is logged in. B. At least one zone is registered. C. IoT controller or simulator is active.
Steps	A. Open the dashboard and go to the Irrigation Control section. B. Select the desired zone. C. Switch the irrigation toggle to ON. D. Confirm the action when prompted.
Input	Zone: Zone B, Command: Irrigation ON
Expected Result	Irrigation starts for Zone B immediately. Status on the dashboard updates to Active. An irrigation log entry is created with the start time, zone, and mode as Manual.
Status	Tested, Passed.

Table 95 Manual Irrigation Control Positive Test Case

Negative Test Case	
Test Case ID	TC_FR09_NEG
Priority	High
Description	Verify system response when a manual irrigation command is sent but the device is not responding.
Reference	FR_09 – Manual Irrigation Control
Users	Farmer
Pre-Requisites	A. Farmer is logged in. B. The IoT device for the selected zone is offline.
Steps	A. Stop the simulator for a specific zone. B. Try to manually turn on irrigation from the dashboard for that zone. C. Observe the response.
Input	Zone: Zone C (device offline), Command: Irrigation ON
Expected Result	Dashboard shows an error or warning that the device is not reachable. Irrigation is not activated. The system logs the failed command attempt.
Status	Tested, Passed.

Table 96 Manual Irrigation Control Negative Test Case

J. Test Case 10: FR_10 – Irrigation Scheduling

Positive Test Case	
Test Case ID	TC_FR10_POS
Priority	High
Description	Confirm that a farmer can set a scheduled irrigation time and that the system executes irrigation automatically at that time.
Reference	FR_10 – Irrigation Scheduling
Users	Farmer
Pre-Requisites	A. Farmer is logged in. B. A zone is configured. C. System clock is correctly set.
Steps	A. Go to the Schedules section. B. Click Add New Schedule. C. Select the zone, set a time 2 minutes from now, and set a duration of 5 minutes. D. Save the schedule and wait.
Input	Zone: Zone A, Time: [current time + 2 min], Duration: 5 minutes, Days: Daily
Expected Result	Irrigation activates automatically at the scheduled time. A log entry is created. After 5 minutes, irrigation stops automatically. The schedule remains active for the next day.
Status	Tested, Passed.

Table 97 Irrigation Scheduling Positive Test Case

Negative Test Case	
Test Case ID	TC_FR10_NEG
Priority	High
Description	Test that the system rejects or handles a schedule entry with conflicting times or invalid values.
Reference	FR_10 – Irrigation Scheduling
Users	Farmer
Pre-Requisites	A. Farmer is logged in. B. A schedule already exists for Zone A at 08:00.
Steps	A. Try to add another schedule for Zone A at 08:00. B. Also try saving a schedule with duration set to zero. C. Observe system response.
Input	Case 1: Duplicate schedule time for the same zone. Case 2: Duration = 0 minutes.
Expected Result	Case 1 — System shows a conflict warning and does not save the duplicate. Case 2 — Validation error for zero duration. Both invalid entries are rejected.
Status	Tested, Passed.

Table 98 Irrigation Scheduling Negative Test Case

K. Test Case 11: FR_11 – Activity Logging

Positive Test Case	
Test Case ID	TC_FR11_POS
Priority	High
Description	Confirm that all irrigation events, sensor data submissions, and system actions are automatically recorded in the activity log.
Reference	FR_11 – Activity Logging
Users	System
Pre-Requisites	A. System is running. B. An irrigation event has occurred (manual or automatic). C. Sensor readings have been submitted.
Steps	A. Perform a manual irrigation start. B. Let the simulator send a few sensor readings. C. Log in as Admin. D. Navigate to the Activity Logs section.
Input	Action: Irrigation started manually for Zone A. Sensor reading submitted by device dk_device key.
Expected Result	All events are recorded with accurate timestamps, user or system source, event type, and relevant details. Logs are visible and filterable in the admin panel.
Status	Tested, Passed.

Table 99 Activity Logging Positive Test Case

Negative Test Case	
Test Case ID	TC_FR11_NEG
Priority	High
Description	Verify that the system handles a scenario where the database is temporarily unavailable during a log write operation.
Reference	FR_11 – Activity Logging
Users	System
Pre-Requisites	A. System is running. B. Database connection is intentionally interrupted during an event.
Steps	A. Simulate a DB connection drop. B. Trigger an irrigation event. C. Restore the DB connection. D. Check whether the event was captured.
Input	DB unavailable during irrigation event.
Expected Result	System does not crash. An error is logged internally. Once the DB reconnects, normal logging resumes. No partial or corrupt log entries are written.
Status	Tested, Passed.

Table 100 Activity Logging Negative Test Case

L. Test Case 12: FR_12 – Generate Reports

Positive Test Case	
Test Case ID	TC_FR12_POS
Priority	High
Description	Test that the system generates a downloadable PDF and CSV irrigation report for a given date range.
Reference	FR_12 – Generate Reports
Users	Farmer / Administrator
Pre-Requisites	A. At least one irrigation event exists in the database. B. User is logged in. C. Sensor readings have been stored for the selected period.
Steps	A. Go to the Reports section from the navigation menu. B. Select a date range covering the last 7 days. C. Click Generate PDF. D. Also click Export CSV. E. Open the downloaded files.
Input	Report Type: Irrigation Summary, Date Range: Last 7 days, Zone: All Zones
Expected Result	PDF report downloads and opens with irrigation events, timestamps, durations, and zone names. CSV file opens correctly in Excel with accurate column data. Both files reflect the same data.
Status	Tested, Passed.

Table 101 Generate Reports Positive Test Case

Negative Test Case	
Test Case ID	TC_FR12_NEG
Priority	High
Description	Check system behavior when a report is requested for a date range that has no recorded data.
Reference	FR_12 – Generate Reports
Users	Farmer
Pre-Requisites	A. No irrigation events exist for the selected date range. B. User is logged in.
Steps	A. Open the Reports section. B. Set a date range for a future period or a period with no events. C. Click Generate PDF.
Input	Date Range: Next month (no data available)
Expected Result	System informs the user that no data is available for the selected period. No empty or broken PDF is generated. A clear message is shown suggesting the user to change the date range.
Status	Tested, Passed.

Table 102 Generate Reports Negative Test Case

M. Test Case 13: FR_13 – Notifications

Positive Test Case	
Test Case ID	TC_FR13_POS
Priority	High
Description	Confirm that the system sends a real-time alert to the dashboard when a critical condition such as very low soil moisture is detected.
Reference	FR_13 – Notifications
Users	System / Farmer
Pre-Requisites	A. Dashboard is open in the browser with an active SSE connection. B. Alert thresholds are configured. C. Simulator is running.
Steps	A. Open the dashboard. B. Allow the simulator to drop soil moisture to a critically low level (below 15%). C. Wait for the next alert engine cycle. D. Observe the notifications panel.
Input	Soil Moisture: 12% (critical threshold breach)
Expected Result	A real-time notification pops up in the dashboard notifications panel without any page refresh. The alert shows the affected zone, the type of alert, and the timestamp.
Status	Tested, Passed.

Table 103 Notifications Positive Test Case

Negative Test Case	
Test Case ID	TC_FR13_NEG
Priority	High
Description	See how the system handles alert delivery when the browser SSE connection is lost.
Reference	FR_13 – Notifications
Users	System
Pre-Requisites	A. Dashboard was open but browser tab was closed or network dropped.
Steps	A. Close the dashboard tab while the system is running. B. Trigger a critical moisture alert. C. Reopen the dashboard. D. Check the notifications panel.
Input	SSE connection: disconnected. Alert triggered during downtime.
Expected Result	When the dashboard is reopened, previously missed alerts are shown in the notifications history. The system did not crash during the disconnect. New alerts continue to deliver after reconnection.
Status	Tested, Passed.

Table 104 Notifications Negative Test Case

N. Test Case 14: FR_14 – Device Health Monitoring

Positive Test Case	
Test Case ID	TC_FR14_POS
Priority	High
Description	Verify that the system correctly identifies all active devices as healthy when they are sending regular heartbeat signals.
Reference	FR_14 – Device Health Monitoring
Users	System / Administrator
Pre-Requisites	A. At least one device is registered. B. Simulator is running and sending data every 5 seconds. C. Device health service is active.
Steps	A. Register a device using the dashboard. B. Start the simulator for that device. C. Navigate to the Device Health or Devices section. D. Check the device status indicator.
Input	Device: Lenovo, Last Seen: within 1 minute
Expected Result	Device shows as Online or Healthy in the devices list. Last seen timestamp updates regularly. No health alerts are generated.
Status	Tested, Passed.

Table 105 Device Health Monitoring Positive Test Case

Negative Test Case	
Test Case ID	TC_FR14_NEG
Priority	High
Description	Test that the system raises a device offline alert when a registered device stops sending data for an extended period.
Reference	FR_14 – Device Health Monitoring
Users	System
Pre-Requisites	A. A device is registered. B. Simulator is stopped intentionally. C. Health check interval has passed.
Steps	A. Stop the simulator. B. Wait for the device heartbeat timeout (approximately 2 minutes). C. Check the dashboard and device list.
Input	No heartbeat received for over 2 minutes.
Expected Result	Device status changes to Offline. A device health alert appears in the notifications panel. Admin can see the last known active timestamp of the device.
Status	Tested, Passed.

*Table 106 Device Health Monitoring Negative Test Case***O. Test Case 15: FR_15 – System Configuration**

Positive Test Case	
Test Case ID	TC_FR15_POS

Priority	High
Description	Test that the admin can successfully update system-wide settings such as moisture thresholds, notification modes, and decision engine intervals.
Reference	FR_15 – System Configuration
Users	Administrator
Pre-Requisites	A. Admin is logged in. B. System configuration panel is accessible.
Steps	A. Navigate to the Admin panel and open System Configuration. B. Update the default moisture threshold to 40%. C. Change the decision engine cycle interval. D. Save changes.
Input	Default Moisture Threshold: 40%, Engine Interval: 20 seconds
Expected Result	Settings are saved and confirmed with a success message. The decision engine immediately applies the new threshold value in subsequent cycles. No restart is needed.
Status	Tested, Passed.

Table 107 System Configuration Positive Test Case

Negative Test Case	
Test Case ID	TC_FR15_NEG
Priority	High
Description	Check that the system prevents saving invalid configuration values such as a threshold above 100% or a negative interval.
Reference	FR_15 – System Configuration
Users	Administrator
Pre-Requisites	A. Admin is logged in.
Steps	A. Open System Configuration. B. Set moisture threshold to 150%. C. Set engine interval to -5 seconds. D. Try to save.
Input	Moisture Threshold: 150%, Engine Interval: -5 seconds
Expected Result	System shows validation errors for both fields. Invalid values are not saved. Current configuration remains unchanged.
Status	Tested, Passed.

Table 108 System Configuration Negative Test Case

P. Test Case 16: FR_16 – IoT Controller Setup

Positive Test Case	
Test Case ID	TC_FR16_POS
Priority	High
Description	Confirm that a new IoT device can be registered in the system and a unique device key is issued for it.
Reference	FR_16 – IoT Controller Setup
Users	Administrator

Pre-Requisites	A. Admin is logged in. B. No device with the same name is already registered.
Steps	A. Open the Devices section. B. Click Add New Device. C. Enter a device name and assign it to a zone. D. Save the device.
Input	Device Name: Field Sensor Unit 3, Assigned Zone: Zone B
Expected Result	Device is registered. A unique device key (dk_<device key>) is generated and displayed. The device appears in the devices list and is ready to accept sensor data.
Status	Tested, Passed.

Table 109 IoT Controller Setup Positive Test Case

Negative Test Case	
Test Case ID	TC_FR16_NEG
Priority	High
Description	Verify that the system prevents registering a device with an empty name or assigning it to a zone that does not exist.
Reference	FR_16 – IoT Controller Setup
Users	Administrator
Pre-Requisites	A. Admin is logged in.
Steps	A. Try to register a device with an empty name. B. Try to assign it to a zone ID that does not exist. C. Attempt to save in both cases.
Input	Device Name: (empty), Zone ID: 9999 (non-existent)
Expected Result	System rejects both entries with relevant validation errors. No device key is generated. The devices list remains unchanged.
Status	Tested, Passed.

Table 110 IoT Controller Setup Negative Test Case

Q. Test Case 17: FR_17 – Backup and Restore

Positive Test Case	
Test Case ID	TC_FR17_POS
Priority	High
Description	Test that the admin can trigger a system backup and that the backup file is correctly generated and downloadable.
Reference	FR_17 – Backup and Restore
Users	Administrator
Pre-Requisites	A. Admin is logged in. B. System has data including users, fields, zones, and logs.
Steps	A. Navigate to the Backup and Restore section. B. Click Create Backup. C. Wait for the process to complete. D. Download the generated backup file.

Input	Backup Command: Trigger full system backup
Expected Result	Backup file is generated and made available for download. A success message confirms the backup completion. The file includes all relevant system data.
Status	Tested, Passed.

Table 111 Backup and Restore Positive Test Case

Negative Test Case	
Test Case ID	TC_FR17_NEG
Priority	High
Description	Test how the system behaves when a restore is attempted using a corrupted or incompatible backup file.
Reference	FR_17 – Backup and Restore
Users	Administrator
Pre-Requisites	A. Admin is logged in. B. A deliberately corrupted backup file is prepared for testing.
Steps	A. Go to the Backup and Restore section. B. Upload the corrupted backup file. C. Click Restore. D. Observe system response.
Input	Corrupted backup file (manually broken JSON or SQL structure)
Expected Result	System detects the file integrity issue. A clear error message is shown. The restore operation is cancelled. Existing live data remains intact and unaffected.
Status	Tested, Passed.

Table 112 Backup and Restore Negative Test Case

R. Test Case 18: FR_18 – Logout

Positive Test Case	
Test Case ID	TC_FR18_POS
Priority	High
Description	Confirm that a logged-in user can successfully log out and that their session is fully terminated.
Reference	FR_18 – Logout
Users	Farmer / Administrator
Pre-Requisites	A. User is currently logged in and active on the dashboard.
Steps	A. Click the Logout button from the top navigation. B. Confirm logout if a prompt appears. C. Observe the redirection.
Input	Action: Logout clicked by logged-in user
Expected Result	User is logged out immediately. Session token is invalidated. Browser redirects to the login page. Attempting to go back using the back button does not restore the session.
Status	Tested, Passed.

Table 113 Logout Positive Test Case

Negative Test Case	
Test Case ID	TC_FR18_NEG
Priority	High
Description	Verify that a user whose session has already expired cannot access dashboard routes directly.
Reference	FR_18 – Logout
Users	Farmer / Administrator
Pre-Requisites	A. User was previously logged in. B. Session has expired or been manually invalidated on the server.
Steps	A. Allow the JWT session to expire. B. Try to access http://localhost:4000/dashboard directly in the browser. C. Also try using an old token in an API request.
Input	Expired JWT token in request header
Expected Result	Server returns 401 Unauthorized. Browser redirects the user back to the login page. The old session token is not accepted for any API calls.
Status	Tested, Passed.

Table 114 Logout Negative Test Case

5.2 Black Box Test Cases

Black box testing, also known as Behavioral Testing, is a software testing method where the internal structure, design, or implementation of the item being tested is not known to the tester. These tests can be functional or non-functional, though usually functional. This method is named so because the software program, in the eyes of the tester, is like a black box — inside which one cannot see.

In case of an IoT-based smart irrigation system, black box testing has been used to ensure that the system produces the correct output for any input, ignoring how the code achieves this internally. It was used for authentication, sensor data processing, the decision engine, report generation, and notification delivery.

The method was used to find errors in the following categories:

- 18 functional requirements with incorrect or missing functions.
- Interface errors between frontend dashboard and backend REST API.
- Errors accessing data structures or databases such as sensor readings and irrigation events tables.
- Decision engine and alert engine cycle behavior or performance errors.
- Irrigation event and scheduled job start/stop errors

5.2.1 Equivalence Partitions (EP)

Equivalence class partitioning (EP) is a commonly used technique to reduce the number of test cases required to achieve good coverage. The idea is , if one value of a group works , then all values of that group work the same way . Instead of testing all possible inputs, one from each group is sufficient.

Equivalence partitioning was applied on the main input variables of the Smart Irrigation System such as login, sensor data ingestion, device registration & scheduling. The table below shows the valid and invalid classes for each variable.

Variables	Valid Classes	Invalid Classes
Username / Email	1. Correct email format with @ and domain. 2. Already registered in the database. 3. Accepted case-insensitively.	1. Missing @ symbol or domain extension. 2. Not found in the database. 3. Completely empty field.
Password	1. Length between 8 and 64 characters. 2. Mix of letters, numbers, and symbols. 3. Matches stored bcrypt hash correctly.	1. Shorter than 8 characters. 2. Field is left empty. 3. Contains only whitespace characters.
Soil Moisture (%)	1. Integer value from 1 to 99. 2. Numeric reading from a real or simulated sensor.	1. Negative number such as -5. 2. Value above 100. 3. Non-numeric text string.
Device Key	1. Follows dk_ prefix format with alphanumeric suffix. 2. Registered and active in the devices table. 3. Passed correctly in the X-Device-Key header.	1. Does not match the dk_ format. 2. Not found in the database. 3. Empty or null value in the request header.
Schedule Duration (min)	1. Any whole number from 1 to 480. 2. Entered as a positive integer.	1. Set to zero. 2. Negative number. 3. Non-numeric characters.

Table 115 Equivalence Partitions for the Smart Irrigation System.

5.2.2 Boundary Value Analysis

Boundary Value Analysis (BVA) targets the values at the edges of each equivalence partition, where defects most commonly occur. The Smart Irrigation System is driven by numeric thresholds, so its decision boundaries are the most defect-prone inputs. For every boundary, the values tested are $\text{min}-1$, min , $\text{min}+1$, a nominal value, and $\text{max}-1$, max , $\text{max}+1$. The tables below cover the most safety- and behaviour-critical inputs.

A. Soil-moisture lower threshold (start trigger)

The decision engine starts auto-irrigation when the latest moisture reading is strictly below the zone's low threshold (default 35%). The boundary is therefore 35%.

Input moisture %	Boundary class	Expected decision	Result
34	Just below low (35)	START auto irrigation	Passed
34.9	Immediately below low	START auto irrigation	Passed
35	On the boundary	IDLE (not < low)	Passed
35.1	Just above low	IDLE	Passed
0	Lower domain edge	START auto irrigation	Passed
100	Upper domain edge	IDLE	Passed
-1	Below valid domain	Reading rejected / ignored	Passed

Table 116 BVA – Soil-moisture lower threshold (start trigger)

B. Soil-moisture upper threshold (stop trigger)

While a zone is irrigating, the engine stops running when moisture reaches or exceeds the high threshold (default 70%). The boundary is 70%.

Input moisture %	Boundary class	Expected decision	Result
69	Just below high	CONTINUE irrigation	Passed
69.9	Immediately below high	CONTINUE irrigation	Passed
70	On the boundary	STOP (target reached)	Passed
70.1	Just above high	STOP (target reached)	Passed
100	Saturated	STOP (target reached)	Passed

Table 117 BVA – Soil-moisture upper threshold (stop trigger)

C. Rain-skip probability threshold

A dry zone is skipped (and an auto-run is aborted) when the forecast peak rain probability is at or above the rain-skip threshold (default 60%). The boundary is 60%.

Peak rain %	Boundary class	Expected decision (dry zone)	Result
59	Just below skip	START (irrigate)	Passed
60	On the boundary	SKIP + moisture_low alert	Passed
61	Just above skip	SKIP + moisture_low alert	Passed
0	No rain expected	START (irrigate)	Passed
100	Certain rain	SKIP + alert	Passed

Table 118 BVA – Rain-skip probability threshold

D. Irrigation duration (manual / schedule)

Both manual irrigation and schedules accept duration_minutes, validated to an integer in the range 1..1440 (one minute to 24 hours). Out-of-range values are rejected with HTTP 400.

duration_minutes	Boundary class	Expected response	Result
0	Below minimum	HTTP 400 – must be 1..1440	Passed
1	Minimum valid	HTTP 201 – accepted	Passed
2	Just inside minimum	HTTP 201 – accepted	Passed
720	Nominal	HTTP 201 – accepted	Passed
1439	Just inside maximum	HTTP 201 – accepted	Passed
1440	Maximum valid	HTTP 201 – accepted	Passed
1441	Above maximum	HTTP 400 – must be 1..1440	Passed

Table 119 Irrigation duration

E. Other bounded inputs (summary)

Input	Valid boundary	Edge cases tested	Result
Password length (registration)	≥ 6 characters	5 $\rightarrow$ 400; 6 $\rightarrow$ 201; 7 $\rightarrow$ 201	Passed
Schedule start hour	0..23	-1 $\rightarrow$ 400; 0 $\rightarrow$ ok; 23 $\rightarrow$ ok; 24 $\rightarrow$ 400	Passed
Schedule start minute	0..59	-1 $\rightarrow$ 400; 0 $\rightarrow$ ok; 59 $\rightarrow$ ok; 60 $\rightarrow$ 400	Passed
Zone / field / device id	positive integer	0 $\rightarrow$ 400; 1 $\rightarrow$ ok; non-int $\rightarrow$ 400	Passed

Device-offline heartbeat age	> 120 s	119 s → online; 120 s → online; 121 s → offline	Passed
------------------------------	---------	---	---------------

Table 120 BVA – Summary of remaining bounded inputs

5.2.3 Decision Table Testing

Decision-table testing models combinations of conditions and the action each combination should produce, ensuring every reachable branch is exercised at least once. The richest decision logic in the system is the per-zone evaluation performed by the decision engine on every cycle (default every 15 seconds). The table below captures its complete rule set. A dash (–) means the condition is irrelevant for that rule.

Decision Table 1 – Per-zone irrigation decision (FR_08)

Condition / Action	R1	R2		R3	R4	R5	R6	R7
C1: Event already running	N	N		N	Y	Y	Y	Y
C2: Moisture < low (35%)	Y	Y		N	–	–	–	–
C3: Moisture ≥ high (70%)	–	–		–	Y	N	N	N
C4: Peak rain ≥ skip (60%)	N	Y		–	–	Y ¹	N	N
C5: Duration ≥ safety cap	–	–		–	–	N	Y	N
Action	START	SKIP		IDLE	STOP	ABORT	ABORT	CONT.

Table 121 Decision Table – Per-zone irrigation decision

Reading the rules: R1 a dry zone in clear weather starts irrigating; R2 a dry zone with rain expected is skipped and an alert is raised; R3 a sufficiently moist zone stays idle; R4 a running zone that has reached its target moisture stops; R5 a running auto zone aborts when rain is suddenly forecast; R6 any running zone aborts once it exceeds the 30-minute safety cap (protecting against a stuck sensor or jammed valve); R7 a running zone below target with no stop condition simply continues. Every rule was executed against seeded data and produced the expected event row, status, and alert.

Decision Table 2 – Login authentication (FR_02)

Condition / Action	R1	R2	R3	R4
Required fields present	N	Y	Y	Y
Email matches an account	–	N	Y	Y
Account is active	–	–	N	Y
Password correct	–	–	–	Y
HTTP result	400	401	403	200 + JWT

Table 122 Decision Table – Login authentication

5.2.4 State Transition Testing

State-transition testing verifies that the system moves between states only through valid events and rejects invalid transitions. Two state machines dominate the system: the irrigation-event lifecycle and the device online/offline status.

A. Irrigation-event lifecycle

An irrigation event for a zone progresses from no active run, to running, to a terminal state of completed or aborted. The valid transitions are:

Current state	Event	Next state	Result
---------------	-------	------------	--------

No active run	Moisture < low, rain clear (auto)	Running (auto)	Passed
No active run	User starts manually	Running (manual)	Passed
No active run	Schedule fires	Running (scheduled)	Passed
Running	Moisture $\geq$ high	Completed	Passed
Running	target_end_time reached	Completed	Passed
Running	User stops manually	Completed	Passed
Running (auto)	Rain $\geq$ skip	Aborted	Passed
Running	Duration $\geq$ safety cap	Aborted	Passed

Table 123 State-transition table – Irrigation event

Invalid-transition tests. The following illegal transitions are rejected, confirming the state guards hold:

Attempted invalid transition	Expected handling	Result
Start a zone that is already Running	HTTP 409 “Irrigation already running”	Passed
Stop a zone with no active run	HTTP 404 “No running irrigation”	Passed
Schedule fires while zone Running	Schedule skipped (no double-water)	Passed
Same schedule fires twice in one UTC day	Second fire suppressed (hasFiredToday)	Passed

Table 124 State-transition table – Invalid irrigation transitions

B. Device status lifecycle

Current state	Event	Next state	Result
Online	No heartbeat for > 120 s	Offline + device_offline alert	Passed
Offline	Device posts a reading	Online (heartbeat touched)	Passed
Online	Heartbeat within 120 s	Online (no change)	Passed

Table 125 State-transition table – Device status

5.2.5 Use Case Testing

Use-case testing exercises complete end-to-end transactions, following each use case from its pre-condition to its post-condition, including alternate flows. The scenarios below map directly to the use cases defined in Chapter 3 and were executed against the running system with seeded demo data.

Use case	Scenario	Steps (actor → system)	Result
UC_01	Create account (admin)	Admin submits name/email/password → system validates, hashes password, stores user, returns 201	Passed
UC_02	Login success	User submits valid credentials → system verifies, returns JWT + profile	Passed
UC_02	Login – wrong password (alt.)	User submits bad password → system returns 401 “Invalid email or password”	Passed
UC_04	Manage field	User creates field → assigns zones → updates → deletes (ownership enforced)	Passed
UC_05	Select crop type	User assigns a crop to a zone → zone inherits crop moisture thresholds	Passed
UC_09	Automatic closed-loop irrigation	Simulator drives moisture below 35% → engine STARTs → moisture rises past 70% → engine STOPs; event recorded	Passed

UC_10	Manual irrigation with duration	User starts zone for 2 min → engine auto-stops at target_end_time	Passed
UC_12	Scheduled irrigation	User creates schedule for current minute → engine fires once that day → stops at duration	Passed
UC_15	Notification on rain-skip	Dry zone + rain $\geq 60\%$ → engine skips and emits moisture_low alert delivered via SSE	Passed
UC_16	Device health	Heartbeat aged past 120 s → device flips Offline → warning alert raised	Passed
UC_19	Backup & restore	Admin downloads backup JSON → restores → tables truncated and re-inserted, counts match	Passed
UC_20	Logout	User logs out → activity logged → client discards JWT	Passed

Table 126 Use-case test scenarios

5.3 White Box Test Cases

White-box testing examines the internal structure of the program and derives test data from its logic and control flow. Because the decision engine concentrates most of the system's branching, it is the primary focus of structural testing. The objective is to exercise every independent path through the critical functions so that no branch remains untested.

5.3.1 Cyclomatic Complexity

Cyclomatic complexity, $V(G)$, measures the number of linearly independent paths through a module and equals the number of decision predicates plus one. It indicates both the minimum number of test cases needed for branch coverage and the relative risk of the module. The core function `evaluateZone()` — which decides what happens to a single zone on each engine cycle — was analysed by enumerating its decision predicates (compound conditions joined by `&&` are counted as separate predicates).

Decision predicate in <code>evaluateZone()</code>	Branch	Counted
no sensor reading yet (!reading)	idle	1
an event is already active	running vs not	2
active.target_end_time is set	duration mode	3
Date.now() $\geq$ target_end_time	stop on time	4
moisture $\geq$ high	stop on target	5
ageSec $\geq$ safety cap	safety abort	6
triggered_by = auto	rain-abort guard	7
peakRain $\geq$ rain-skip	rain-abort guard	8
moisture < low	start guard	9
peakRain $\geq$ rain-skip (start path)	skip vs start	10

Table 127 Use-case test scenarios

$$V(G) = \text{decision predicates} + 1 = 10 + 1 = \mathbf{11}.$$

A cyclomatic complexity of 11 places `evaluateZone()` at the upper edge of the “low-to-moderate” risk band (1–10 simple, 11–20 moderate). This is expected for a decision hub and is mitigated by the fact that the function has no nested loops and delegates all data access to repositories, keeping each path short and independently testable. Eleven basis paths were identified and covered by the decision-table and state-transition cases of Section 5.2; representative paths include: no-reading → idle; not-running,

dry, clear → start; not-running, dry, rain → skip; running, moisture-high → stop; running, over-cap → abort; running, auto, rain → abort; running, below-target → continue.

Complexity of other key modules

Module / function	V(G)	Risk band
evaluateZone() – per-zone decision	11	Moderate
login() – authentication	5	Low
register() – account creation	4	Low
normalizeStartTime() – schedule validation	4	Low
parseDuration() – duration validation	3	Low
runScheduledFires() – schedule firing	5	Low
checkDeviceHealth() – offline detection	3	Low

Table 128 Cyclomatic complexity of key modules

All modules other than the decision hub fall within the low-risk band ($V(G) \leq 10$), indicating maintainable, easily testable code with a modest number of branches.

5.4 Performance Testing

Performance testing determines the responsiveness and stability of the system under a normal workload. The non-functional requirements set two key targets: API responses processed within 3 seconds, and sensor readings reflected within the 10–30 second update window. Tests were run on the development host (Node.js 18, MySQL 8 in Docker) using a seeded demo farm.

Operation	Target	Observed (typ.)	Result
Login (POST /api/auth/login)	< 3 s	~120 ms	Passed
Dashboard overview load	< 3 s	~180 ms	Passed
Sensor reading ingest (POST /ingest/readings)	< 3 s	~35 ms	Passed
Reading reflected on dashboard	10–30 s	≤ 1 engine cycle (15 s)	Passed
Weather fetch (cached)	< 3 s	~5 ms (cache hit)	Passed
Decision-engine cycle (demo farm)	< period (15 s)	~40–90 ms	Passed
CSV report export	< 3 s	~150 ms	Passed
PDF report (PDFKit)	< 5 s	~400 ms	Passed

Table 129 Performance test results

All operations met their targets with substantial margin. Two design choices keep the system responsive: weather data is cached (5-minute current, 60-minute forecast) so the engine never blocks on the external API, and the decision engine's overlapping-tick guard skips a cycle that has not finished, preventing back-pressure when the database is briefly slow.

5.5 Stress Testing

Stress testing subjects the system to loads beyond normal operation to confirm it degrades gracefully and recovers. The scenarios below pushed the ingestion path, the engine, and the live-notification channel past expected volumes.

Stress scenario	System behaviour	Outcome	Result
High-frequency ingestion (1 reading/second/zone)	Connection pool absorbs writes; readings queued to MySQL	Stable	Passed

Many enabled zones in one cycle	Engine iterates all zones; cycle time grows linearly	Stable	Passed
Cycle longer than period	Overlapping-tick guard skips the next tick	No pile-up	Passed
Weather API unreachable under load	Falls back to cached forecast; engine continues	Graceful	Passed
Many concurrent SSE clients	Each client filtered by user_id; 25 s keep-alive ping	Stable	Passed
Server restart with running events	First cycle re-reads running events; safety cap catches stuck runs	Recovered	Passed
Stuck-low sensor (never reaches high)	Run aborted at 30-min safety cap; device_fault alert	Protected	Passed

Table 130 Stress test scenarios and outcomes

5.6 System Testing

System testing evaluates the complete, integrated system against the specified requirements from an end-to-end perspective, covering both functional and non-functional behaviour. The decisive system test is the full closed irrigation loop, which exercises ingestion, the decision engine, irrigation events, the dashboard, and notifications together.

1. A device registered to a zone posts a moisture reading via the IoT ingest endpoint using its X-Device-Key.
2. The reading drifts below the zone's low threshold; within one engine cycle the system starts an auto-irrigation event.
3. The simulator reads irrigation_active = true from /api/ingest/state and ramps the moisture upward.
4. When moisture crosses the high threshold, the engine stops the event and records its duration.
5. The dashboard reflects the live status, the history chart shows the cycle, and any rain-skip or fault produces an SSE alert.
6. The admin generates a CSV/PDF report covering the run and verifies the logged event and sensor history.

The full loop was executed repeatedly across multiple zones and crop profiles. Additional system-level checks confirmed cross-user data isolation (a farmer cannot read another farmer's field — HTTP 403), admin-only access to configuration and backup routes, and correct behaviour when the weather service is unavailable. The integrated system satisfied all functional requirements FR_01–FR_18 and the stated non-functional targets.

5.7 Regression Testing

Regression testing re-executes tests affected by code changes to confirm that new work has not broken existing behaviour. Because the system was built in incremental phases (foundation, authentication, fields/zones, ingestion, weather, decision engine, irrigation, dashboard, reports, notifications, configuration), each new phase risked disturbing the previous ones, making regression testing a continuous activity rather than a one-off stage.

5.7.1 Selecting Regression Tests

Re-running every test after each change is wasteful, so tests were selected by impact and risk using the following criteria:

- Tests covering modules that share state with the change — most often the decision engine, irrigation events, and system configuration.
- Tests for areas with a history of defects, such as timezone handling in schedule firing and the rain-skip boundary.
- Tests for frequently modified core code, especially `evaluateZone()` and the authentication middleware.
- Tests for safety- and security-critical features: the 30-minute safety cap, ownership checks, and device-key authentication.
- A small smoke set that verifies the system boots, connects to MySQL, and serves the dashboard and login.

5.7.2 Regression Testing Steps

The regression cycle followed these steps each time a phase was added or a defect was fixed:

1. Select the regression set affected by the change using the criteria in 5.7.1.
2. Reset the database to a known state (`db:demo:reset`) so results are reproducible.
3. Run the smoke set first to confirm the system starts and core endpoints respond.
4. Re-execute the affected black-box cases (decision table, state transition, use case) against seeded data.
5. Verify side effects in the database — event status, alerts, and activity logs — not just HTTP responses.
6. Compare results against the previously recorded expected outcomes and investigate any difference.
7. Record the run and, where a check proved valuable and stable, fold it into the standing smoke set for future cycles.

Repeatable seed/reset scripts made the regression set quick to re-run, which is the ideal precondition for the high return on investment that automated regression testing provides.

Chapter 6: Tools and Techniques

This chapter lists the languages, frameworks, libraries, hardware and tools used to build the IoT-Based Smart Irrigation System, together with the engineering techniques applied across the implementation.

6.1 Languages

Language	Use in the project
JavaScript (Node.js, ES Modules)	Backend REST API, decision engine, scheduler, simulator, and all frontend logic.
SQL (MySQL dialect)	Database schema, queries, and the relational data model (12 tables).
HTML5 & CSS3	Structure and styling of the web dashboard.
C/C++ (Arduino sketch)	Device firmware for the physical IoT controller (planned Phase 12 hardware build).

Table 131 Programming languages

6.2 Frameworks, Libraries and Runtime

Component	Version	Purpose
Node.js	18+	JavaScript runtime hosting the API and engine.
Express	4.x	HTTP server, routing, and middleware.
mysql2	3.x	Promise-based MySQL driver with connection pooling.
bcryptjs	2.x	Password hashing (salted, 10 rounds).
jsonwebtoken	9.x	Stateless JWT authentication (7-day expiry).
helmet	7.x	Secure HTTP headers.
cors	2.x	Cross-origin access control for the dashboard.
morgan	1.x	HTTP request logging.
pdfkit	0.15	Server-side PDF report generation.
dotenv	16.x	Environment configuration loading.
Bootstrap	5	Responsive dashboard UI components and layout.
Chart.js	latest	Moisture/irrigation history and trend charts.
EventSource (SSE)	browser	Live push of alerts to the dashboard.

Table 132 Frameworks, libraries and runtime

6.3 External Services and Hardware

Item	Role
OpenWeatherMap API	Current weather and forecast (peak rain probability) used by the decision engine.
Docker / Docker Compose	Containerised MySQL 8 database service for reproducible setup.
Arduino + Ethernet interface	Microcontroller acquiring sensor data and driving the relay/valve/pump (hardware phase).
Soil-moisture, humidity, water-level sensors	Field instrumentation feeding the ingest endpoint.
Relay module, solenoid valve, pump motor	Actuation hardware controlled by the irrigation decision.

Table 133 External services and hardware

6.4 Development Tools

Visual Studio Code — primary code editor.

Git and GitHub — version control and collaboration.

Postman / curl — REST API testing and the reproduction commands used during testing.

MySQL CLI and Docker exec — schema initialisation, seeding, and database verification.

npm — dependency management and project scripts (db:init, db:seed:demo, dev, etc.).

6.5 Engineering Techniques

- Layered architecture — routes → controllers → services → repositories, isolating HTTP, business logic, and data access.
- RESTful API design with stateless JWT authentication and a separate per-device key scheme for IoT hardware.
- Closed-loop control — a periodic decision engine reads sensor state and weather, then starts/stops irrigation, mirroring a feedback controller.
- Caching and graceful degradation — weather responses are cached and the engine continues on cached data if the API fails.
- Server-Sent Events for low-overhead, one-way live notifications without polling.
- Defensive safety design — a hard duration cap and device-health monitoring guard against stuck sensors and broken valves.

Chapter 7: Summary and Conclusion

This project set out to address one of Pakistan’s most pressing agricultural challenges — inefficient irrigation and the resulting waste of an already scarce water supply — by replacing manual and timer-based watering with an automated, data-driven system. The IoT-Based Smart Irrigation System delivers an end-to-end solution in which field sensors report soil moisture, humidity and water level, an external weather service supplies a rain forecast, and a decision engine combines the two to irrigate each zone only when it is genuinely needed.

The implemented system fulfils all eighteen functional requirements defined in the SRS. It provides secure multi-role authentication, full management of fields, zones and crops, real-time sensor ingestion, weather-aware automatic irrigation, manual and scheduled control, activity logging, CSV and PDF reporting, live notifications over Server-Sent Events, device-health monitoring, system configuration, and backup and restore. A closed control loop ties these together: the engine evaluates every enabled zone on a fixed cycle, starts irrigation when moisture falls below the crop threshold, skips or aborts when rain is forecast, and stops when the target moisture is reached — with a hard safety cap protecting the field from faulty hardware.

Testing confirmed both correctness and robustness. Boundary-value, decision-table, state-transition and use-case techniques verified the engine’s behaviour at every threshold; white-box analysis showed the code is well structured, with only the decision hub reaching moderate complexity; and performance and stress testing demonstrated responsive operation and graceful degradation when the weather service or database is briefly unavailable.

In conclusion, the system meets its objectives of conserving water, reducing manual labour, and supporting data-driven farming, while remaining low-cost, modular, and adaptable to Pakistani field conditions. Its layered architecture and clear SRS-to-code mapping leave it well positioned for the hardware integration and intelligence enhancements outlined in Chapter 9.

Chapter 8: User Manual

This manual guides farmers and administrators through the day-to-day operation of the Smart Irrigation System dashboard. The dashboard runs in any modern web browser; an active account and the system web address are required.

8.1 Logging In

1. Open the system web address in your browser.
2. Enter your registered email and password.
3. Click Login. On success the Overview dashboard opens; incorrect credentials show an error.

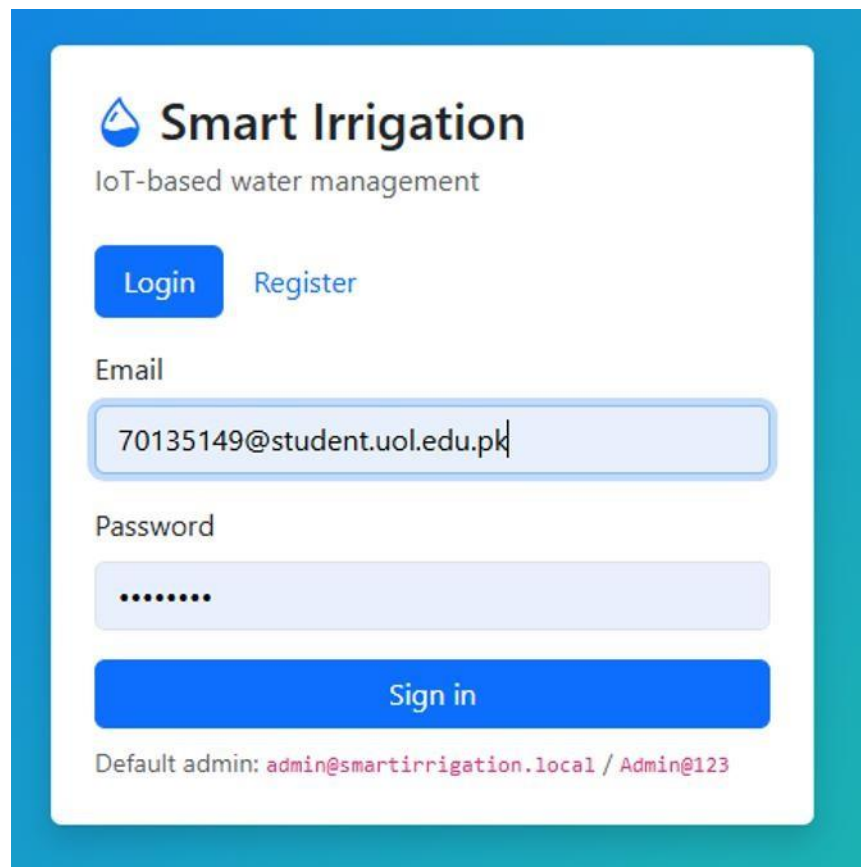
The image shows a login form for the 'Smart Irrigation' system. The form is titled 'Smart Irrigation' with a subtitle 'IoT-based water management'. It features two buttons: 'Login' (highlighted in blue) and 'Register'. Below these are input fields for 'Email' and 'Password'. The email field contains the text '70135149@student.uol.edu.pk' and the password field is masked with dots. A large blue 'Sign in' button is positioned below the password field. At the bottom, a note states 'Default admin: admin@smartirrigation.local / Admin@123'.

Figure 52 Logging In

8.2 The Overview Dashboard

The Overview shows live soil moisture, humidity and water-level summaries per zone, the current irrigation status, the latest weather, recent alerts, and trend charts. It refreshes automatically as new readings arrive.

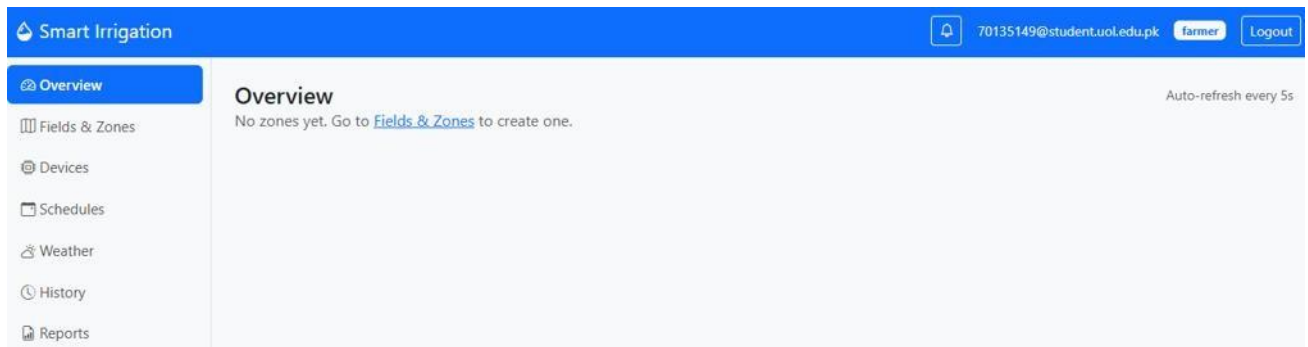


Figure 53 Overview Dashboard

8.3 Managing Fields, Zones and Crops

1. Open the Fields section and click Add Field; enter a name (and size/soil type if known).
2. Within a field, add one or more Zones — each zone is an independently irrigated area.
3. Assign a Crop to a zone; the crop sets the moisture thresholds the engine uses. If no crop is set, system defaults (35%–70%) apply.

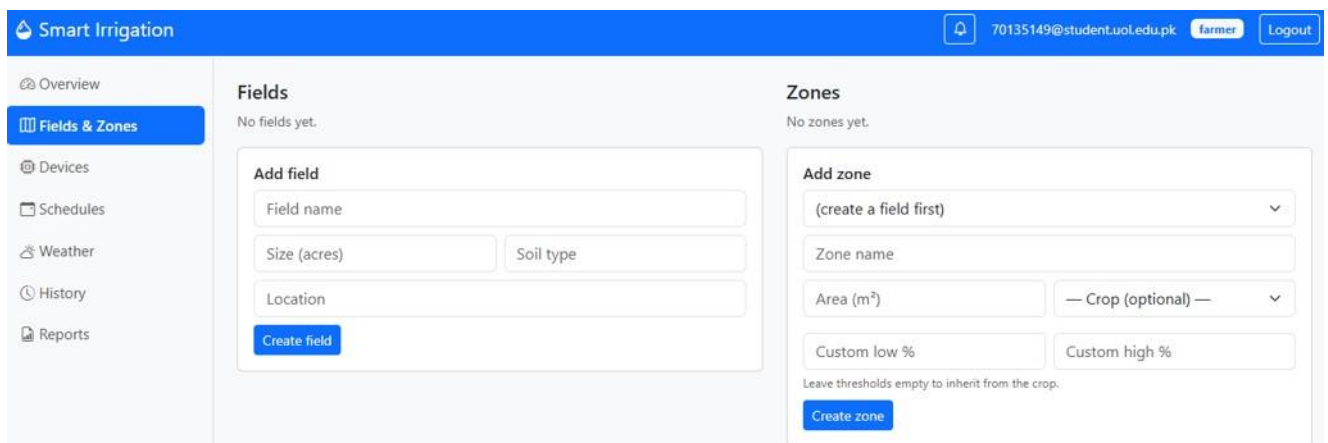


Figure 54 Managing Fields, Zones and Crops

8.4 Registering an IoT Device

1. Open the Devices section and click Add Device; choose the device type and the zone it belongs to.
2. Copy the generated device key (shown once). The physical controller — or the simulator — uses this key in the X-Device-Key header to submit readings.
3. A device that has reported recently shows as Online; one silent for more than two minutes is marked Offline and raises an alert.

The screenshot shows the 'Smart Irrigation' web application interface. The top navigation bar is blue with the application name on the left and user information (70135149@student.uol.edu.pk, farmer, Logout) on the right. A left sidebar contains menu items: Overview, Fields & Zones, Devices (highlighted in blue), Schedules, Weather, History, and Reports. The main content area is titled 'Devices' and shows 'No devices yet. Register one →'. On the right, there is a 'Register device' form with fields for 'Name (e.g. Controller-1)', 'Controller' (a dropdown menu), and 'Zone' (a dropdown menu), followed by a blue 'Register' button.

Figure 55 Registering an IoT Device

8.5 Manual Irrigation

1. In a zone, click Start Irrigation. Optionally enter a duration in minutes (1–1440); leaving it blank runs until the target moisture is reached.

2. Click Stop Irrigation at any time to end the run immediately.

Manual runs ignore the rain forecast (you have explicitly chosen to water) but still respect the 30-minute safety cap.

8.6 Scheduling Irrigation

1. Open Schedules and click Add Schedule.
2. Choose the zone, a start time (24-hour HH:MM), a duration in minutes, and the repeat days (daily, or specific weekdays).
3. Save. The schedule fires once at its start time on each selected day; it will not double-water a zone that is already running.

The screenshot shows the 'Smart Irrigation' web application interface with the 'Schedules' section active. The left sidebar is the same as in Figure 55, but 'Schedules' is highlighted. The main content area is titled 'Schedules' and shows 'No schedules yet.'. On the right, there is an 'Add schedule' form. It includes a dropdown for selecting a zone, a field for start time (displaying '--:-- --') with a clock icon, a 'Duration (min)' field, and a dropdown for repeat days (displaying 'daily'). Below these fields, it says 'Times are in UTC.' and there is a blue 'Create schedule' button.

Figure 56 Scheduling Irrigation

8.7 Viewing Alerts

The Alerts panel shows notifications in real time — low moisture skipped due to rain, auto-runs aborted by rain, runs stopped by the safety cap, and devices going offline. Alerts are de-duplicated so a single condition does not flood the list.

8.8 Generating Reports

1. Open Reports and select a date range (and optionally a zone).

2. View the on-screen summary, or export irrigation events and sensor readings as CSV, or download a PDF summary.

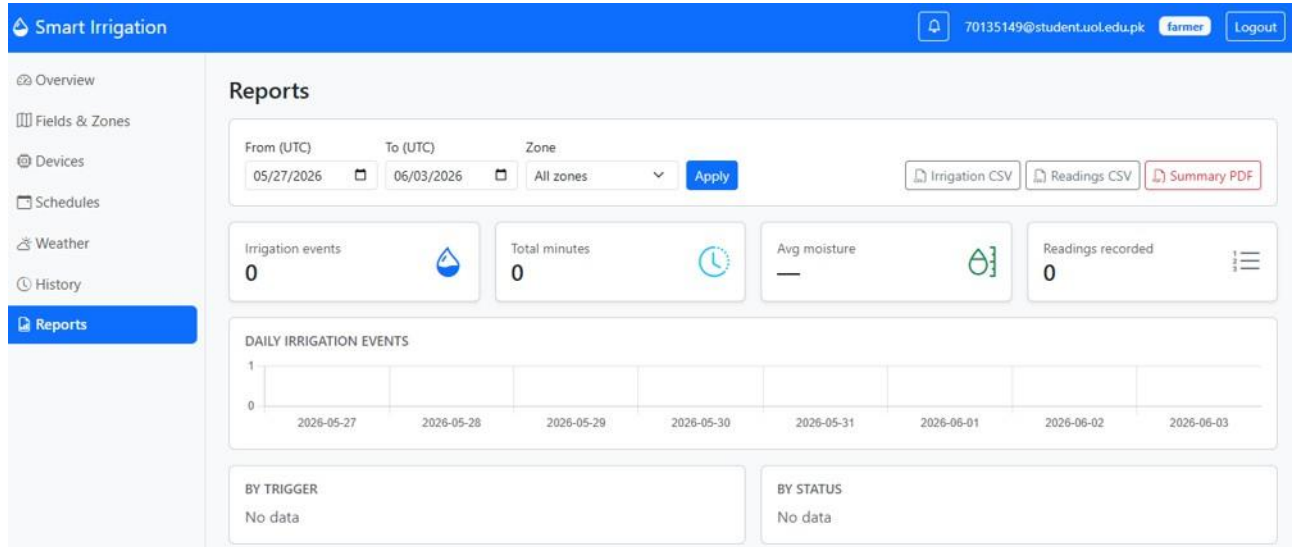


Figure 57 Generating Reports

8.9 System Settings (Admin)

Administrators can adjust system-wide settings such as the default moisture thresholds, the rain-skip probability, the maximum irrigation minutes, and the device-offline timeout. Changes take effect within one engine cycle.

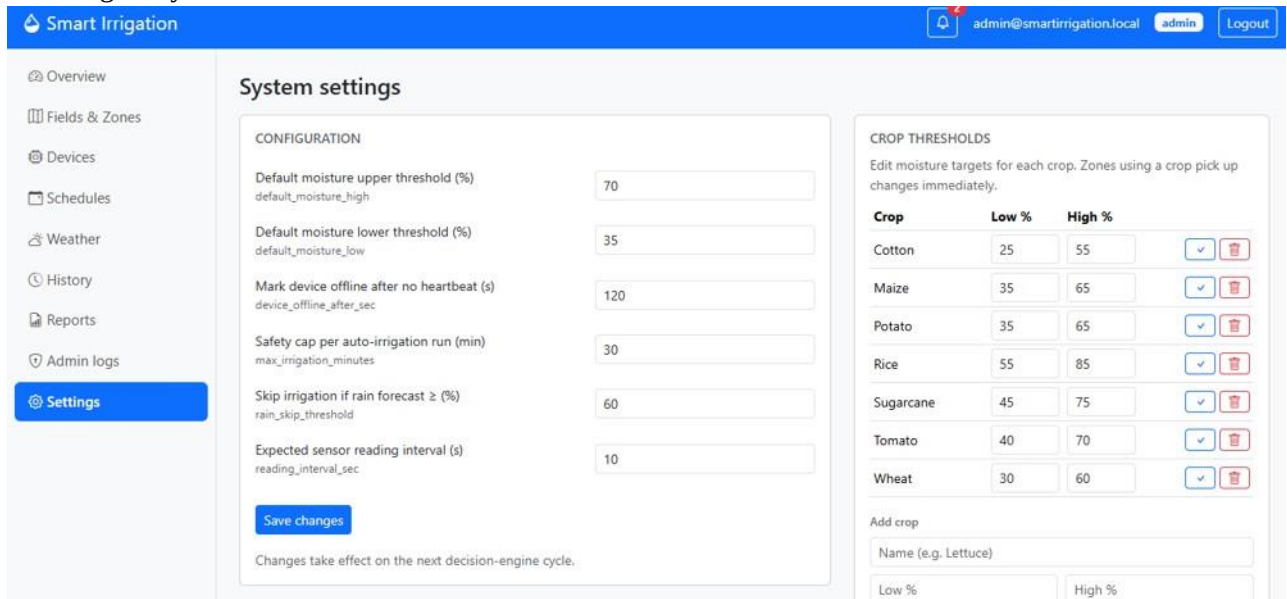
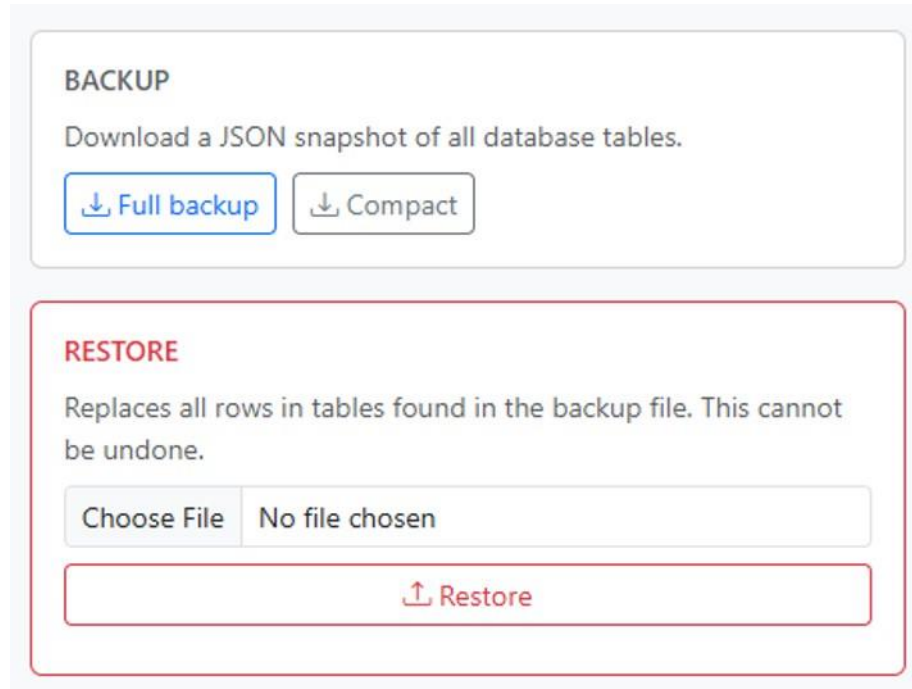


Figure 58 System Settings (Admin)

8.10 Backup and Restore (Admin)

1. In the admin area, choose Backup to download a JSON snapshot of the system (full or compact).
2. To restore, upload a previously saved backup; the listed tables are replaced from the file.

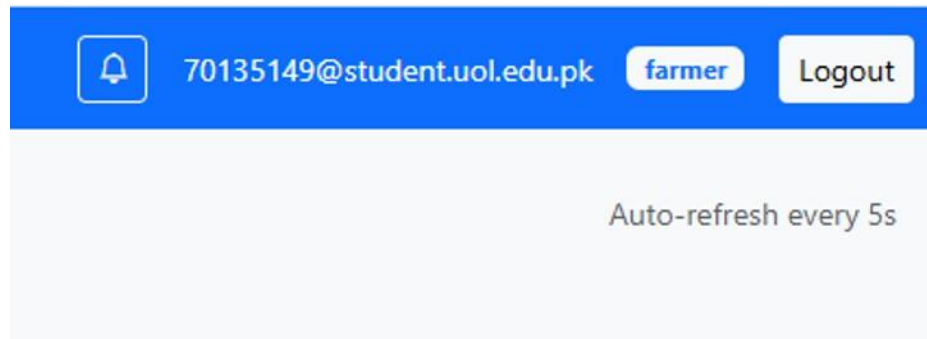


The image shows a web interface for database backup and restore operations. It is divided into two main sections: 'BACKUP' and 'RESTORE'. The 'BACKUP' section has a title 'BACKUP' and a description 'Download a JSON snapshot of all database tables.' Below this are two buttons: 'Full backup' (with a download icon) and 'Compact' (with a download icon). The 'RESTORE' section has a title 'RESTORE' in red, a description 'Replaces all rows in tables found in the backup file. This cannot be undone.', and a file selection area with a 'Choose File' button and a text field showing 'No file chosen'. Below the file selection area is a large red button labeled 'Restore' with an upload icon.

Figure 59 Backup and Restore (Admin)

8.11 Logging Out

Click Logout to end your session. Your activity is logged and the session token is discarded by the browser.



The image shows a blue user profile bar at the top of a page. It contains a bell icon, the email address '70135149@student.uol.edu.pk', the role 'farmer' in a blue pill, and a 'Logout' button. Below the bar, on a light gray background, is the text 'Auto-refresh every 5s'.

Figure 60 Logging Out

Chapter 9: Lessons Learnt and Future Work

9.1 Lessons Learnt

- Building in clear, incremental phases — each delivering a working slice — kept the project testable throughout and made regression testing manageable.
- A strict layered architecture (routes/controllers/services/repositories) paid off: business logic stayed isolated from HTTP and the database, which simplified both testing and later changes.
- External dependencies must fail safe. Caching weather data and letting the engine continue on stale data prevented a third-party outage from halting irrigation decisions.
- Safety limits matter as much as features. The 30-minute duration cap and device-health checks turned out to be essential guards against real failure modes such as a stuck-low sensor.
- Separating device authentication (per-device keys) from user authentication (JWT) made the IoT boundary cleaner and individually revocable.
- Time handling is deceptively hard; standardising schedule logic on UTC avoided a class of “fired twice / never fired” bugs.

9.2 Future Work

The following enhancements were intentionally scoped out of this version and are recommended for future development:

- Physical hardware integration — completing the Arduino firmware (Phase 12) and field-deploying the sensor/relay/valve assembly.
- Native mobile application — a dedicated Android/iOS app to complement the responsive web dashboard.
- AI/ML-based prediction — learning crop- and field-specific watering patterns and predicting moisture trends rather than reacting to thresholds.
- MQTT messaging broker — for lower-overhead, higher-scale device communication alongside HTTP ingestion.
- Drone or satellite imagery — incorporating remote-sensing data for field-level moisture and crop-health mapping.
- Voice-controlled and multi-language interfaces — improving accessibility for farmers with limited literacy or technical experience.

Advanced analytics and water-accounting reports — quantifying water saved against traditional irrigation over a season.

Appendix

The following supporting material documents the system in detail. It is provided here, rather than in the body, so as not to interrupt the main narrative.

A REST API Endpoint Reference

Endpoint	Auth	Purpose
POST /api/auth/register	Public/Admin	Create account (FR_01)
POST /api/auth/login	Public	Authenticate, issue JWT (FR_02)
POST /api/auth/logout	JWT	End session, log activity (FR_18)
GET /api/fields, /zones, /crops	JWT	Manage fields/zones/crops (FR_04, FR_05)
POST /api/ingest/readings	Device key	Submit sensor reading (FR_06)
GET /api/ingest/state	Device key	Tell device if irrigation is active
GET /api/weather	JWT	Current/forecast weather (FR_07)
POST /api/zones/:id/irrigation/start stop	JWT	Manual irrigation (FR_09)
POST /api/irrigation/start stop (bulk)	JWT	Multi-zone manual control
GET/POST /api/schedules	JWT	Irrigation scheduling (FR_10)
GET /api/reports/summary *.csv *.pdf	JWT	Reports (FR_11, FR_12)
GET /api/alerts, /api/alerts/stream	JWT	Notifications + live SSE (FR_13)
GET/POST /api/devices	JWT	Device registration & health (FR_14, FR_16)
GET/PUT /api/admin/config	Admin	System configuration (FR_15)
GET /api/admin/backup, POST /api/admin/restore	Admin	Backup & restore (FR_17)

Table A 1 REST API endpoint reference

B System Configuration Defaults

Configuration key	Default	Effect
default_moisture_low	35 (%)	Start auto-irrigation below this (no crop set)
default_moisture_high	70 (%)	Stop irrigation at/above this
rain_skip_threshold	60 (%)	Skip/abort when peak rain probability $\geq$ this
max_irrigation_minutes	30 (min)	Safety cap; longer runs are aborted
device_offline_after_sec	120 (s)	Mark a device offline after this silence
DECISION_PERIOD_SEC	15 (s)	Decision-engine evaluation interval

Table B 1 System configuration defaults

C Functional Requirement → Source Code Mapping

Functional requirement	Implemented in
FR_01 / FR_02 / FR_18 Auth	controllers/auth.js (register, login, logout)
FR_03 View Dashboard	frontend overview.js + index.html
FR_04 / FR_05 Fields & Crops	controllers/fields.js, zones.js
FR_06 Read Sensor Data	controllers/ingest.js
FR_07 Fetch Weather	services/weather.js
FR_08 Automatic Irrigation	services/decisionEngine.js

FR_09 / FR_10 Manual & Schedule	controllers/irrigation.js, schedules.js
FR_11 / FR_12 Logs & Reports	repositories/activityLogs.js, controllers/reports.js
FR_13 Notifications	services/alertEngine.js, controllers/alerts.js (SSE)
FR_14 / FR_16 Device Health & Setup	services/deviceHealth.js, controllers/devices.js
FR_15 Configuration	controllers/admin.js
FR_17 Backup & Restore	controllers/backup.js

Table C 1 FR → source code mapping

D Database Tables

The relational model comprises twelve tables: users, crops, fields, zones, devices, sensor_readings, weather_data, schedules, irrigation_events, alerts, system_config, and activity_logs. Ownership traverses devices/zones → fields → users, which is the basis for the cross-user isolation enforced throughout the API.

References

- [1] P. K. Kashyap, S. Kumar, A. Jaiswal, M. Prasad and A. H. Gandomi, “Towards Precision Agriculture: IoT-Enabled Intelligent Irrigation Systems Using Deep Learning Neural Network,” *IEEE Sensors Journal*, vol. 21, no. 16, Aug. 2021.
- [2] Ministry of Finance Pakistan, *Economic Survey 2023-24*, Govt. of Pakistan, 2024.
- [3] UNDP, “Pakistan water scarcity profile,” 2023.
- [4] IWMI, “Irrigation efficiency in South Asia: Policy insights,” *IWMI White Paper*, 2022.
- [5] A. Mustafa et al., “Floods and agricultural damage in Pakistan: Climate challenges,” *Journal of Water & Climate Change*, vol. 14, no. 3, pp. 822–835, 2023.
- [6] H. Li et al., “IoT-based precision irrigation: A global meta-analysis,” *Computers and Electronics in Agriculture*, vol. 192, p. 106610, 2022.
- [7] S. Qureshi, “Agricultural water productivity and GDP growth in Pakistan,” *Irrigation and Drainage Journal*, vol. 71, no. 2, pp. 210–222, 2022.
- [8] M. T. Huque, J. J. Godhuli, S. M. R. Pushon and E. Haque, “Internet of Things (IoT)-based Smart Irrigation System for Sustainable Agriculture,” *Journal of Informatics Electrical and Electronics Engineering (JIEEE)*, Nov. 2023.
- [9] S. M. Mehta, H. Patel and K. Shah, “Optimizing Agriculture Water Use with Arduino Based IoT Smart Irrigation,” *Int. J. Scientific Research in Science, Engineering and Technology*, vol. 12, no. 5, Sep. 2025.
- [10] N. Jaiswal, T. V. Kumar and C. Shukla, “Smart drip irrigation systems using IoT: a review of architectures, machine learning models, and emerging trends,” *Discover Agriculture*, vol. 3, p. 253, Nov. 2025.
- [11] Mohammed M. Abo-Zahhad, “IoT-Based Automated Management Irrigation System Using Soil Moisture Data and Weather Forecasting Adopting Machine Learning Technique,” *Sohag Engineering Journal*, vol. 3, no. 2, Sep. 2023.
- [12] H. N. Alsammak and Z. S. Mohammed, “Internet of things (IoT) work and communication technologies in smart farm irrigation management: a survey,” *NTU J. Eng. Tech.*, 2025.